ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT
KHOA CÔNG NGHỆ SỐ

BÀI GIẢNG

THIẾT KẾ WEB

Tài liệu lưu hành nội bộ - dành cho sinh viên

Ngành Công nghệ thông tin



Biên soạn: ThS. Trần Bửu Dung

Khoa Công nghệ số
Bộ môn Công nghệ thông tin

Đà Nẵng – Tháng 09/2022

THIẾT KẾ WEB

- **Mã học phần: KD5505222 - Số tín chỉ: 2 (LT)**
- **Mục tiêu của học phần**
  - Kiến thức: Hiểu được cấu trúc của 1 trang web và cách thức hoạt động của 1 website trên các trình duyệt; nắm vững các nguyên tắc và quy trình thiết kế giao diện trang web.
  - Kỹ năng: Có khả năng phân tích, tư duy, sáng tạo để thiết kế giao diện web đẹp mắt; có kỹ năng sử dụng các ngôn ngữ HTML, CSS, Javascript để xây dựng hoàn chỉnh trang web; sử dụng các thư viện hỗ trợ để lập trình; vận dụng thành thạo kỹ năng lập kế hoạch và viết báo cáo; thích nghi với xu hướng và sự thay đổi của công nghệ.
  - Mức tự chủ và trách nhiệm: Sinh viên có thái độ, động cơ học tập rõ ràng, từ đó có niềm đam mê và thích thú với thiết kế, kết quả là xây dựng được các website áp dụng trong thực tiễn.
- **Mô tả tóm tắt về học phần**
  - Học phần trang bị kiến thức và kỹ năng thiết kế website tĩnh, làm cơ sở cho môn lập trình web nâng cao sau này.
  - Sau khi học xong, sinh viên có thể sử dụng phần mềm Photoshop để tự thiết kế giao diện web (layout) và áp dụng thành thạo ngôn ngữ HTML, CSS, Javascript để xây dựng hoàn chỉnh trang web cho phía máy khách.

Học phần học trước: Lập trình cơ bản với C

Hoàn thành biên soạn ngày 01/09/2022

Khoa Công nghệ số phê duyệt ngày 15/09/2022

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU CHUNG 1

1.1. Các thuật ngữ cơ bản 1

1.2. Mô hình 3 lớp trong thiết kế web 3

1.3. Qui trình xây dựng trang web 3

BÀI TẬP CHƯƠNG 1 4

Bài tập 1 4

Bài tập 2 4

Bài tập 3 4

CHƯƠNG 2. CÔNG CỤ HỖ TRỢ THIẾT KẾ WEB 5

2.1. Các khái niệm cơ bản 5

2.1.1. Hệ màu 5

2.1.2. Định dạng ảnh 5

2.1.3. Thuật ngữ cơ bản 5

2.1.4. Phần mềm thiết kế 6

2.2. Phần mềm Adobe Photoshop 6

2.2.1. Giao diện phần mềm Photoshop 6

2.2.1.2. Thanh trình đơn 6

2.2.1.3. Thanh công cụ 7

2.2.1.4. Thanh tùy chọn 7

2.2.2. Chế độ màn hình 8

2.2.3. Thiết lập cơ bản 8

2.2.4. Thao tác trên file 8

2.2.5. Công cụ thường dùng 8

2.2.6. Trích xuất ảnh cho web 9

2.3. Phần mềm hỗ trợ biên soạn mã nguồn 9

2.3.1. IDE 9

2.3.2. Code Editor 9

2.3.3. Phần mềm Visual Studio Code 10

2.3.3.1. Giới thiệu phần mềm VS Code 10

2.3.3.2. Phần mở rộng trong phần mềm VS Code 10

2.3.3.3. Giao diện phần mềm VS Code....10
2.3.3.4. Các phím tắt....11
BÀI TẬP CHƯƠNG 2....13
Bài tập 1....13
Bài tập 2....14
Bài tập 3....14
Bài tập 4....15
Bài tập 5....15
Bài tập 6....15
CHƯƠNG 3. NGÔN NGỮ HTML....19
3.1. Tổng quan về HTML....19
3.2. Cấu trúc tài liệu HTML....19
3.2.1. Khai báo Doctype....20
3.2.2. Thẻ <html>....20
3.2.3. Thẻ <head>....20
3.2.4. Thẻ <body>....21
3.2.5. Chú thích trong HTML....21
3.3. Các loại thẻ HTML....21
3.3.1. Khái niệm thẻ....21
3.3.2. Phân loại thẻ....22
3.3.2.1. Phân loại theo đặc điểm....22
3.3.2.2. Phân loại theo chức năng....22
3.4. Các kí tự đặc biệt trong HTML....22
3.5. Thẻ tạo bố cục....23
3.6. Thẻ định dạng....24
3.7. Thẻ tạo nội dung....24
3.7.1. Thẻ tiêu đề....24
3.7.2. Thẻ tạo văn bản....24
3.7.3. Thẻ hình ảnh....25
3.7.4. Thẻ liên kết....26
3.7.5. Thẻ danh sách....26
3.7.6. Thẻ tạo bảng....27

3.7.7. Thẻ đa phương tiện 28
3.7.8. Thẻ tạo khung 29
3.7.9. Thẻ tạo biểu mẫu 30
BÀI TẬP CHƯƠNG 3 38
Bài tập 1 38
Bài tập 2 38
Bài tập 3 38
Bài tập 4 38
Bài tập 5 38
Bài tập 6 38
Bài tập 7 39
Bài tập 8 39
CHƯƠNG 4. NGÔN NGỮ CSS 41
4.1. Tổng quan CSS 41
4.2. Khai báo CSS 42
4.2.1. Nội dòng 42
4.2.2. Nội tuyến 42
4.2.3. Ngoại tuyến 42
4.2.4. Độ ưu tiên khai báo 42
4.3. Đơn vị đo và màu sắc 42
4.3.1. Đơn vị đo 42
4.3.2. Màu sắc 43
4.4. Bộ chọn – Selectors 44
4.4.1. Khái niệm bộ chọn 44
4.4.2. Khai báo bộ chọn 44
4.4.3. Các loại bộ chọn 44
4.4.3.1. Simple selectors 44
4.4.3.2. Combinators selectors 44
4.4.3.3. Pseudo-elements selectors 44
4.4.3.4. Pseudo-class selectors 44
4.4.3.5. Attribute selectors 45
4.4.3.6. Độ ưu tiên của các loại bộ chọn 45

4.5. Mô hình hộp 45
4.5.1. Khái niệm 45
4.5.2. Các thuộc tính trong mô hình hộp 46
4.5.3. Một số kỹ thuật trong mô hình hộp 47
4.6. Các thuộc tính thường dùng 48
4.6.1. Định dạng cho text 48
4.6.1.1. Họ font 48
4.6.1.2. Cách sử dụng Google Fonts 48
4.6.1.3. Cách sử dụng Icons CSS 49
4.6.1.4. Các thuộc tính định dạng cho text 50
4.6.2. Định dạng cho danh sách 51
4.6.3. Định dạng cho bảng 51
4.6.4. Định dạng nền 52
4.6.5. Định dạng viền 53
4.6.6. Định dạng đổ bóng 53
4.7. Thuộc tính định vị trí 55
4.7.1. Float và Clear 55
4.7.1.1. Thuộc tính Float 55
4.7.1.2. Thuộc tính Clear 55
4.7.2. Position và z-index 56
4.7.2.1. Thuộc tính position 56
4.7.2.2. Thuộc tính z-index 57
4.8. Xây dựng giao diện 58
4.8.1. Bố cục giao diện 58
4.8.2. Flexbox 60
4.8.2.1. Định nghĩa Flexbox 60
4.8.2.2. Bố cục Flexbox 61
4.8.2.3. Các thuộc tính trong Flexbox 61
4.8.3. Grid 64
4.8.3.1. Định nghĩa Grid 64
4.8.3.2. Bố cục Grid 64
4.8.3.3. Các thuộc tính trong Grid 65

4.8.4. Responsive website 67
4.8.4.1. Định nghĩa 67
4.8.4.2. Một số lưu ý khi responsive website 67
4.8.4.3. Các bước thiết kế responsive website 68
4.9. Hiệu ứng chuyển động 70
4.9.1. Thuộc tính transform 70
4.9.2. Thuộc tính transition 73
4.9.3. Thuộc tính animation 74
BÀI TẬP CHƯƠNG 4 77
Bài tập 1 77
Bài tập 2 77
Bài tập 3 77
Bài tập 4 77
Bài tập 5 77
Bài tập 6 77
Bài tập 7 77
Bài tập 8 78
Bài tập 9 78
Bài tập 10 78
CHƯƠNG 5. NGÔN NGỮ JAVASCRIPT 79
5.1. Tổng quan về Javascript 79
5.1.1. Khái niệm JS 79
5.1.2. Ưu điểm của JS 79
5.1.3. Ứng dụng của JS 79
5.1.4. Khai báo JS 79
5.1.5. Chú thích JS 79
5.1.6. Strict Mode 80
5.1.7. Developer Tools 80
5.2. Biến và Hằng 81
5.2.1. Biến 81
5.2.2. Hằng 81
5.3. Các kiểu dữ liệu trong JS 81

5.3.1. Các kiểu dữ liệu 81
5.3.1.1. Kiểu null 81
5.3.1.2. Kiểu undefined 81
5.3.1.3. Kiểu boolean 82
5.3.1.4. Kiểu number 82
5.3.1.5. Kiểu bitint 82
5.3.1.6. Kiểu string 82
5.3.1.7. Kiểu symbol 82
5.3.1.8. Kiểu object 83
5.3.2. Xác định kiểu dữ liệu 83
5.3.3. Chuyển đổi kiểu dữ liệu 83
5.4. Các toán tử trong JS 84
5.5. Các lệnh cơ bản 85
5.5.1. Lệnh rẽ nhánh 85
5.5.2. Lệnh lựa chọn 86
5.5.3. Lệnh lặp 86
5.5.3.1. Lệnh lặp for 86
5.5.3.2. Lệnh lặp while 86
5.5.3.3. Lệnh lặp do...while 87
5.5.3.4. Break và Continue 87
5.6. Hàm 87
5.6.1. Khai báo hàm 87
5.6.2. Biểu thức hàm 89
5.6.3. Hàm mũi tên 89
5.6.4. Các hàm tương tác người dùng 90
5.7. Object 90
5.7.1. Khái niệm Object 90
5.7.2. Cách sử dụng Object 91
5.7.3. So sánh 2 Object 92
5.7.4. Sao chép Object 95
5.7.5. Phương thức trong JS 95
5.7.6. Toán tử this 96

5.7.7. Khởi tạo đối tượng bằng new 96
5.7.8. Chuyển đổi object về kiểu nguyên thủy 97
5.8. Các phương thức của kiểu dữ liệu nguyên thủy 97
5.8.1. Phân biệt giữa kiểu dữ liệu nguyên thủy và object 97
5.8.2. Kiểu Number 97
5.8.3. Kiểu String 99
5.8.4. Kiểu Array 101
5.8.5. Kiểu Date 105
5.9. Biểu thức qui tắc 106
5.9.1. Khái niệm 106
5.9.2. Khởi tạo biểu thức qui tắc 106
5.9.3. Phương thức của biểu thức qui tắc 106
5.9.4. Một số khuôn mẫu cơ bản thường dùng 108
5.10. Mô hình đối tương tài liệu DOM 109
5.10.1. Khái niệm 109
5.10.2. Liên kết giữa các node 110
5.10.3. Một số phương thức trong DOM 110
5.10.4. Sự kiện trong DOM 110
5.10.5. Chuyển HTML String sang DOM 111
5.10.6. Chuyển HTML Template sang DOM 111
5.11. Mô hình đối tượng trình duyệt BOM 112
BÀI TẬP CHƯƠNG 5 114
Bài tập 1 114
Bài tập 2 114
Bài tập 3 114
Bài tập 4 114
Bài tập 5 114
Bài tập 6 114
Bài tập 7 114
Bài tập 8 115
Bài tập 9 115
Bài tập 10 115

DANH MỤC HÌNH ẢNH

Hình 1.1: Mô hình Client – Server 2
Hình 1.2: Qui trình hiển thị trang web trên tình duyệt 2
Hình 1.3: Mô hình 3 lớp trong thiết kế web 3
Hình 1.4: Qui trình xây dựng dự án web 3
Hình 2.1: Giao diện phần mềm Photoshop 6
Hình 2.2: Thanh trình đơn 7
Hình 2.3: Thanh công cụ 7
Hình 2.4: Thanh tùy chọn 7
Hình 2.5: Giao diện phần mềm VS Code 10
Hình 2.6: Bài tập thiết kế menu-1 13
Hình 2.7: Bài tập thiết kế menu-2 13
Hình 2.8: Bài tập thiết kế menu-3 và menu-4 13
Hình 2.9: Bài tập thiết kế menu-5 13
Hình 2.10: Bài tập thiết kế menu-6 14
Hình 2.11: Bài tập thiết kế giao diện news 14
Hình 2.12: Bài tập thiết kế giao diện header 14
Hình 2.13: Bài tập thiết kế giao diện footer 15
Hình 2.14: Bài tập thiết kế giao diện trang chủ của layout-1 15
Hình 2.15: Bài tập thiết kế giao diện trang chủ của layout-2 16
Hình 2.16: Bài tập thiết kế giao diện trang Hot Coffee của layout-2 16
Hình 2.17: Bài tập thiết kế giao diện trang Fruit Juice của layout-2 17
Hình 2.18: Bài tập thiết kế giao diện trang About Us của layout-2 17
Hình 2.19: Bài tập thiết kế giao diện trang Speciality của layout-2 18
Hình 2.20: Bài tập thiết kế giao diện trang Contact của layout-2 18
Hình 3.1: Cấu trúc tài liệu HTML 19
Hình 3.2: Bố cục giao diện 23
Hình 3.3: Thẻ tạo bản đồ 26
Hình 3.4: Thẻ tạo bảng biểu 28
Hình 3.5: Thẻ iframe 30
Hình 3.6: Thẻ form 31
Hình 3.7: Bài tập html-5 38
Hình 3.8: Bài tập html-7 39
Hình 3.9: Bài tập html-8 40
Hình 4.1: Tổng quan CSS 41

Hình 4.2: Khai báo bộ chọn....44
Hình 4.3: Mô hình hộp....45
Hình 4.4: Thuộc tính outline....47
Hình 4.5: Thuộc tính box-sizing....47
Hình 4.6: Các họ font....48
Hình 4.7: Thiết lập Awesome Icons....49
Hình 4.8: Thiết lập Google Icons....50
Hình 4.9: Thuộc tính text-shadow....54
Hình 4.10: Thuộc tính box-shadow....54
Hình 4.11: Thuộc tính float....55
Hình 4.12: Thuộc tính float có giá trị left....55
Hình 4.13: Thuộc tính float có giá trị right....55
Hình 4.14: Thuộc tính clear....55
Hình 4.15: Thuộc tính clear có giá trị left....55
Hình 4.16: Thuộc tính clear có giá trị both....56
Hình 4.17: Thuộc tính position và z-index....56
Hình 4.18: Thuộc tính position....57
Hình 4.19: Thuộc tính z-index....57
Hình 4.20: Giao diện 3 khối hộp....58
Hình 4.21: Giao diện 5 khối hộp....58
Hình 4.22: Giao diện lưới....58
Hình 4.23: Giao diện lưới đa năng....59
Hình 4.24: Giao diện có thanh lề cố định....59
Hình 4.25: Giao diện màn hình 3 chiều....59
Hình 4.26: Giao diện sủ dụng đồ họa làm trung tâm....60
Hình 4.27: Giao diện có nền kín trang....60
Hình 4.28: Giao diện thư viện ảnh....60
Hình 4.29: Bố cục Flexbox....61
Hình 4.30: Bố cục Grid....65
Hình 4.31: Thuộc tính grid-template-areas....66
Hình 4.32: Responsive website....67
Hình 4.33: Thuộc tính transform-style....70
Hình 4.34: Thuộc tính transform-origin....71
Hình 4.35: Thuộc tính translate 2D....71
Hình 4.36: Thuộc tính scale 2D....72
Hình 4.37: Thuộc tính skew 2D....72
Hình 4.38: Thuộc tính transform 2D....72

Hình 4.39: Thuộc tính translate 3D 73
Hình 4.40: Thuộc tính rotate 3D 73
Hình 4.41: Bài tập tạo nút lệnh download 77
Hình 4.42: Bài tập transform 78
Hình 4.43: Bài tập tạo menu 78
Hình 5.1: Các phần tử trong tài liệu phân cấp theo mô hình DOM 109
Hình 5.2: Mô hình đối tượng trình duyệt BOM 113

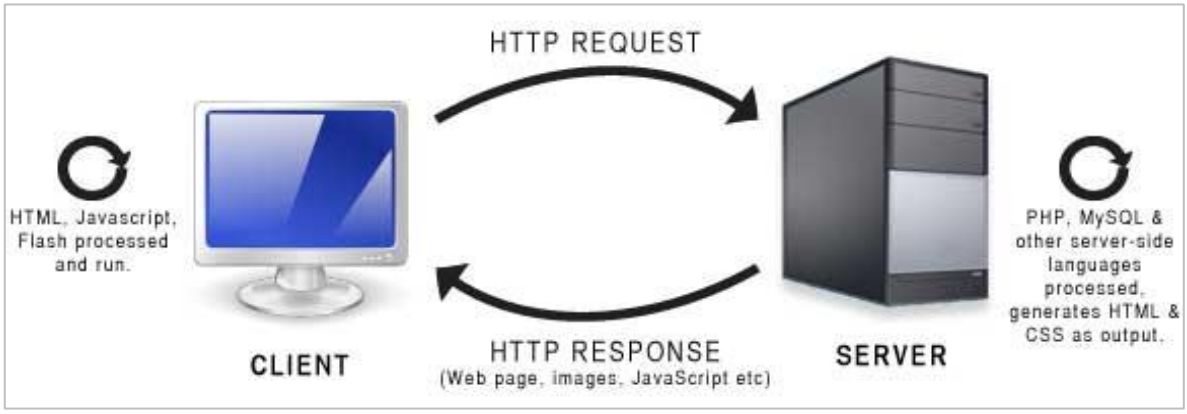
DANH MỤC BẢNG

Bảng 2.1: Phím tắt phần mềm Visual Studio Code....12
Bảng 3.1: Khai báo Doctype cho tài liệu....20
Bảng 3.2: Các thẻ trong phần head....21
Bảng 3.3: Các kí hiệu đặc biệt....22
Bảng 3.4: Các thẻ tạo bố cục....23
Bảng 3.5: Các thẻ định dạng cho text....24
Bảng 3.6: Các thẻ danh sách....27
Bảng 3.7: Các thành phần trong form....32
Bảng 3.8: Các sự kiện liên quan đến form....35
Bảng 3.9: Các sự kiện liên quan đến phím....35
Bảng 3.10: Các sự kiện liên quan đến chuột....36
Bảng 3.11: Các sự kiện liên quan đến đa phương tiện....37
Bảng 4.1: Các bộ chọn dựa vào thuộc tính....45
Bảng 4.2: Thuộc tính định dạng cho text....51
Bảng 4.3: Thuộc tính định dạng cho danh sách....51
Bảng 4.4: Thuộc tính định dạng cho bảng....52
Bảng 4.5: Thuộc tính background....53
Bảng 4.6: Thuộc tính border....53
Bảng 4.7: Thuộc tính cho thành phần Flex Container....63
Bảng 4.8: Thuộc tính cho thành phần Flex Item....64
Bảng 4.9: Thuộc tính cho thành phần Grid Container....65
Bảng 4.10: Thuộc tính cho thành phần Grid Item....66
Bảng 4.11: Thuộc tính transform 2D....71
Bảng 4.12: Thuộc tính transform 3D....73
Bảng 4.13: Thuộc tính transition....74
Bảng 4.14: Thuộc tính animation....76

CHƯƠNG 1. GIỚI THIỆU CHUNG

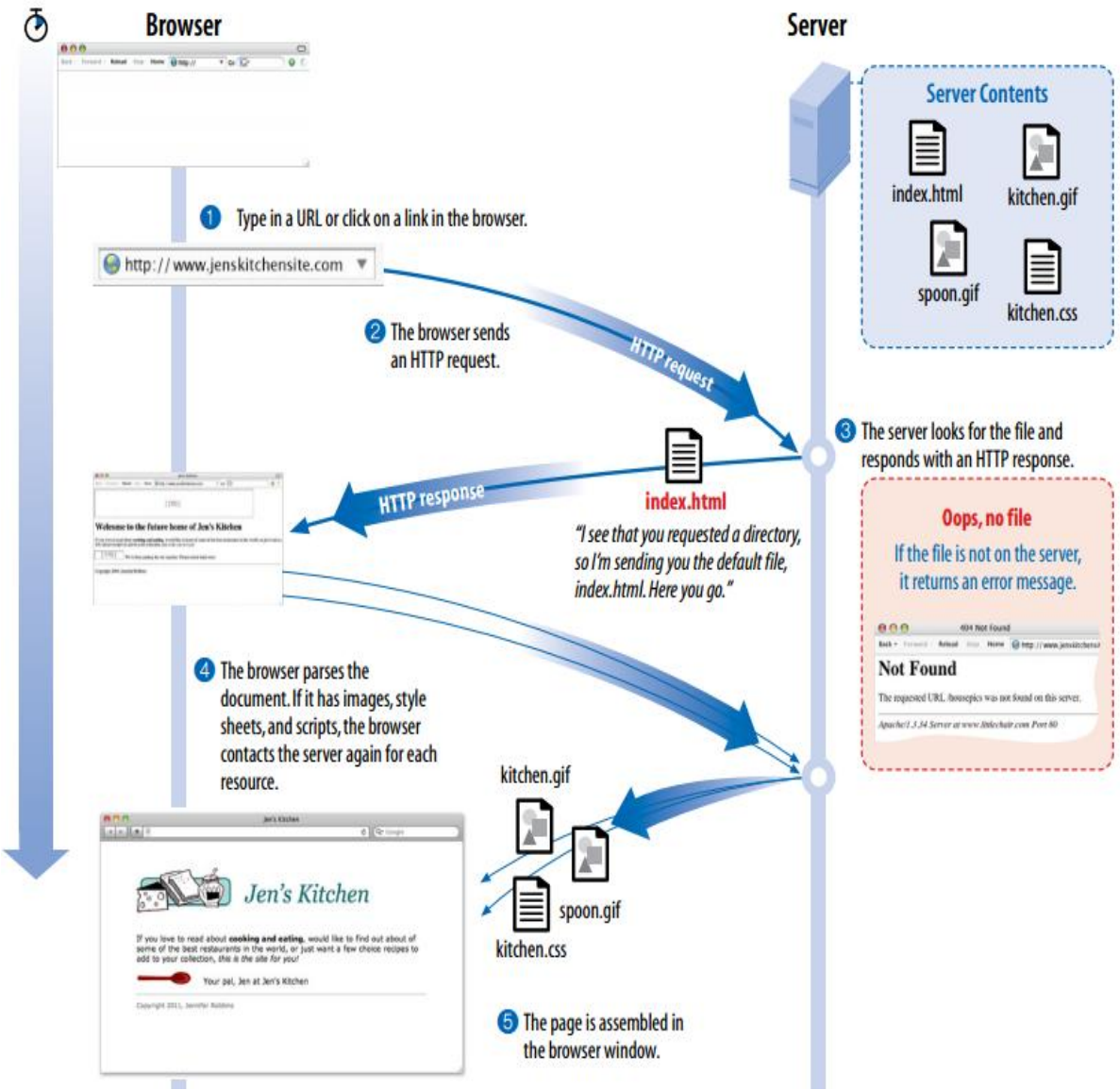
1.1. Các thuật ngữ cơ bản

- WWW (World Wide Web): là mạng lưới toàn cầu, là không gian thông tin mà mọi người có thể truy cập qua các máy tính có kết nối với mạng Internet
- W3C (World Wide Web Consortium): là 1 hiệp hội lập ra các chuẩn cho Internet, nhất là WWW
- Protocol (Giao thức): là tập hợp các quy tắc truyền thông của mạng mà tất cả các thực thể tham gia truyền thông phải tuân theo
- Internet Protocol (Địa chỉ IP): là một địa chỉ đơn nhất mà những thiết bị điện tử hiện nay đang sử dụng để nhận diện và liên lạc với nhau trên mạng máy tính. Các loại địa chỉ IP: IPv4 (sử dụng 32 bit để mã hoá dữ liệu), IPv6 (sử dụng 128 bit để mã hoá dữ liệu)
  - VD: 14.225.11.14: IP của website ute.udn.vn
- Hosting: là dịch vụ lưu trữ các trang web trên máy chủ kết nối Internet.
- Domain Name: là tên miền, hay còn gọi là định danh của website
- DNS (Domain Name System): là hệ thống tên miền được phát minh vào năm 1984 cho Internet, cho phép thiết lập tương ứng giữa địa chỉ IP và tên miền
- URL (Uniform Resource Locator): được dùng để tham chiếu tới tài nguyên trên Internet
  - Đường dẫn tuyệt đối: không thay đổi, gồm có 3 thành phần là giao thức, tên domain, tên file, thường thấy trên thanh địa chỉ của trình duyệt
    - VD**:** http://ute.udn.vn/default.aspx hoặc https://www.google.com
  - Đường dẫn tương đối: thay đổi, là đường dẫn có điểm xuất phát là thư mục hiện tại, hoặc thư mục cha của thư mục hiện tại.
    - VD1: kinhte.html
    - VD2: mucluc/kinhte.html
    - VD3: ../kinhte.html
    - VD4: ../mucluc/kinhte.html
- Mô hình Client – Server: là mô hình nổi tiếng và được áp dụng trong mọi trang web. Ý tưởng của mô hình là máy khách (client) gởi một yêu cầu (request) cho máy chủ (server), máy chủ sẽ xử lý và trả về kết quả cho máy khách
  - Client: là máy tính dùng để kết nối và khai thác các tài nguyên trên máy chủ. Một máy tính có thể vừa là client vừa là server.Việc kết nối client với server và việc khai thác dịch vụ của server tạo nên mô hình client – server
  - Server: Chứa dữ liệu, tài nguyên và dịch vụ cho phép các máy khác có thể khai thác và truy cập. Một máy chủ có thể dùng cho một hay nhiều mục đích. VD: Mail Server, File Server, Web Server, …



Hình 1.1: Mô hình Client – Server

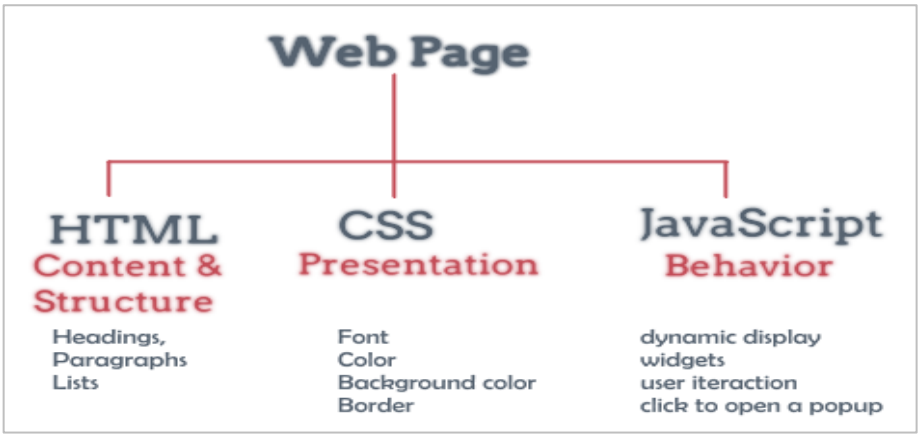
- Web browser: là các phần mềm dùng để truy cập và hiển thị nội dung trang web. VD: Internet Explorer, Mozilla Firefox, Google Chrome, Safari, Opera,…
- Web server: là máy tính phân phối dịch vụ và thông tin cho máy khách, có cấu hình cao, tốc độ xử lý nhanh, hoạt động 24/7, thường cài đặt các phần mềm máy chủ như Apache, IIS, Nginx, …



Hình 1.2: Qui trình hiển thị trang web trên tình duyệt

- **Web page**: là 1 tài liệu HTML hoặc XHTML có thể truy cập bằng cách sử dụng giao thức HTTP
- **Website**: là tập hợp các trang web có liên quan nhau, chứa các văn bản, đối tượng đồ họa, âm thanh, được lưu trữ trên máy chủ web, truy cập thông qua Internet và trình duyệt web; trong 1 website chỉ có 1 trang chủ (index.html hoặc default.html)
  - Web tĩnh: sử dụng HTML, CSS, Javascript cùng với hình ảnh, video, audio, flash để tạo giao diện cho trang web. Trong web tĩnh không có hệ cơ sở dữ liệu như MySQL hay MSSQL.
  - Web động: web có hệ thống cơ sở dữ liệu dùng để cung cấp thông tin cho website, điểm mạnh của web động so với web tĩnh là khả năng quản lý dữ liệu web tốt, dễ dàng cập nhật nội dung và thêm các tính năng dễ dàng quản lý

1.2. Mô hình 3 lớp trong thiết kế web



Hình 1.3: Mô hình 3 lớp trong thiết kế web

1.3. Qui trình xây dựng trang web

- Giai đoạn 1: Tiếp xúc, tìm hiểu - Initial Planning Phase
- Giai đoạn 2: Phân tích - Analysis Phase
- Giai đoạn 3: Thiết kế - Design Phase
- Giai đoạn 4: Xây dựng - Production Phase
- Giai đoạn 5: Kiểm thử - Test Phase
- Giai đoạn 6: Chuyển giao - Deployment Phase



Hình 1.4: Qui trình xây dựng dự án web

BÀI TẬP CHƯƠNG 1

Bài tập 1

Trong qui trình thiết kế web, cho biết nhiệm vụ của các cá nhân, tổ chức sau:

a. Graphic Designer (Nhân viên thiết kế đồ họa)
b. Production Department (Bộ phận sản xuất)
c. User Experience Designer (Nhân viên thiết kế trải nghiệm người dùng)
d. Web Programmer (Nhân viên lập trình web)

Bài tập 2

Nêu chức năng của tổ chức W3C

Bài tập 3

Phân biệt Front-end, Back-end và Full stack

CHƯƠNG 2. CÔNG CỤ HỖ TRỢ THIẾT KẾ WEB

2.1. Các khái niệm cơ bản

2.1.1. Hệ màu

- **CMYK** là hệ màu được sử dụng cho ngành in ấn. 4 chữ CMYK là chữ viết tắt của 4 kênh màu chính bao gồm: **C**yan (xanh da trời), **M**agenta (đỏ sen), **Y**ellow (vàng), và Blac**K** (đen).
- **RGB** là hệ màu gồm 3 kênh màu dành cho các ảnh hiển thị trên màn hình như web, quảng cáo trên tv ... bao gồm **R**ed (đỏ), **G**reen (xanh lá) và **Bl**ue (xanh dương)

2.1.2. Định dạng ảnh

- **.psd** : tập tin *psd* là tập tin định dạng của photoshop được lưu lại mặc định. Nó chứa nhiều *layer* và cùng các tham số đã soạn thảo trước đó. Trong nhiều trường hợp, bạn có thể chuyển đổi tập tin *.psd* thành các tập tin ảnh ở nhiều định dạng khác nhau.
- **.gif:** ảnh dạng nén(nén hình với những màu giống nhau), giới hạn trong 256 màu, dung lượng nhỏ, hỗ trợ hiệu ứng động và chế độ trong suốt, sử dụng cho hình logo, icons và hình đồ hoạ
- **.jpeg:** hỗ trợ 16.7 triệu màu, nó được dùng chủ yếu cho hình chụp tự nhiên, thường dùng trong web, không hỗ trợ chế độ trong suốt
- **.png:** thường dùng trong web, hỗ trợ chế độ trong suốt

2.1.3. Thuật ngữ cơ bản

- **Layer**: Tập tin photoshop được kết hợp bởi nhiều *layer* (lớp), theo cách giống như từng lớp trong suốt chứa từng phần của một bức ảnh chồng lên nhau. Các *layer* có thể chứa ảnh, chữ hoặc các hình đồ họa vector … chúng có thể được nhóm (*group*) lại với nhau hoặc đổi vị trí trên dưới lẫn nhau. Trong photoshop, khi thêm mới chữ, một *layer* mới sẽ được tạo tự động. Khi muốn ghép 2 layer lại với nhau, bạn có thể sử dụng chức năng *merge down* hoặc *faltten image* để ghép tất cả *layer* thành một *layer* duy nhất.
- **Selection** (vùng chọn): Vùng chọn là khu vực chịu sự tác động bởi các công cụ và chức năng được áp dụng lên ảnh. Khi có một vùng chọn, bạn có thể áp dụng các công cụ như *paintbrush* (cọ vẽ), *copy* (nhân bản) hoặc *crop* (cắt xén). Vùng chọn trong photoshop có các hình dáng và kích cỡ bất kỳ tùy theo bạn sử dụng công cụ *selection* nào. Vùng chọn chỉ có tác động lên *layer* hiện hành. Nếu vùng chọn nằm ở layer rỗng, chương trình sẽ báo lỗi. Khi gặp trường hợp này, cần chú ý đến bảng *layer panel* và xem đã chọn đúng *layer* hay chưa.
- **Resolution** (độ phân giải): là thuật ngữ chỉ đến số *pixel* trên toàn bộ một bức ảnh. Một bức ảnh *High Resolution* (độ phân giải cao) chứa nhiều thông tin hơn một bức ảnh *Low Resolution* (độ phân giải thấp). Khi chuyển đổi ảnh có độ phân giải cao xuống ảnh có độ phân giải thấp thì không có vấn đề gì, nhưng nếu làm ngược lại, bức ảnh sẽ bị nhòe hoặc không đạt chất lượng bởi vì lượng thông tin bị mất không

được bù đắp đúng cách. Hình ảnh tạo ra với mục đích hiển thị trên các thiết bị màn hình thì độ phân giải thường dùng là 72 dpi (*pixel per inches*). Đối với các ảnh dùng cho in ấn, tùy theo loại máy in sử dụng, 300 dpi là độ phân giải chuẩn và 150 dpi là độ phân giải thấp nhất có thể chấp nhận

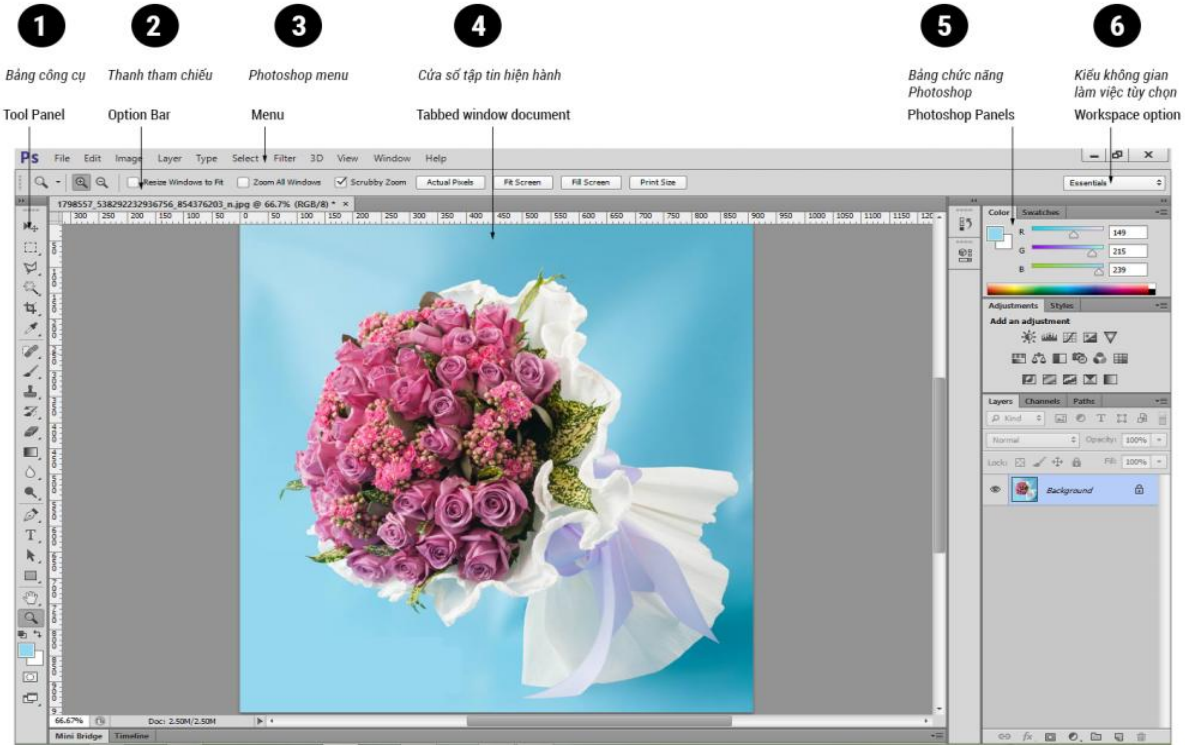
- **Image size** (kích thước ảnh): Kích thước ảnh được tính bằng số pixel dọc và ngang của một bức ảnh. Độ phân giải đôi khi bị nhầm lẫn với kích thước ảnh, bởi vì kích thước ảnh cũng có đơn vị bằng *pixel*.

2.1.4. Phần mềm thiết kế

- Adobe Photoshop: Thường dùng thiết kế giao diện web, thiết kế ảnh động,... (chủ yếu thao tác trên điểm ảnh - bitmap)
- Adobe Illustrator (AI): Thường dùng thiết kế logo, banner, icon, các sản phẩm in ấn,...(chủ yếu thao tác trên các đồ họa vector)
- Ngoài ra còn có các phần mềm Adobe Fireworks, Corel Draw, ...

2.2. Phần mềm Adobe Photoshop

2.2.1. Giao diện phần mềm Photoshop

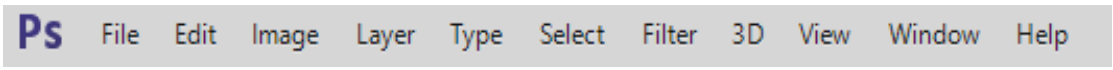


Hình 2.1: Giao diện phần mềm Photoshop

2.2.1.2. Thanh trình đơn

- **File:** chứa những chức năng cơ bản nhất một phần mềm và chức năng *Import* để đưa các hình ảnh từ máy *Scan* hay chức năng *Save for Web* dùng để lưu các hình ảnh ở định dạng được hỗ trợ dành cho web.
- **Edit**: bao gồm chức năng cơ bản để thay đổi các bức ảnh, như *Fill & Stroke* (tô màu và đường viền), *Transform* (xoay, lật, kéo giản ảnh)
- **Image:** bao gồm các chức năng có thể tác động lên toàn bộ bức ảnh như *Color Adjustments* (chỉnh màu sắc), *Size Adjustments* (chỉnh kích thước) và một số chức năng khác có thể thay đổi toàn bộ một bức ảnh trong photoshop.

- **Layer:** bao gồm các chức năng tác động sự thay đổi chỉ tác động tại một *Layer* Một ảnh trong photoshop được cấu tạo bởi nhiều *Layer* (lớp) trong suốt chồng lên nhau. Do vậy, những chức năng trong nhánh *Layer* chỉ sẽ có tác dụng ở một hoặc nhiều *Layer* được chọn, khác với *menu Image* là ảnh hưởng đến toàn bộ bức ảnh.
- **Type:** bao gồm các chức năng để thay đổi hiệu ứng cho chữ và kiểu chữ.
- **Select:** bao gồm các chức năng về *Selection* (vùng chọn) như lưu vùng chọn, tinh chỉnh vùng chọn hoặc thêm, bớt, đảo vùng chọn.
- **Filter:** tạo và hiệu chỉnh bộ lọc, có thể áp dụng lên bất kỳ phần nào của ảnh và tạo những hiệu ứng độc đáo.
- **View:** thay đổi các thông số hiển thị để xem trên giao diện photoshop, có thể sử dụng chức năng tắt / mở các đường dẫn (*guidelines*) trên ảnh, phóng to (*zoom out*) thu nhỏ (*zoom in*)
- **Window:** hiển thị hoặc giấu các bảng chức năng trong giao diện *photoshop*.
- **Help** là phần thông tin trợ giúp bằng tiếng Anh.



Hình 2.2: Thanh trình đơn

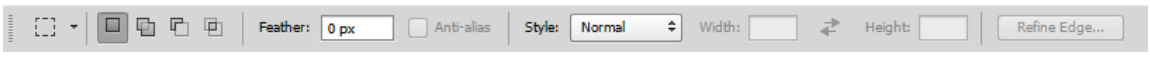
2.2.1.3. Thanh công cụ



Hình 2.3: Thanh công cụ

2.2.1.4. Thanh tùy chọn

Mỗi công cụ khác nhau sẽ có thanh tùy chọn khác nhau tương ứng.



Hình 2.4: Thanh tùy chọn

2.2.2. Chế độ màn hình

- **Standard Screen Mode**: Xem ảnh ở chế độ mặc định
- **Full Screen Mode with menu bar**: Xem ảnh trên toàn bộ màn hình với 1 thanh trình đơn
- **Full Screen Mode**: Xem ảnh trên toàn màn hình màu đen
- Thay đổi giữa các chế độ màn hình: View → Screen Mode → Chọn chế độ (**F**)

2.2.3. Thiết lập cơ bản

- Thay đổi đơn vị đo trong PS: Edit → Preferences → Units and Rulers → Chọn mục Rulers và Type đều là PIXELS
- Tạo và thiết lập môi trường làm việc mới (workspace)
  - Khôi phục cấu hình mặc định: **Ctrl + Alt + Shift** (nhấn giữ khi khởi động PS)
  - Khôi phục môi trường làm việc mặc định:
    Window → Workspace → Reset Essentials
- **Tab:** Ẩn/Hiện Toolbox, các Palettes
- **Ctrl + ":** Hiển thị các ô ly trên màn hình hiển thị ảnh
- **Ctrl+R:** Ẩn/Hiện Ruler
- Kích chuột phải vào thanh thước để thay đổi đơn vị đo phù hợp
- Tạo đường guide
  - C1: Vào View → New Guide: để tạo đường guide tại vị trí như ý
  - C2: Kéo từ thanh thước ra
- Khóa các đường guide: View → Lock Guides
- **Ctrl+H:** Ẩn/hiện các vùng chọn, gridlines, extras

2.2.4. Thao tác trên file

- **Ctrl + N**: New (Tạo mới file)
- **Ctrl + O**: Open (Mở file)
- Ctrl + S: Lưu file
- **Ctrl + W**: Close (Đóng file)
- **Ctrl + Alt + W**: Close All (Đóng tất cả các file)
- Ctrl + P: In
- Ctrl + Alt + Shift + S: Save for web
- **Ctrl + Q**: Thoát PS
- **Ctrl + Alt + I:** Xem kích thước ảnh
- **Ctrl + Alt + C:** Thay đổi kích thước ảnh

2.2.5. Công cụ thường dùng

- Công cụ Move (V): di chuyển đối tượng tư vị trí này đến vị trí khác
- Công cụ Hand (H): kéo những vùng ảnh bị khuất ra màn hình trong trường hợp file ảnh đang zoom quá lớn
- Công cụ Zoom (Z): phóng to, thu nhỏ độ hiển thị của ảnh
- Công cụ Foreground/Background Color: thiết lập màu đối tượng/màu nền
- Công cụ EyeDropper (I): lấy mã màu của 1 điểm ảnh

- Làm việc với Text: công cụ Type (T)
- Làm việc với vùng chọn: công cụ Marquee (M), Lasso (L), Quick Selection (W), Pen (P) (thêm, xóa, sửa, di chuyển,…)
- Làm việc với layer: Khóa, tạo, ẩn/hiện, chọn, sao chép, di chuyển vị trí, liên kết, trộn, nhóm layer, độ trong suốt của layer

2.2.6. Trích xuất ảnh cho web

- Cắt ảnh
  - Bước 1: Ctrl + N (Chọn nền trong suốt hoặc có màu)
  - Bước 2: Chỉ chọn layer có chứa đối tượng cần cắt (Nhấn Alt + Kích vào con mắt trên layer palette để ẩn hết các layer còn lại)
  - Bước 3: Kích chuột phải vào layer → Duplicate Layer → Chọn tên file cần sao chép qua ở mục Document
  - Bước 4: **Image** → **Trim** → Chọn nền trong suốt hoặc có màu → OK
  - Bước 5: Ctrl + Alt + Shift + S
- Trích xuất ảnh
  - Bước 1: Sử dụng công cụ Slice (C) để cắt
  - Bước 2: Ctrl + Alt + Shift + S
  - Bước 3: Nhấn giữ phím Shift và chọn các hình cần lấy
  - Bước 4: Nhấn Save → Image Only (chỉ xuất ra hình)
  - Bước *5:* Chọn **Selected Slice** để xuất ra những hình cần lấy

2.3. Phần mềm hỗ trợ biên soạn mã nguồn

2.3.1. IDE

IDE là viết tắt của "Integrated Development Environment" hay "môi trường phát triển tích hợp". Về bản chất, IDE là một loại phần mềm máy tính cung cấp các công cụ cần thiết cho lập trình viên trong việc phát triển phần mềm.

IDE thường bao gồm những thành phần sau:

- Trình soạn thảo (source code editor): dùng để viết mã nguồn (code).
- Trình biên dịch (compiler) để dịch mã nguồn thành mã máy
- Trình thông dịch (interpreter) dùng để thực thi mã nguồn.
- Trình gỡ lỗi (debugger) dùng để hỗ trợ dò tìm lỗi trong mã nguồn.
- Ngoài ra, IDE có thể tích hợp hệ thống quản lý phiên bản (VCS), công cụ gợi ý viết code, công cụ tìm kiếm, công cụ kéo thả giúp xây dựng giao diện đồ họa (GUI) dễ dàng hơn...

2.3.2. Code Editor

Code editor cũng là một loại phần mềm máy tính hỗ trợ lập trình viên trong phát triển phần mềm, nhưng so với IDE thì code editor đơn giản, nhẹ và nhanh hơn.

Nguyên nhân chính là do code editor thường chỉ làm việc với một tệp tin (file). Trong khi đó, IDE lại làm việc trên toàn bộ thư mục project - bao gồm nhiều file khác nhau.

Một số code editor nổi tiếng hỗ trợ lập trình JavaScript:

- Visual Studio Code (miễn phí, đa nền tảng).

- Atom (miễn phí, đa nền tảng).
- Sublime Text (đa nền tảng, có phiên bản miễn phí bị giới hạn một số tính năng).
- Notepad++ (miễn phí, dùng trong hệ điều hành Windows).
- Vim, Emacs (miễn phí, dùng trong hệ điều hành Linux).

2.3.3. Phần mềm Visual Studio Code

2.3.3.1. Giới thiệu phần mềm VS Code

Visual Studio Code là một trình mã biên tập được phát triển bởi Microsoft. Một số chức năng tiêu biểu của VS Code là: Git, Debug, Syntax Highlighting, Snippets.

VS Code hỗ trợ nhiều ngôn ngữ lập trình như: HTML, CSS, JavaScript, C/C++, C#, F#, JSON. Người dùng có thể sử dụng phần mềm VS Code trên nền tảng Windows, Mac, Linux.

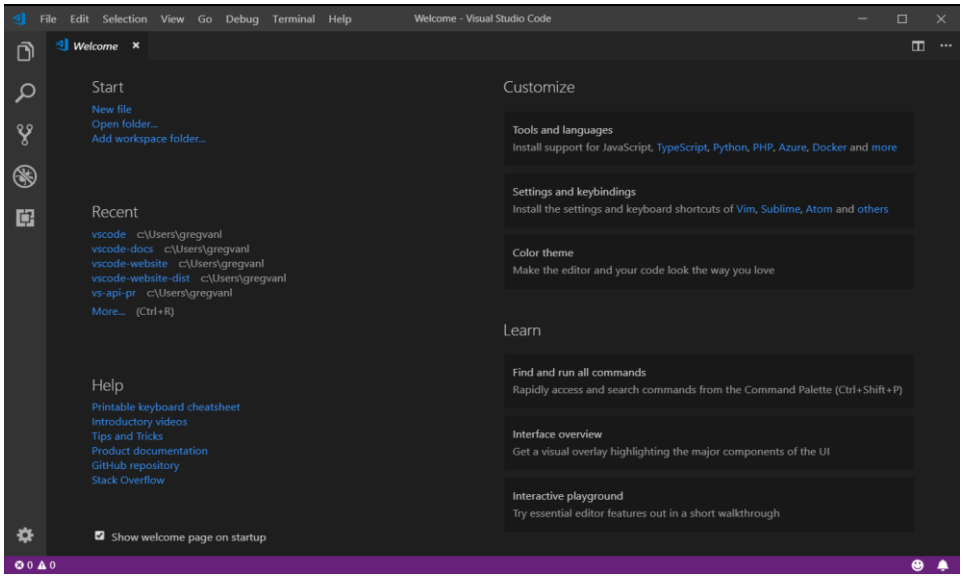
Link download phần mềm: https://code.visualstudio.com/

2.3.3.2. Phần mở rộng trong phần mềm VS Code

Extension là các phần mở rộng được cài đặt thêm vào phần mềm để hỗ trợ viết code nhanh hơn, báo lỗi, nhắc lệnh hoặc chạy một file code bất kỳ...

- Emmet: Soạn thảo html nhanh
- Live Server: Hiển thị trực tiếp kết quả trong khi code
- Code Spell Checker: Kiểm tra, thông báo phần code bị gõ sai
- Bracket Pair Colorizer: Phân biệt các cặp dấu đóng, mở ngoặc
- Colorize: Phân biệt các mã màu CSS
- Prettier Code Format: Tự động format code gọn gàng, dễ đọc
- Path Intellisens: Đề xuất đường dẫn thư mục, tập tin
- Better Comments: Tạo comment code nhiều màu, dễ phân loại
- Auto Rename Tag: Tự động thay đổi tên tag
- Auto Close Tag: Tự động đóng tag
- Drawio: Vẽ biểu đồ trực tiếp tại Vscode
- Material Icons: Bộ Icon đẹp cho các folder, file trong VScode

2.3.3.3. Giao diện phần mềm VS Code



Hình 2.5: Giao diện phần mềm VS Code

2.3.3.4. Các phím tắt

STT	Phím tắt	Chức năng
1	Ctrl + N	Tập tin mới
2	Ctrl + O	Mở tệp
3	Ctrl + S	Lưu file
4	Ctrl + Shift + S	Lưu bản sao
5	Ctrl + K + S	Lưu tất cả
6	Ctrl + F4	Đóng
7	Ctrl + K Ctrl + W	Đóng tất cả
8	Ctrl + Shift + T	Mở lại editor đã đóng
9	Ctrl + Shift + P	Hiển thị bảng lệnh
10	Ctrl + P	Mở nhanh vào tập tin
11	Ctrl + Shift + N	Tạo cửa sổ mới
12	Ctrl + Shift + W	Đóng cửa sổ
13	Ctrl + X	Cắt dòng
14	Ctrl + C	Sao chép dòng
15	Di chuyển lên / xuống	Alt + Up / Down
16	Shift + Alt + Up / Down	Sao chép dòng lên / xuống
17	Ctrl + Shift + K	Xóa dòng
18	Ctrl + Enter	Chèn dòng bên dưới
19	Ctrl + Shift + Enter	Chèn dòng trên
20	Ctrl +] / [	Thò / thụt dòng
21	Thu gọn theo vùng	Ctrl + Shift + [
22	Ctrl + /	Chuyển đổi nhận xét dòng
23	Shift + Alt + A	Chuyển đổi nhận xét khối
24	Ctrl + T	Hiển thị tất cả các biểu tượng
25	Ctrl + G	Chuyển đến dòng...
26	Ctrl + P	Chuyển đến file
27	Ctrl + Shift + M	Bảng điều khiển vấn đề
28	F8	Chuyển đến lỗi hoặc cảnh báo tiếp theo
29	Shift + F8	Chuyển đến lỗi hoặc cảnh báo trước đó
30	Ctrl + K Ctrl + X	Cắt khoảng trắng
31	F11	Bật chế độ toàn màn hình
32	Shift + Alt + 0	Chuyển đổi bố cục editor (ngang / dọc)

33	Ctrl + = / -	Phóng to / thu nhỏ
34	Ctrl + B	Chuyển đổi chế độ hiển thị Sidebar
35	Ctrl + Shift + E	Hiển thị Explorer / Toggle focus
36	Ctrl + Shift + F	Hiển thị tìm kiếm
37	Ctrl + Shift + G	Hiển thị Source Control
38	Ctrl + Shift + D	Hiển thị Debug
39	Ctrl + Shift + X	Hiển thị Extension
40	Ctrl + Shift + H	Thay thế trong tập tin
41	Ctrl + Shift + J	Chuyển đổi chi tiết tìm kiếm
42	Ctrl + K + Z	Chế độ Zen (Esc để thoát)

Bảng 2.1: Phím tắt phần mềm Visual Studio Code

BÀI TẬP CHƯƠNG 2

Bài tập 1

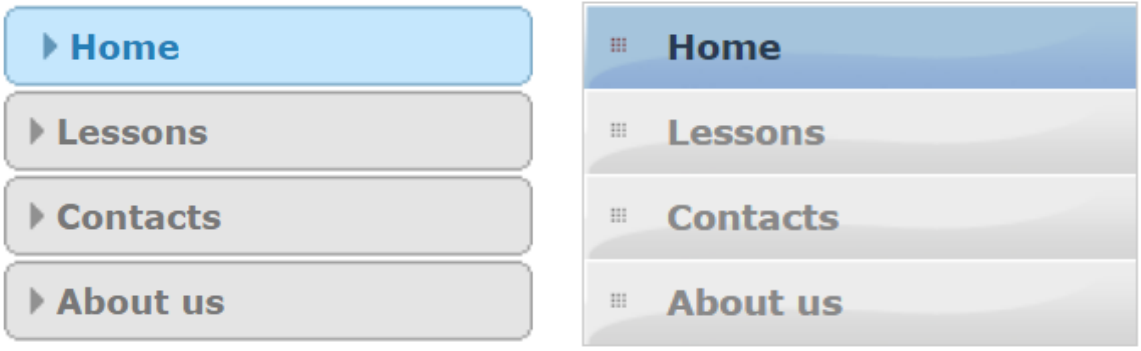
Thiết kế các menu có giao diện như hình bên dưới:



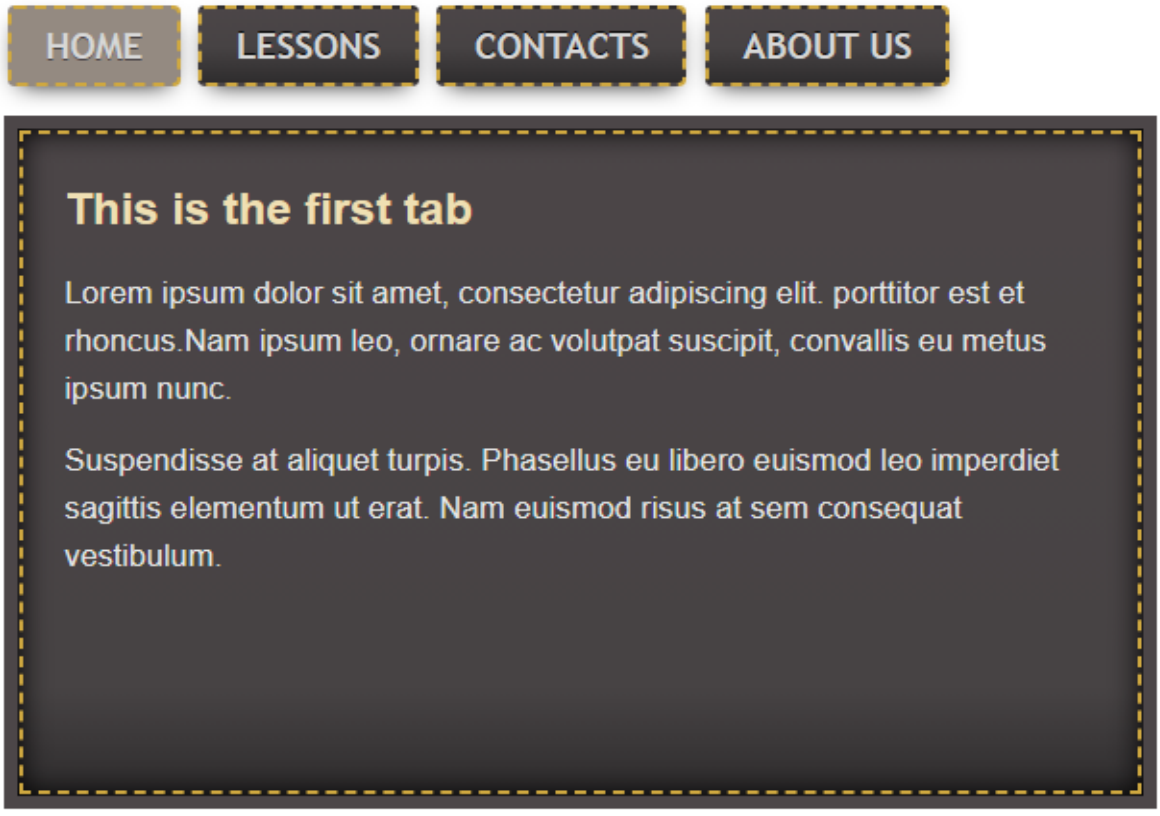
Hình 2.6: Bài tập thiết kế menu-1



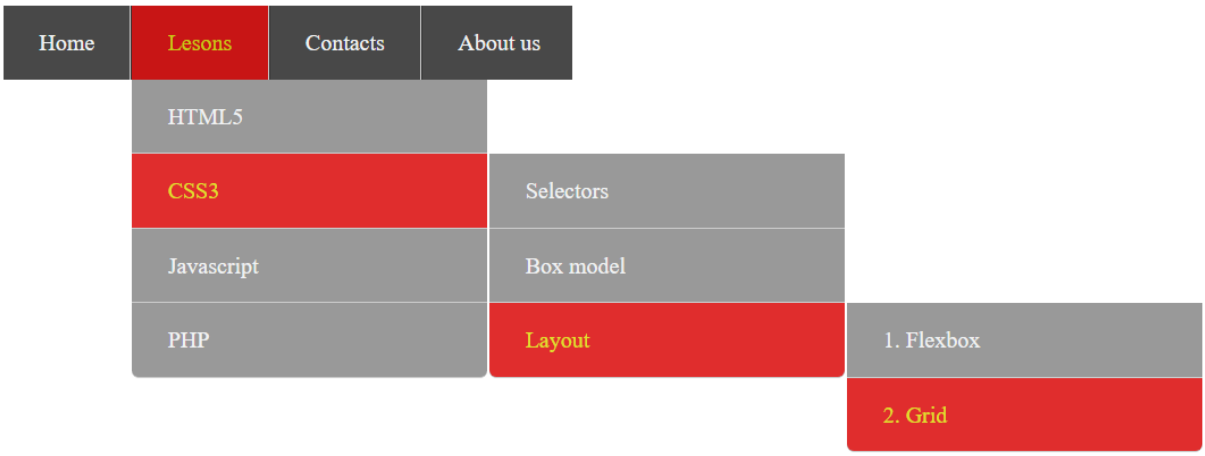
Hình 2.7: Bài tập thiết kế menu-2



Hình 2.8: Bài tập thiết kế menu-3 và menu-4



Hình 2.9: Bài tập thiết kế menu-5



Hình 2.10: Bài tập thiết kế menu-6

Bài tập 2

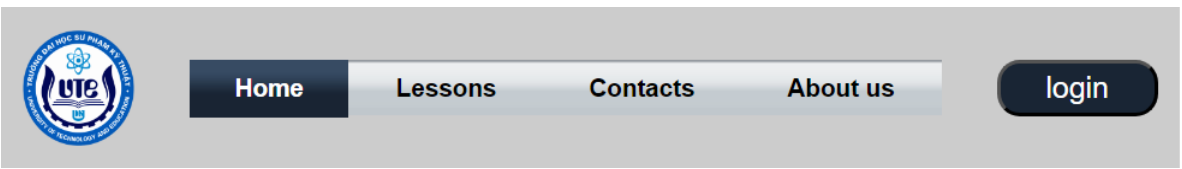
Thiết kế giao diện tin tức như hình dưới:



Hình 2.11: Bài tập thiết kế giao diện news

Bài tập 3

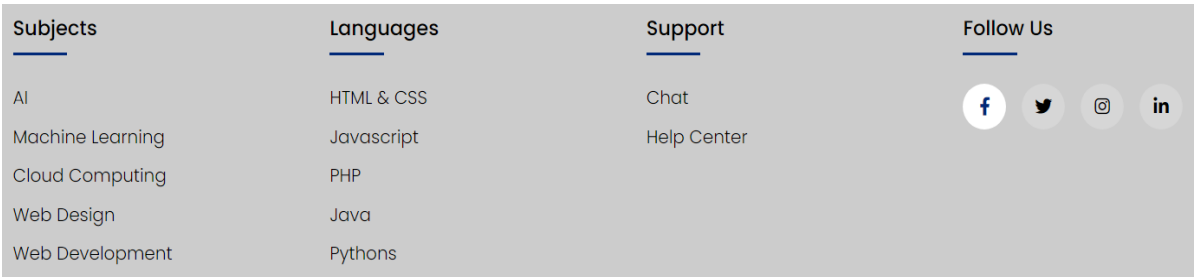
Thiết kế giao diện header như hình dưới:



Hình 2.12: Bài tập thiết kế giao diện header

Bài tập 4

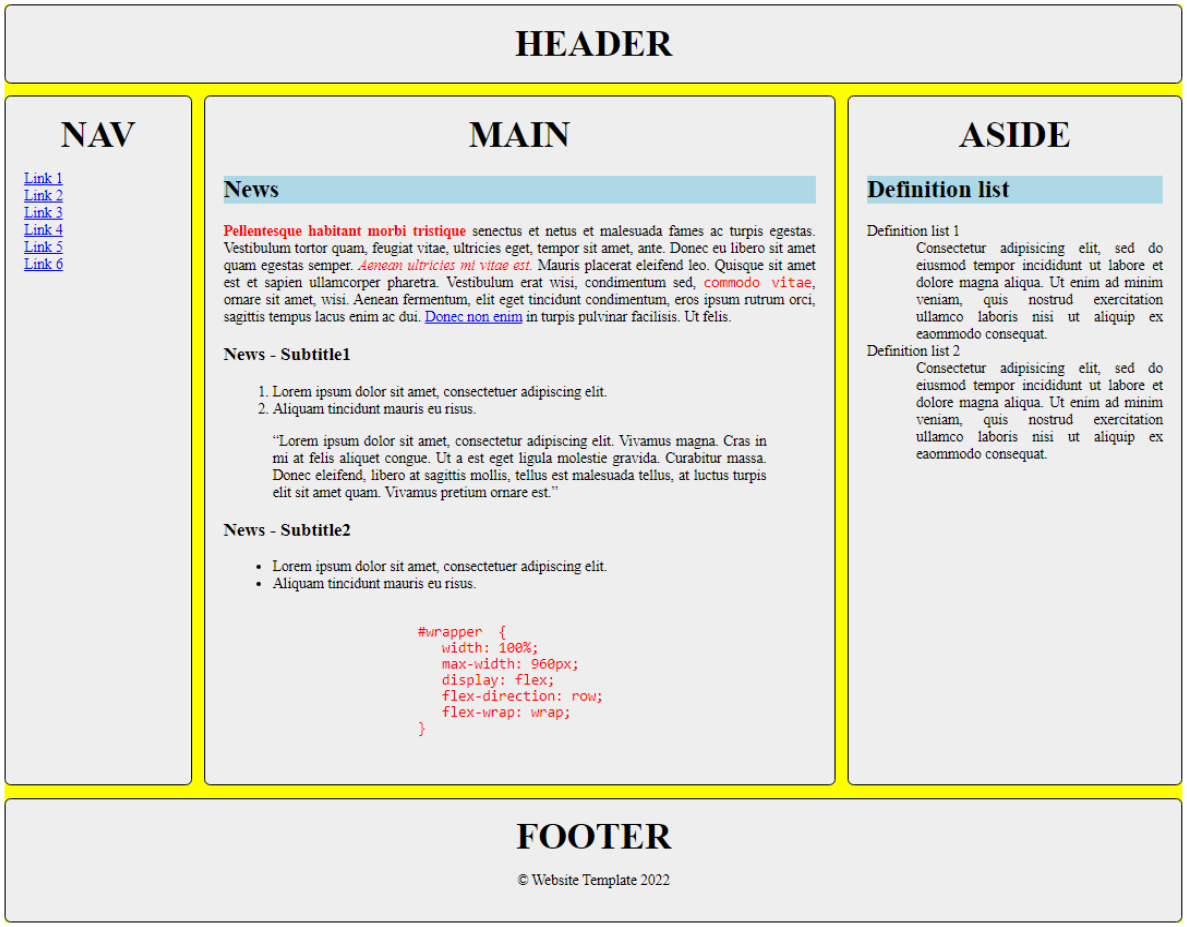
Thiết kế giao diện footer như hình dưới:



Hình 2.13: Bài tập thiết kế giao diện footer

Bài tập 5

Thiết kế giao diện trang web như hình dưới:

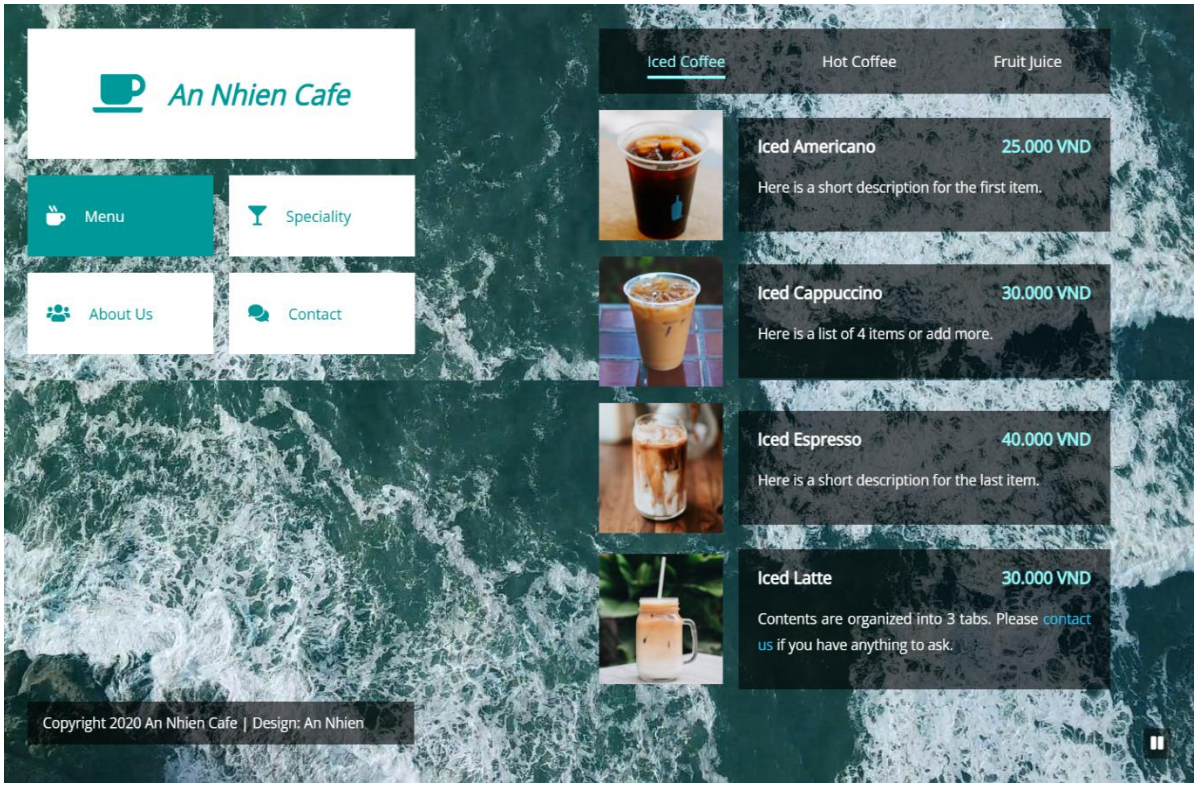


Hình 2.14: Bài tập thiết kế giao diện trang chủ của layout-1

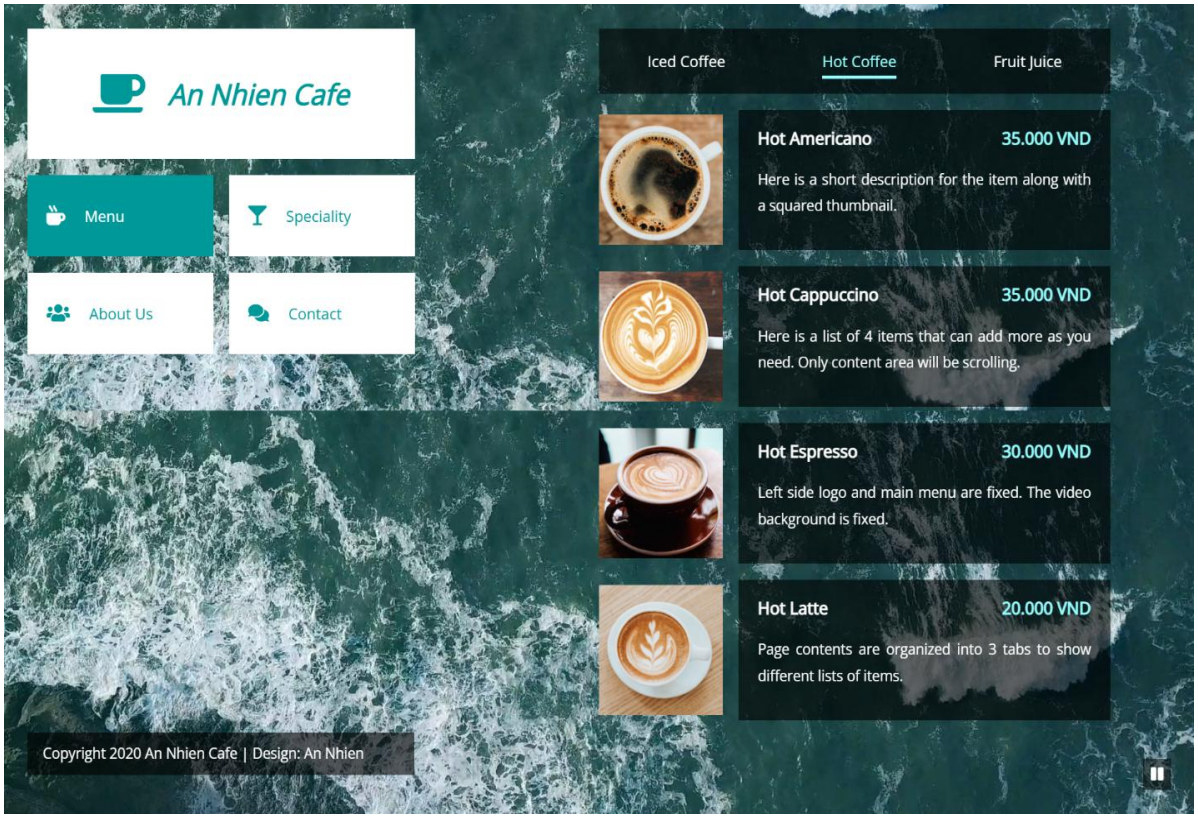
Bài tập 6

Thiết kế giao diện trang chủ và các trang con của website An Nhiên Café:

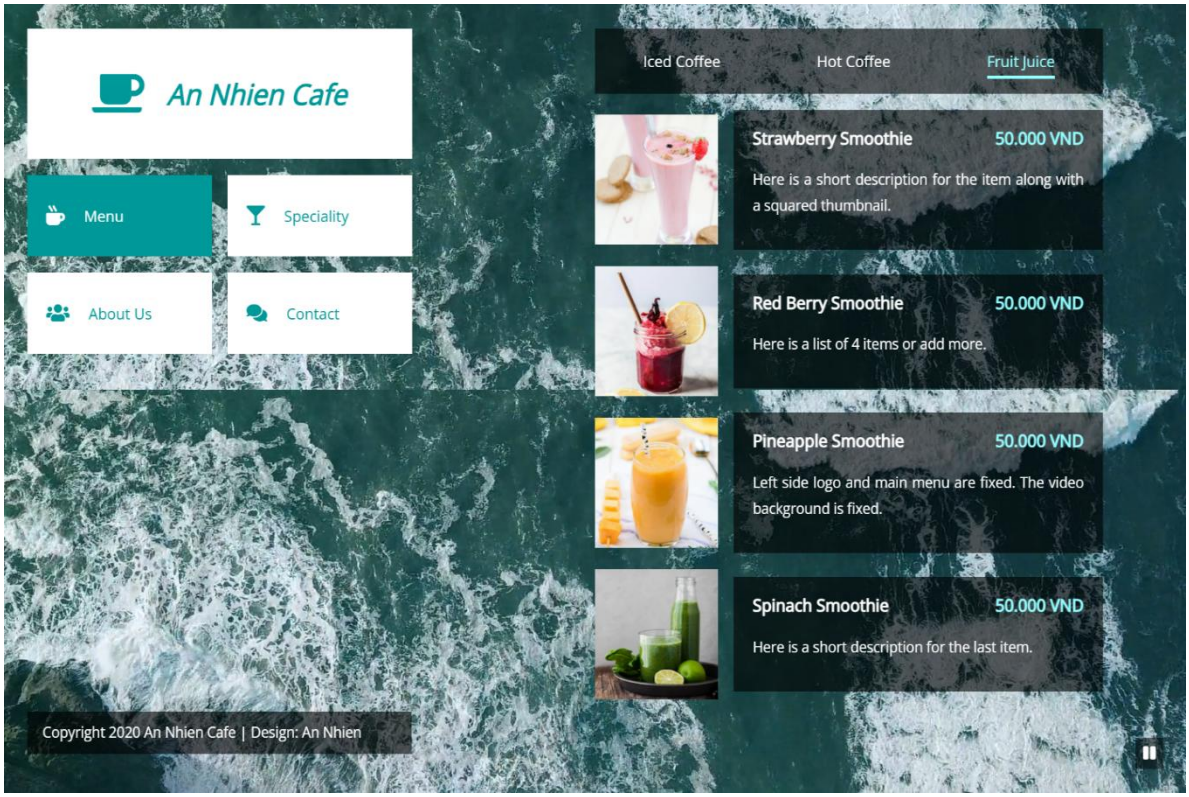
a. Thiết kế trang chủ
b. Thiết kế trang Hot Coffee
c. Thiết kế trang Fruit Juice
d. Thiết kế trang Speciality
e. Thiết kế trang About us
f. Thiết kế trang Contact



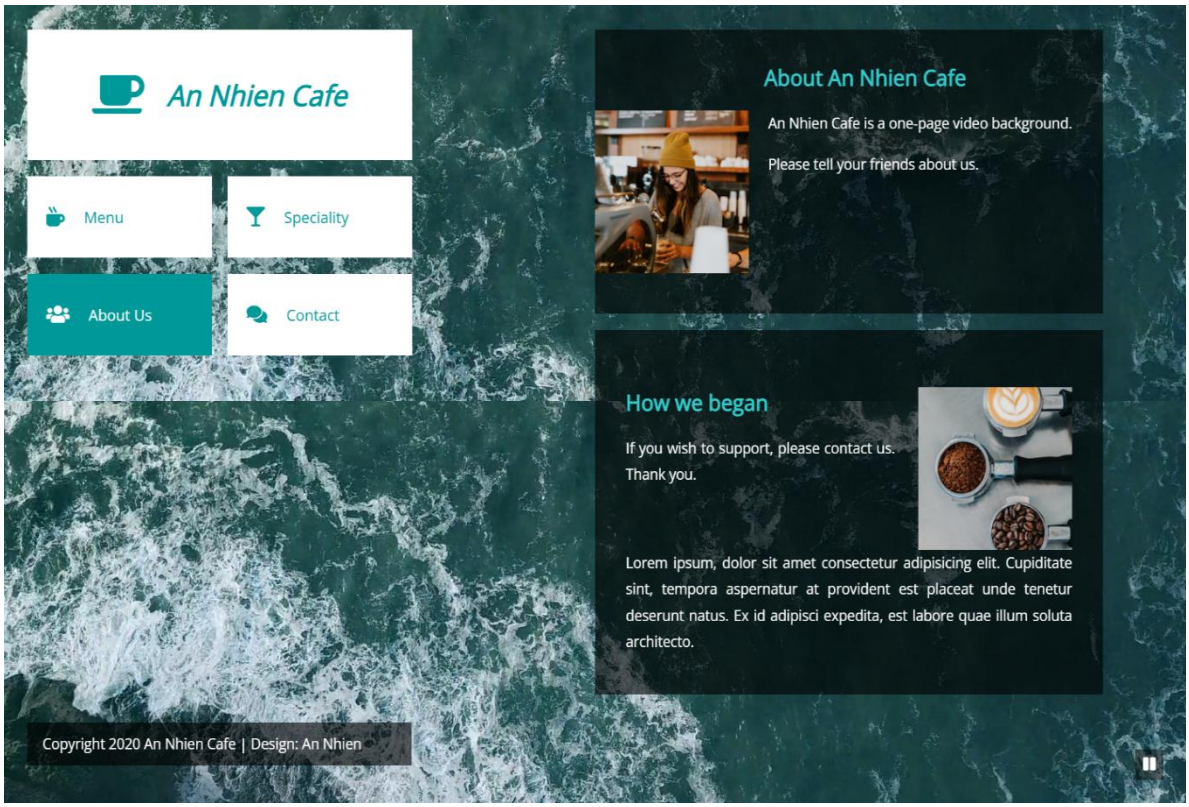
Hình 2.15: Bài tập thiết kế giao diện trang chủ của layout-2



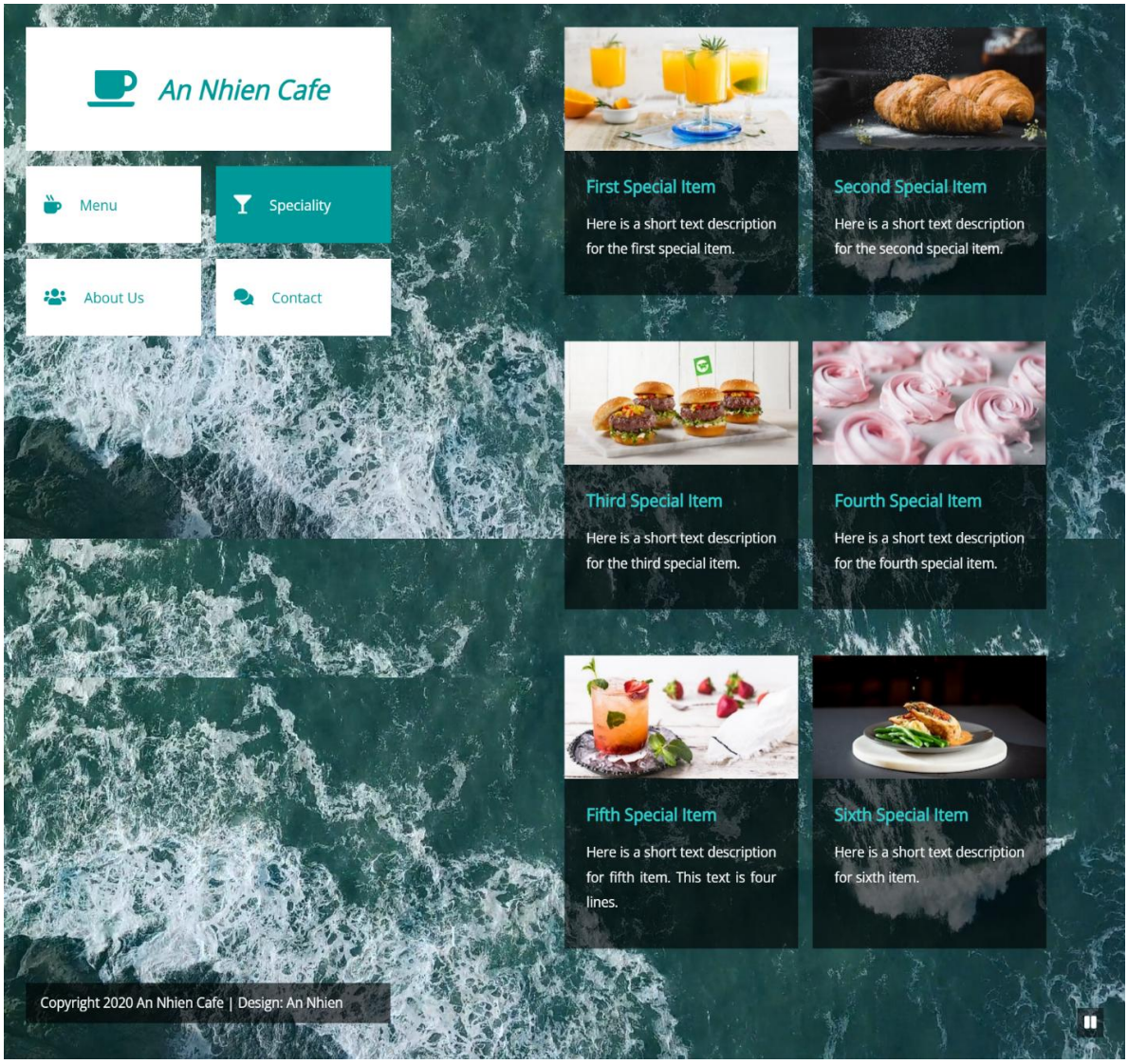
Hình 2.16: Bài tập thiết kế giao diện trang Hot Coffee của layout-2



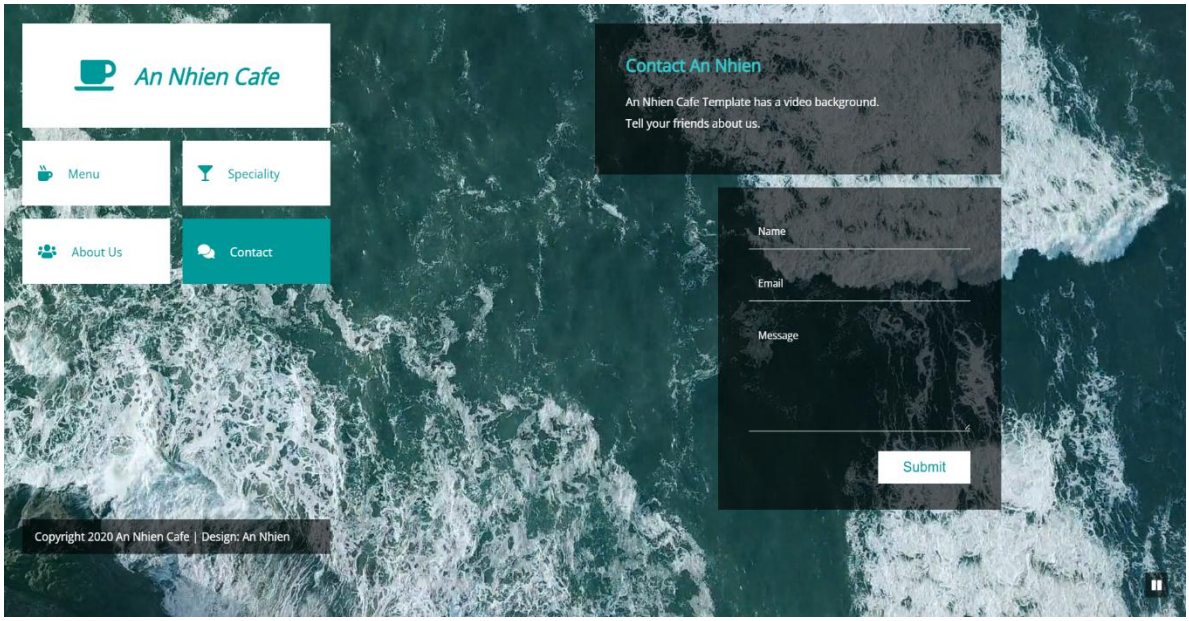
Hình 2.17: Bài tập thiết kế giao diện trang Fruit Juice của layout-2



Hình 2.18: Bài tập thiết kế giao diện trang About Us của layout-2



Hình 2.19: Bài tập thiết kế giao diện trang Speciality của layout-2



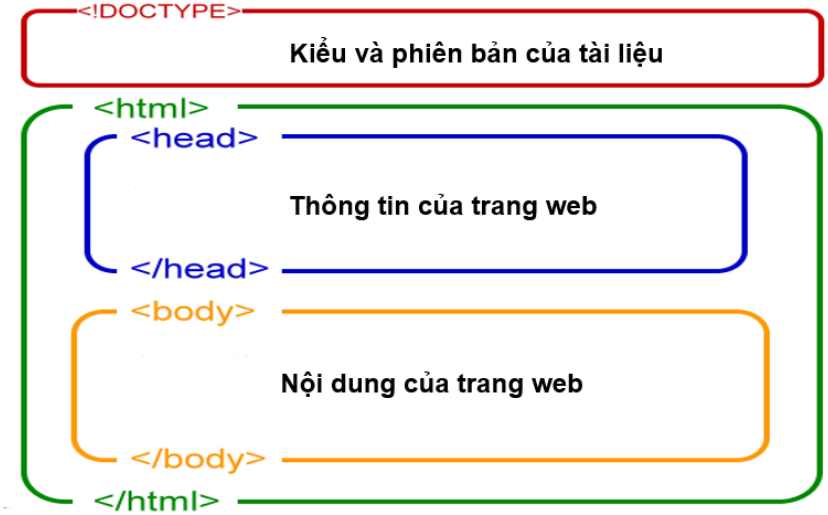
Hình 2.20: Bài tập thiết kế giao diện trang Contact của layout-2

CHƯƠNG 3. NGÔN NGỮ HTML

3.1. Tổng quan về HTML

- Là ngôn ngữ đánh dấu siêu văn bản, viết tắt từ Hyper Text Markup Language
- Thường dùng để tạo cấu trúc và nội dung của trang web.
- Một tài liệu HTML bao gồm nhiều thẻ. Mỗi thẻ HTML sẽ mô tả các thành phần khác nhau của tài liệu
- Phiên bản mới nhất hiện nay là HTML5 (2012)

3.2. Cấu trúc tài liệu HTML



Hình 3.1: Cấu trúc tài liệu HTML

```
<!DOCTYPE html>
<html>
    <head>
       <title>Tiêu đề trang web</title>
       <meta charset="utf-8"/>
       <link rel="icon" type="image/x-con" href="/images/favicon.ico">
       <link rel="stylesheet" href="styles.css">
       <script src="myScript.js"></script>
       <script>
               Mã nguồn javascript
       </script>
    </head>

    <body>
       <!--Chú thích-->
       <h1>Nội dung trang web</h1>
    </body>
</html>
```

3.2.1. Khai báo Doctype

Khai báo cho trình duyệt biết phiên bản HTML đang sử dụng, giúp hiển thị trang web một cách chính xác hơn.

Phiên bản	Khai báo Doctype
HTML5	<!DOCTYPE html>
HTML 4.01	<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
XHTML 1.1	<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN" "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

Bảng 3.1: Khai báo Doctype cho tài liệu

3.2.2. Thẻ <html>

- Cho trình duyệt biết đây là một tài liệu HTML
- Thuộc tính của thẻ:
  - **lang="mã-quốc-gia"**: ngôn ngữ được sử dụng chủ yếu trong trang.
    VD: lang=”en” – Tiếng Anh, lang=”vi” – Tiếng Việt
    Tham khảo mã quốc gia theo liên kết sau:
    https://www.w3schools.com/tags/ref_country_codes.asp

3.2.3. Thẻ <head>

- Cho biết thông tin của trang web như: ngôn ngữ chính (English, Chinese, French,.v.v), tập các ký tự, kết hợp với các tệp style sheet và script, thông tin tác giả, các từ khóa cho bộ máy tìm kiếm, nội dung phần này sẽ không hiển thị trên trình duyệt (trừ thẻ <title>).
- Các thẻ thường dùng trong phần <head>
  - <meta/>: mô tả trang, từ khóa, tác giả, sửa đổi lần cuối
  - <title>: tiêu đề trang web
  - <link/>: liên kết với các file bên ngoài
  - <style>: viết các đoạn mã css
  - <script>: viết các đoạn mã js (javascript) ở máy khách, hoặc để nhúng các file js từ bên ngoài vào trang html

Tên thẻ	Khai báo
<meta />	HTML5: <meta charset="UTF-8"/>
	<meta http-equiv="refresh" content="30"/>
	<meta name="keywords" content="Trường đại học sư phạm kỹ thuật, giáo dục, đào tạo, đại học, sư phạm"/>
	<meta name="description" content="Website Trường đại học Sư phạm Kỹ thuật, 48 Cao Thắng, Đà Nẵng" />

	<meta name="author" content="SPKT"/>
	<meta name="name" content="ute.udn.vn"/>
<title>	<title>Trang Chủ - Trường đại học Sư phạm Kỹ thuật - Đại học Đà Nẵng</title>
<style>	<style type="text/css"> body {background-color:yellow} </style>
<link />	<link rel="stylesheet" type="text/css" href="mystyle.css" />
<script>	<script src="scripts/core.js"></script>

Bảng 3.2: Các thẻ trong phần head

3.2.4. Thẻ <body>

Chứa nội dung chính của trang web

3.2.5. Chú thích trong HTML

<!-- Nội dung chú thích -->

3.3. Các loại thẻ HTML

3.3.1. Khái niệm thẻ

- Thẻ (tag) dùng để tạo cấu trúc và nội dung tài liệu HTML
- Cú pháp: <tagName listProperties> Object </tagName>
- Trong đó:
  - tagName: Tên thẻ được viết thường, viết liền với dấu "<"
  - list Properties: Danh sách các thuộc tính của thẻ.
  - Object: Đối tượng cần định dạng trong trang web.
- Đặc điểm của thẻ:
  - Một thẻ thông thường sẽ gồm thẻ mở và thẻ đóng:
  - Có 1 số thẻ không có thẻ đóng: ,
, <hr />
  - Các thẻ có thể lồng vào nhau, theo nguyên tắc thẻ nào mở trước thì đóng sau
- Thuộc tính của thẻ
  - Cung cấp thêm các thông tin chi tiết hơn cho thẻ, luôn được đặt trong thẻ mở
  - Khai báo:
    - Thứ tự các thuộc tính trong 1 thẻ là tùy ý.
    - Giá trị các thuộc tính đặt trong cặp dấu nháy đơn ‘ ’ hoặc nháy đôi " "
    - Nếu có từ 2 thuộc tính trở lên thì mỗi thuộc tính cách nhau 1 khoảng trắng
      Ví dụ: Paragraph.
  - Phân loại thuộc tính:
    - Bắt buộc: , <a **href**="…"/>
    - Không bắt buộc: Paragraph

3.3.2. Phân loại thẻ

3.3.2.1. Phân loại theo đặc điểm

- **Block**: khối thường bắt đầu 1 dòng mới và hiển thị trên dòng riêng biệt.
  VD: div, header, main, footer, h1 …h6, p, ul, ol, li, …
- **Inline**: thành phần có thể ở cùng 1 dòng với thành phần khác.
  VD: img, a, …

3.3.2.2. Phân loại theo chức năng

- Thẻ phân chia bố cục trong trang: header, nav, main, aside, footer, section, div…
- Thẻ định dạng: strong, em, mark, …
- Thẻ tạo nội dung: h1…h6, p, ol, ul, li, a, img, audio, video, table, frame, form…
  Ngoài ra, thẻ span là thẻ dạng inline, dùng để làm hộp chứa cho text, thường kết hợp với style và class để định dạng riêng cho text.

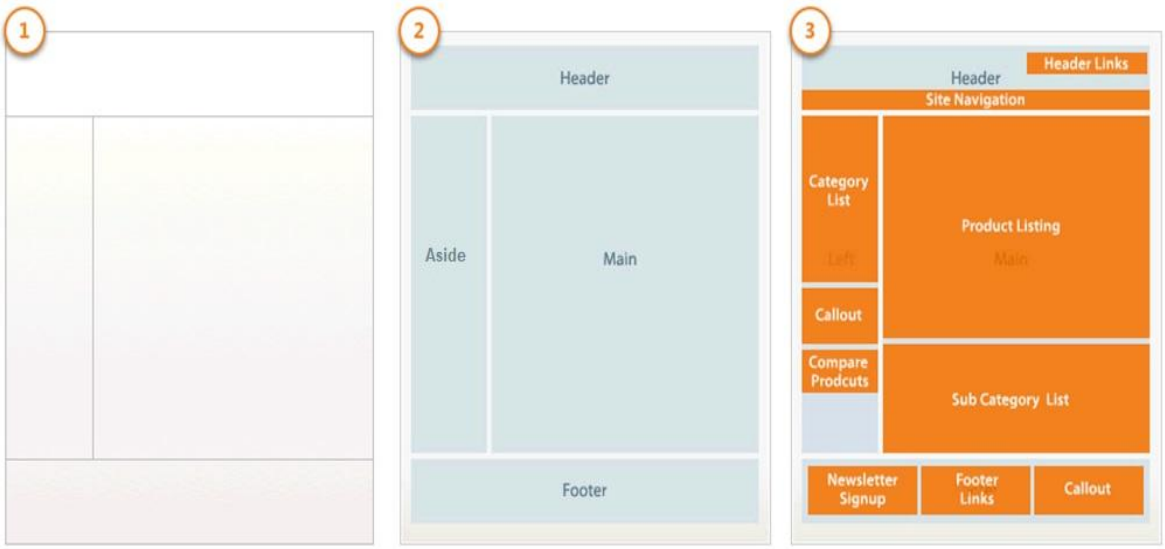
3.4. Các kí tự đặc biệt trong HTML

Tham khảo: https://www.w3schools.com/html/html_symbols.asp

Ký tự	Dạng số	Dạng tên	Mô tả
Ký tự toán học			
∀	∀	∀	Với mọi
∃	∃	∃	Tồn tại
∅	∅	∅	Tập rỗng
∈	∈	∈	Thuộc
∉	∉	∉	Không thuộc
∑	∑	∑	Tổng
Ký tự khác			
®	®	®	Thương hiệu đã đăng ký
©	©	©	Bản quyền
™	™	™	Thương hiệu
←	←	←	Mũi tên trái
↑	↑	↑	Mũi tên lên
→	→	→	Mũi tên phải
↓	↓	↓	Mũi tên xuống
&	&		và
<	&#060	<	dấu nhỏ hơn
>	&#062	>	dấu lớn hơn
…	…	…	dấu ba chấm

Bảng 3.3: Các kí hiệu đặc biệt

3.5. Thẻ tạo bố cục



Hình 3.2: Bố cục giao diện

Tên thẻ	Chức năng
header	Tạo tiêu đề của trang hoặc tiêu đề một khu vực (section) trong trang
nav	Tạo phần chứa liên kết điều hướng trang (thường là menu của trang)
aside	Tạo một khung bên cạnh nội dung
main	Nội dung chính của trang
article	Tạo một bài viết độc lập
section	Tạo một khu vực trong trang
div	Tạo một khối bao gồm các thẻ con bên trong
details	Tạo một nội dung và thông tin chi tiết về nội dung đó, thông tin chi tiết mà người dùng có thể xem hoặc ẩn đi
summary	Tạo tiêu đề cho nội dung trong thẻ details
footer	Tạo chân trang hoặc phần cuối của một khu vực

Bảng 3.4: Các thẻ tạo bố cục

- Thông thường, tạo ra 1 vùng lớn duy nhất bao quanh toàn bộ trang web chứa các thành phần trên (#wrapper)
- Phân biệt giữa 3 thẻ , <section>, <article>
  - <section>: một khối gồm các nội dung liên quan.
  - <article>: một khối gồm nội dung hoàn chỉnh, khép kín.
  - : một khối bao gồm các thẻ con bên trong.
- Các cách tạo bố cục cho trang web
  - Sử dụng framework
  - Sử dụng thuộc tính float, position
  - Sử dụng flexbox
  - Sử dụng grid
  - Kết hợp các cách trên

Ví dụ: bố cục trang web

```
<header>
 <h1>phần header</h1>
</header>
<nav>
 <h1>phần nav</h1>
</nav>
<main>
 <h1>phần main</h1>
</main>
<aside>
 <h1>phần aside</h1>
</aside>
<footer>
 <h1>phần footer</h1>
</footer>
```

Ví dụ: thẻ details và summary

```
<details>
  <summary>Epcot Center</summary>
  <p>Epcot is a theme park at Walt Disney World Resort featuring exciting
attractions, international pavilions, award-winning fireworks and
seasonal special events.</p>
</details>
```

3.6. Thẻ định dạng

Tên thẻ	Chức năng
b/strong	in đậm/quan trọng
i/em	in nghiêng/nhấn mạnh
small	chữ nhỏ
sup/sub	chỉ số trên/chỉ số dưới
ins	gạch chân
del	gạch ngang
mark	đánh dấu, tô màu chữ

Bảng 3.5: Các thẻ định dạng cho text

3.7. Thẻ tạo nội dung

3.7.1. Thẻ tiêu đề

- Bao gồm các thẻ: h1, h2, h3, h4, h5, h6
- Chức năng: tạo tiêu đề các bài viết, nhấn mạnh nội dung chính của chủ đề.

3.7.2. Thẻ tạo văn bản

- <p>: tạo đoạn văn
- <pre>: tạo đoạn văn bản giữ nguyên định dạng ban đầu (bài thơ,...)
- <code>: tạo mã nguồn minh họa
- <address>: thông tin liên lạc của tác giả hay chủ sở hữu của trang web

  Ví dụ:

```
<address>
   Địa chỉ: 48 Cao Thắng, TP. Đà Nẵng<br>
```

```
   Số điện thoại: (0236) 3822571 <br>
   Email: <a href=”mailto:dhspktdn@ute.udn.vn”>
   dhspktdn@ute.udn.vn</a>
</address>
```

- <cite>: tên 1 tác phẩm nổi tiếng (sách, bài hát, phim,…)
  Ví dụ:

```
<p><cite>The Scream</cite> by Edvard Munch. Painted in 1893.</p>
```

- <abbr>: nội dung đầy đủ của từ viết tắt các tổ chức, thuật ngữ.
  Ví dụ:

```
<p>The <abbr title="World Health Organization">WHO</abbr> was founded in
1948.</p>
```

- <bdo>: đảo ngược chiều hiển thị của văn bản
  Ví dụ:

```
<bdo dir="rtl">This line will be written from right to left</bdo>
```

- <q>, <blockquote>: tạo trích dẫn ngắn/dài
  Ví dụ:

```
<p>WWF's goal is to: <q>Build a future where people live in harmony with
nature.</q></p>
<p>Here is a quote from WWF's website:</p>
<blockquote cite="http://www.worldwildlife.org/who/index.html">
For 60 years, WWF has worked to help people and nature thrive. As the
world's leading conservation organization, WWF works in nearly 100
countries. At every level, we collaborate with people around the world to
develop and deliver innovative solutions that protect communities,
wildlife, and the places in which they live.
</blockquote>
```

-
: thẻ ngắt dòng
- <hr/>: thẻ tạo đường kẻ ngang

3.7.3. Thẻ hình ảnh

- <img>
  Ví dụ:

```
<img src="img.jpg" alt="hinh-anh" title="tieu-de-hinh-anh">
```

- <figure>
  Ví dụ:

```
<figure>
  <img src="img.jpg" alt="hinh-anh">
  <figcaption>Đoạn text để mô tả hình ảnh</figcaption>
</figure>
```

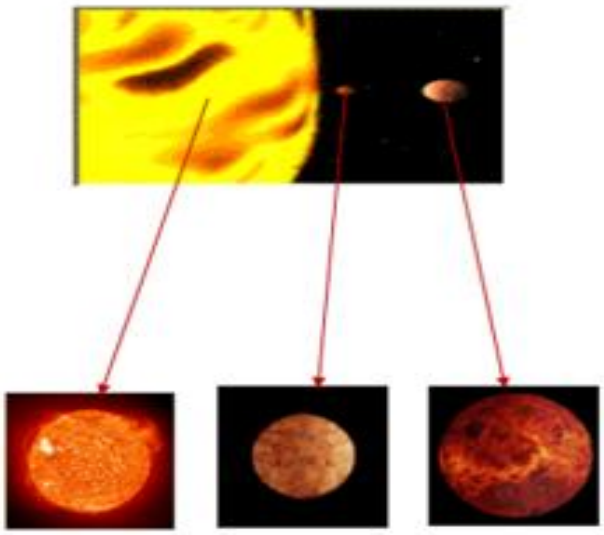
- <picture>: hiển thị các hình ảnh khác nhau trên các thiết bị có kích thước khác nhau
  Ví dụ:

```
<picture>
  <source media="(min-width: 650px)" srcset="car.jpg">
  <source media="(min-width: 465px)" srcset="food.jpg">
  <img src="flower.jpg">
</picture>
```

- <map>: Khai báo các vùng cần liên kết đến trong bản đồ bằng thẻ <area> nằm bên trong thẻ <map>

Ví dụ:

```
<img src="planets.gif" alt="planets" usemap="#planetmap"
<map name="planetmap">
 <area shape="rect"   coords="0,0,82,126" href="sun.html"/>
 <area shape="circle" coords="90,58,3" href="mercury.html"/>
 <area shape="circle" coords="124,58,5" href="venus.html"/>
</map>
```



Hình 3.3: Thẻ tạo bản đồ

3.7.4. Thẻ liên kết

- Cú pháp: …
- Chức năng:
  - Điều hướng trong cùng 1 trang
    Ví dụ: Tạo nút lệnh di chuyển về đầu trang, cuối trang
  - Điều hướng đến các trang khác nhau trong cùng 1 website
    Vi dụ: Các thành phần trong menu dùng để di chuyển đến các trang con.
  - Điều hướng đến các trang web khác
    Ví dụ: Tạo liên kết các trang thanh toán, liên kết đến các đối tác, …
  - Điều hướng để gởi mail
    Ví dụ: dhspktdn@ute.udn.vn
- Các trạng thái của liên kết: link, active, hover, visited
- Các thành phần có thể làm liên kết là text và hình ảnh

3.7.5. Thẻ danh sách

Tên danh sách	Ví dụ	Kết quả
Có thứ tự	 - Home - Products - About us - Contact 	1. Home 2. Products 3. About us 4. Contact

Không có thứ tự	 - Tea - Milk - Coffee - Fruit Juice 	• Tea • Milk • Coffee • Fruit Juice
Tự định nghĩa (tạo danh sách có các mục và có nội dung mô tả cho các mục)	<dl> <dt>Milk</dt> <dd>- white cold drink</dd> <dt>Coffee</dt> <dd>- black hot drink</dd> </dl>	Milk - white cold drink Coffee - black hot drink
Kết hợp	 - Coffee - Tea - Black tea - Green tea - Milk 	1. Coffee 2. Tea ○ Black tea ○ Green tea 3. Milk

Bảng 3.6: Các thẻ danh sách

3.7.6. Thẻ tạo bảng

- table: thead → th, tbody → tr → td
  - Các thuộc tính của thẻ table
    - *width*: thiết lập độ rộng của bảng
    - *border*: thiết lập viền của bảng
  - Các thuộc tính của thẻ th, td
    - *colspan*: nhóm cột
    - *rowspan*: nhóm hàng
  - Định dạng riêng cho các dòng hoặc cột
    - *tr:nth-child(n/even/odd)*: dòng thứ n hoặc chẵn hoặc lẽ
    - *td nth-child(n/even/odd)*: cột thứ n hoặc chẵn hoặc lẽ
- caption: tạo chú thích trong bảng, có thể đặt chú thích ở đầu hoặc cuối bảng.
  Ví dụ:

```
<table style="width:50%; text-align:left;">
<caption>Bảng điểm sinh viên</caption>
<tr>
    <th>Họ tên</th>
    <th>Mã sinh viên</th>
```

```
    <th>Điểm</th>
  </tr>
  <tr>
    <td>Anh</td>
    <td>Hồng</td>
    <td>Minh</td>
  </tr>
  <tr>
    <td>9</td>
    <td>7</td>
    <td>5</td>
  </tr>
</table>
```

Kết quả

Bảng điểm sinh viên

Họ tên	Mã sinh viên	Điểm
Anh	Hồng	Minh
9	7	5

Hình 3.4: Thẻ tạo bảng biểu

- colgroup: nhóm nhiều cột để định dạng giống nhau, kết hợp theo sau là thẻ <col>
  Ví dụ:

```
<colgroup>
  <col span="2" style="background-color: #D6EEEE">
</colgroup>
```

3.7.7. Thẻ đa phương tiện

- audio: chèn âm thanh, âm nhạc (.mp3, .wav, .oga)
  - Các thuộc tính của thẻ audio
    - *controls*: hiển thị thanh điều khiển
    - *autoplay*: tự động phát audio
    - *loop*: tự động lặp lại
    - *muted*: tắt âm thanh
  - Các thuộc tính của thẻ source
    - *src*: đường dẫn đến file
    - *type*: qui định cách thức để truyền dữ liệu
      - .mp3 → audio/mpeg
      - .wav → audio/wav
      - .ogg → audio/ogg
    - *preload*: đối với các tập tin có dung lượng lớn, để load nhạc nhanh
      Ví dụ:

```
<audio controls autoplay muted >
       <source src="intro.ogg" type="audio/ogg">
       <source src="intro.mp3" type="audio/mpeg">
       <p>Trình duyệt không hỗ trợ thẻ audio.</p>
</audio>
```

- video : chèn video, phim (.mp4, .webm, .ogg)
  - Các thuộc tính của thẻ video
    - *controls*: hiển thị thanh điều khiển
    - *autoplay*: tự động phát audio
    - *loop*: tự động lặp lại
    - *muted*: tắt âm thanh
    - *width*: độ rộng thanh điều khiển
    - *height*: chiều cao thanh điều khiển
    - *poster*: hình ảnh thay thế nếu video không tải lên được
  - Các thuộc tính của thẻ source: giống thẻ source của audio
    - *src*: đường dẫn đến file
    - *type*: qui định cách thức để truyền dữ liệu
      - .mp4 → video/mp4
      - .webm → video/webm
      - .ogg → video/ogg
    - *preload*: đối với các tập tin có dung lượng lớn, để load nhạc nhanh
  - Các thuộc tính của thẻ track: tạo phụ đề, chữ chạy khi phát video.
    - *src*: đường dẫn đến file
    - *default*: ngôn ngữ mặc định hiển thị của phụ đề.
    - Tạo file phụ đề có phần mở rộng là.vtt
      Ví dụ: file phude-vi.vtt

```
WEBVTT
00:00:00.500 --> 00:00:02.000
Chào mừng bạn đến với trường Đại học Sư phạm Kỹ thuật!
00:00:02.000 --> 00:00:05.500
Trường gồm có <u>18 ngành học.</u>
```

Ví dụ: thẻ video

```
<video controls autoplay muted >
   <source src="intro.ogg" type="video/ogg">
   <source src="intro.mp4 type="audio/mp4">
   <track src="file/phude-vi.vtt" label="Tiếng Việt" default>
   <track src="file/phude-en.vtt" label="Tiếng Anh">
   <p>Trình duyệt không hỗ trợ thẻ video.</p>
</video>
```

- object: thêm các plug-ins (Java applets, Microsoft ActiveX controls, Flash movies, maps, scan for viruses)
- embed: chèn hình hoặc nhúng các trang web khác vào website.
  Ví dụ:

```
<embed src="flower.jpeg">
<embed width="100%" height="500px" src="data.html">
```

3.7.8. Thẻ tạo khung

- Chèn video youtube
  Ví dụ:

```
<iframe width="420" height="345"
src="https://www.youtube.com/embed/tgbNymZ7vqY">
</iframe>
```

- Chèn 1 trang web khác vào website

  Ví dụ:

```
// Cách 1
<iframe src=" https://ute.udn.vn/default.aspx " height="300px"
width="100%" title="Demo Iframe"></iframe>
// Cách 2
<iframe src="demo-iframe.html" name="ute" height="300px" width="100%"
title="Demo Iframe"></iframe>
<p><a href="https://ute.udn.vn/default.aspx"
target="ute">ute.udn.vn</a></p>
```



ute.udn.vn

Hình 3.5: Thẻ iframe

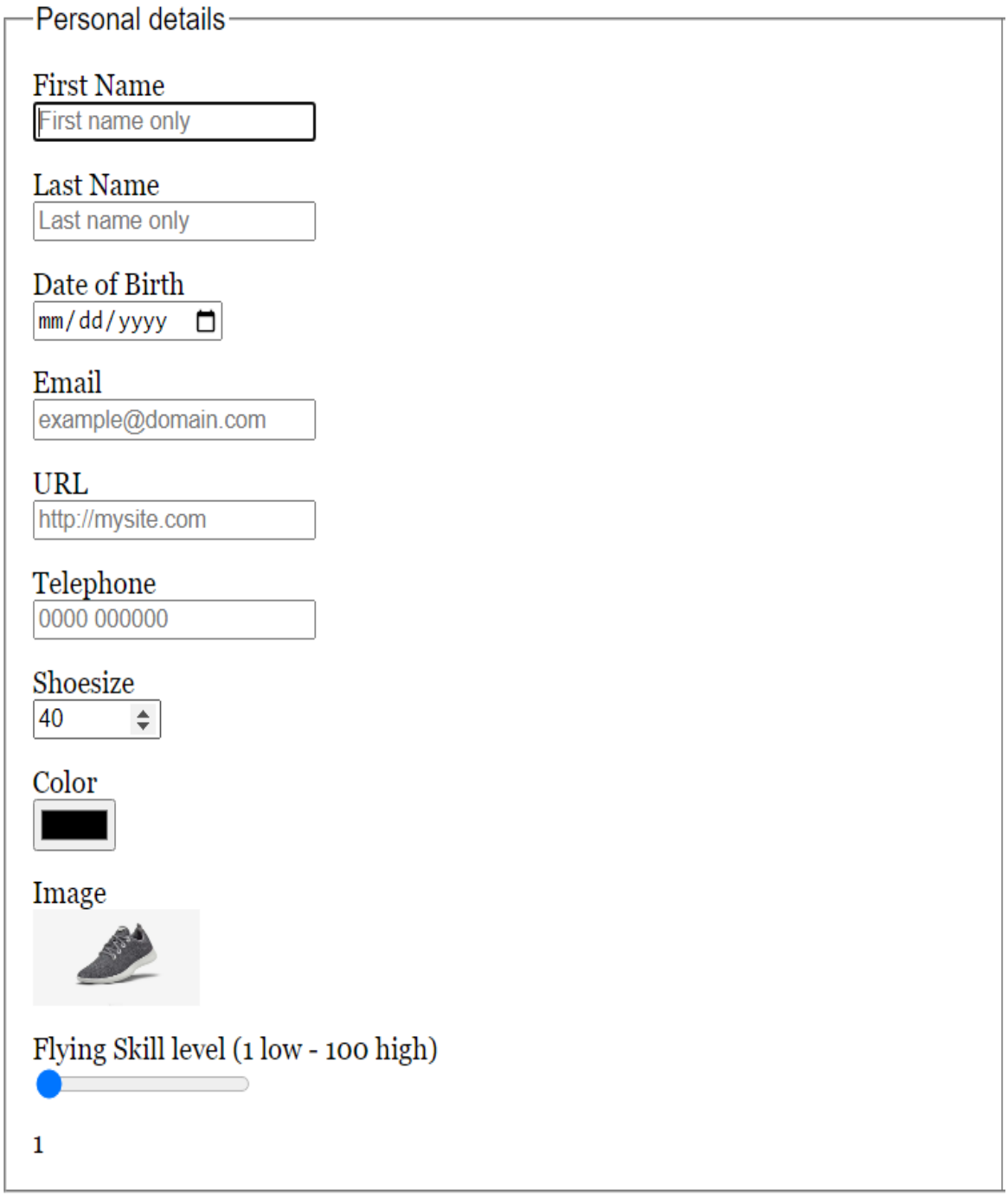
3.7.9. Thẻ tạo biểu mẫu

- Chức năng:
  - Tạo công cụ tìm kiếm cho website.
  - Thu thập thông tin để đặt mua hàng.
  - Thu thập thông tin để người dùng đăng ký một dịch vụ hay sự kiện nào đó.
  - Thu thập thông tin phản hồi để xây dựng mối quan hệ với khách hang và cải tiến dịch vụ.
- Các thuộc tính của form
  - name: tên form
  - action="url": định nghĩa hành động sẽ được thực hiện khi submit form
  - method (phương thức): post và get
  - post: không giới hạn dung lượng dữ liệu, thích hợp với dữ liệu nhạy cảm muốn che dấu

    VD: <form action=" /action_page.php" method="post">

- get: dữ liệu được đưa lên máy chủ và hiển thị trên URL, không thích hợp với dữ liệu nhạy cảm. URL có chiều dài tối đa khoảng 3000 ký tự nên phương thức này không dành để gửi dữ liệu lớn, thường dùng trong các truy vấn tìm kiếm.

- Các thành phần trong form



Hình 3.6: Thẻ form

STT	Thẻ	Chức năng
1	<input type=”…..”> • text • password • button • date	tạo các thành phần như: • textbox • mật khẩu • nút lệnh • ngày tháng năm

	• email • url • tel • number • color • range • image • checkbox • radio	• email • địa chỉ website • số điện thoại • số • màu • dãy số • hình ảnh • cho phép chọn nhiều • cho phép chọn 1
2	label	tạo nhãn
3	output	xuất kết quả tính toán
4	button	nút lệnh
5	textarea	tạo vùng nhập liệu nhiều dòng, mô tả
6	fieldset …legend	nhóm các thành phần có liên quan lại với nhau
7	select…option → optgroup	tạo menu hoặc danh sách thả xuống (dropdown list)
8	datalist…option	

Bảng 3.7: Các thành phần trong form

Ví dụ: form đăng nhập

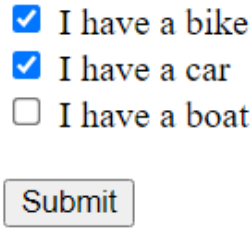
```
<form  action="mysuperscript.php" autocomplete="on">
  <h1>Log in</h1>
  <p>
    <label for="username">Username</label>
    <input type="text" id="username" name="username" required
    placeholder="email@mail.com"/>
  </p>
  <p>
    <label for="password">Password</label>
    <input type="password" id="password" name="password" required
    placeholder="abc@123" />
  </p>
  <p>
    <input type="checkbox" name="login" id="login" value="login" />
    <label for="loginkeeping">Keep me logged in</label>
  </p>
  <p>
    <input type="submit" value="Login" />
  </p>
  <p>
    Not a member yet ?
    <a href="#toregister">Join us</a>
  </p>
</form>
```

Ví dụ: checkbox

```
<form action="/action_page.php">
  <input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
```

```
  <label for="vehicle1"> I have a bike</label><br>
  <input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
  <label for="vehicle2"> I have a car</label><br>
  <input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
  <label for="vehicle3"> I have a boat</label><br><br>
  <input type="submit" value="Submit">
</form>
```

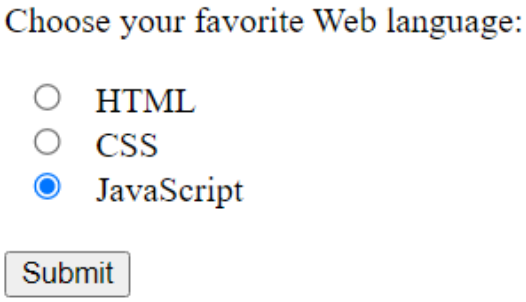
Kết quả:



Ví dụ: radio

```
<p>Choose your favorite Web language:</p>
<form>
  <input type="radio" id="html" name="fav_language"
  value="HTML">
  <label for="html">HTML</label><br>
  <input type="radio" id="css" name="fav_language"
  value="CSS">
  <label for="css">CSS</label><br>
  <input type="radio" id="javascript" name="fav_language"
  value="JavaScript" checked>
  <label for="javascript">JavaScript</label>
</form>
```

Kết quả:



Ví dụ:thẻ button

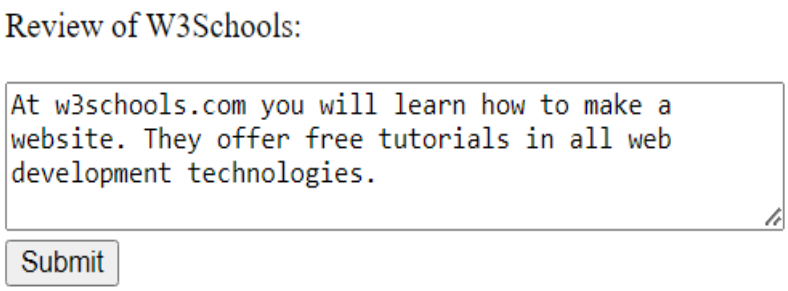
```
<button type="button" onclick="alert('Sign In!')">Sign In</button>
```

- Bên trong thẻ button có thể thêm text, các thẻ định dạng (strong, em, i, b).
- Khi muốn sử dụng 1 hình ảnh làm nút lệnh thì nên sử dụng thẻ button
- Thẻ <input type=”button”> không có 2 đặc tính như trên.

Ví dụ: thẻ textarea

```
<form action="/action_page.php">
  <p><label for="w3review">Review of W3Schools:</label></p>
  <textarea id="w3review" name="w3review" rows="4" cols="50">At
w3schools.com you will learn how to make a website. They offer free
tutorials in all web development technologies.</textarea>
  <br>
  <input type="submit" value="Submit">
</form>
```

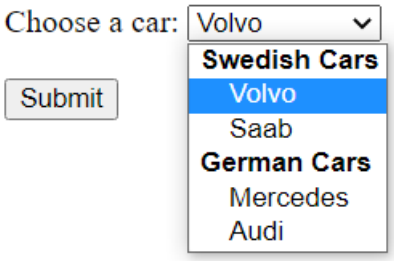
Kết quả:



Ví dụ: thẻ select…option → optgroup

```
<form action="/action_page.php">
  <label for="cars">Choose a car:</label>
  <select name="cars" id="cars">
    <optgroup label="Swedish Cars">
      <option value="volvo">Volvo</option>
      <option value="saab">Saab</option>
    </optgroup>
    <optgroup label="German Cars">
      <option value="mercedes">Mercedes</option>
      <option value="audi">Audi</option>
    </optgroup>
  </select>
  <br><br>
  <input type="submit" value="Submit">
</form>
```

Kết quả:



Ví dụ: thẻ datalist…option

```
<form action="/action_page.php" method="get">
  <label for="list-sach">Tìm kiếm sách thể loại: </label>
  <input id="list-sach" list="sach" placeholder="Nhập thể loại..." />
  <datalist id="sach">
    <option value="Tâm Lý">
    <option value="Kỹ Năng">
    <option value="Kinh Doanh">
    <option value="Y Học">
    <option value="Văn Học">
    <option value="Tiểu Sử">
    <option value="Tiểu Thuyết">
  </datalist>
</form>
```

Kết quả:



- Các sự kiện trong form

Tên sự kiện	Mô tả
onload	Diễn ra khi phần tử được tải
onunload	Diễn ra khi trang unload (phần tử body)
onreset	Diễn ra khi reset form
onsubmit	Diễn ra khi submit form
oninput	Diễn ra khi phần tử nhận giá trị đầu vào từ người dùng
onselect	Diễn ra khi phần tử được chọn
onchange	Diễn ra khi thay đổi nội dung phần tử trong form như khi chọn trong thành phần select/datalist, khi người dùng bấm radio, áp dụng cho các phần tử (input, select, textarea)
onblur	Diễn ra phi phần tử mất focus
onfocus	Diễn ra khi phần tử nhận focus (phần tử đang kích hoạt nhận dữ liệu)

Bảng 3.8: Các sự kiện liên quan đến form

Tên sự kiện	Mô tả
onkeydown	Diễn ra khi một phím được nhấn
onkeyup	Diễn ra khi một phím được nhả ra
onkeypress	Diễn ra khi một phím được nhấn và nhả ra

Bảng 3.9: Các sự kiện liên quan đến phím

Tên sự kiện	Mô tả
onmouseover	Diễn ra khi chuột di chuyển trên phần tử, hoặc phần tử con của phần tử
onmouseout	khi chuột di chuyển ra khỏi phần tử

onmousedown	Diễn ra khi bấm chuột trên phần tử
onmouseup	Diễn ra khi nhả bấm chuột trên phần tử
onmousemove	Diễn ra khi con trỏ chuột di chuyển
onscroll	Diễn ra khi cuộn thanh cuộn
onclick	Diễn ra khi bấm chuột vào phần tử
ondblclick	Diễn ra khi kích đôi chuột vào phần tử
ondrag	Diễn ra khi kéo phần tử

Bảng 3.10: Các sự kiện liên quan đến chuột

Tên sự kiện	**Mô tả**
onabort	Diễn ra khi một sự kiện bị hủy bỏ
oncanplay	Diễn ra khi một đa phương tiện có thể bắt đầu chạy, nhưng có thể phải dừng lại để buffer (ổn định lại xung lượng)
oncanplaythrough	Diễn ra khi một đa phương tiện có thể bắt đầu chơi đến hết, không phải dừng lại để buffer
ondurationchange	Diễn ra khi độ dài của một đa phương tiện được thay đổi
onemptied	Diễn ra khi một phần tử nguồn của đa phương tiện đột nhiên trở nên trống (bị ngắt mạng chẳng hạn).
onended	Diễn ra khi đa phương tiện chạy đến phần cuối (thêm lời cảm ơn: cảm ơn vì đã lắng nghe)
onerror	Diễn ra khi một lỗi (sự cố) Diễn ra
onloadeddata	Diễn ra khi dữ liệu đa phương tiện được tải
onloadedmetadata	Diễn ra trong quá trình dữ liệu đa phương tiện của phần tử media được tải
onloadstart	Diễn ra khi trình duyệt bắt đầu tải dữ liệu đa phương tiện
onpause	Diễn ra khi dữ liệu đa phương tiện bị dừng lại
onplay	Diễn ra khi dữ liệu đa phương tiện đang chuẩn bị để bắt đầu chạy
onplaying	Diễn ra khi dữ liệu đa phương tiện đang bắt đầu chạy
onprogress	Diễn ra khi trình duyệt đang nhập dữ liệu của đa phương tiện
onratechange	Diễn ra khi thay đổi tốc độ chạy của đa phương tiện
onreadystatechange	Diễn ra khi thay đổi trạng thái sẵn sàng
onseeked	Diễn ra khi thuộc tính tìm kiếm của một phần tử media là không còn đúng, và quá trình tìm kiếm đã kết thúc
onseeking	Diễn ra khi thuộc tính tìm kiếm của một phần tử media là đúng, và quá trình tìm kiếm đã bắt đầu
onstalled	Diễn ra khi có một lỗi trong quá trình nhập dữ liệu của đa phương tiện

onsuspend	Diễn ra khi trình duyệt đã đang nhập dữ liệu của đa phương tiện, nhưng dừng lại trước khi toàn bộ dữ liệu được nhập xong
ontimeupdate	Diễn ra khi đa phương tiện thay đổi vị trí chạy (người dùng chuyển sang vị trí khác)
onvolumechange	Diễn ra khi một đa phương tiện thay đổi âm lượng, cả khi âm lượng được thiết lập ở chế độ mute (tắt tiếng)
onwaiting	Diễn ra khi đa phương tiện đã dừng chạy, nhưng được mong đợi tiếp tục chạy lại

Bảng 3.11: Các sự kiện liên quan đến đa phương tiện

BÀI TẬP CHƯƠNG 3

Bài tập 1

Xây dựng menu có bố cục và nội dung như hình 2.6, 2.7, 2.8, 2.9, 2.10, 2.11

Bài tập 2

Xây dựng bài báo có bố cục và nội dung như hình 2.12

Bài tập 3

Xây dựng header có bố cục và nội dung như hình 2.13

Bài tập 4

Xây dựng footer có bố cục và nội dung như hình 2.14

Bài tập 5

Xây dựng thời khóa biểu có bố cục và nội dung như hình sau:

	Monday	Tuesday	Wednesday	Thursday	Friday
	BREAKFAST				
08.30 - 10.00	Development Management	Journalism	Outdoor & Teambuilding	Journalism	Global Challanges
	MORNING FELLOWSHIP				
11.00 - 12.30	Current Affairs	Innovativity	Creative Writing	Current Affairs	Innovativity
	LUNCH				
14.00 - 15.30	Environmental Studies	Movie Making		Environmental Studies	Movie Making
15.45 - 17.15	Sports	European Culture	Photography	Sports	European Culture
	DINNER				
19.00 - 20.30	Choir			Choir	
20.30 - 22.00	African Drum & Dance				

Hình 3.7: Bài tập html-5

Bài tập 6

Xây dựng các form sau:

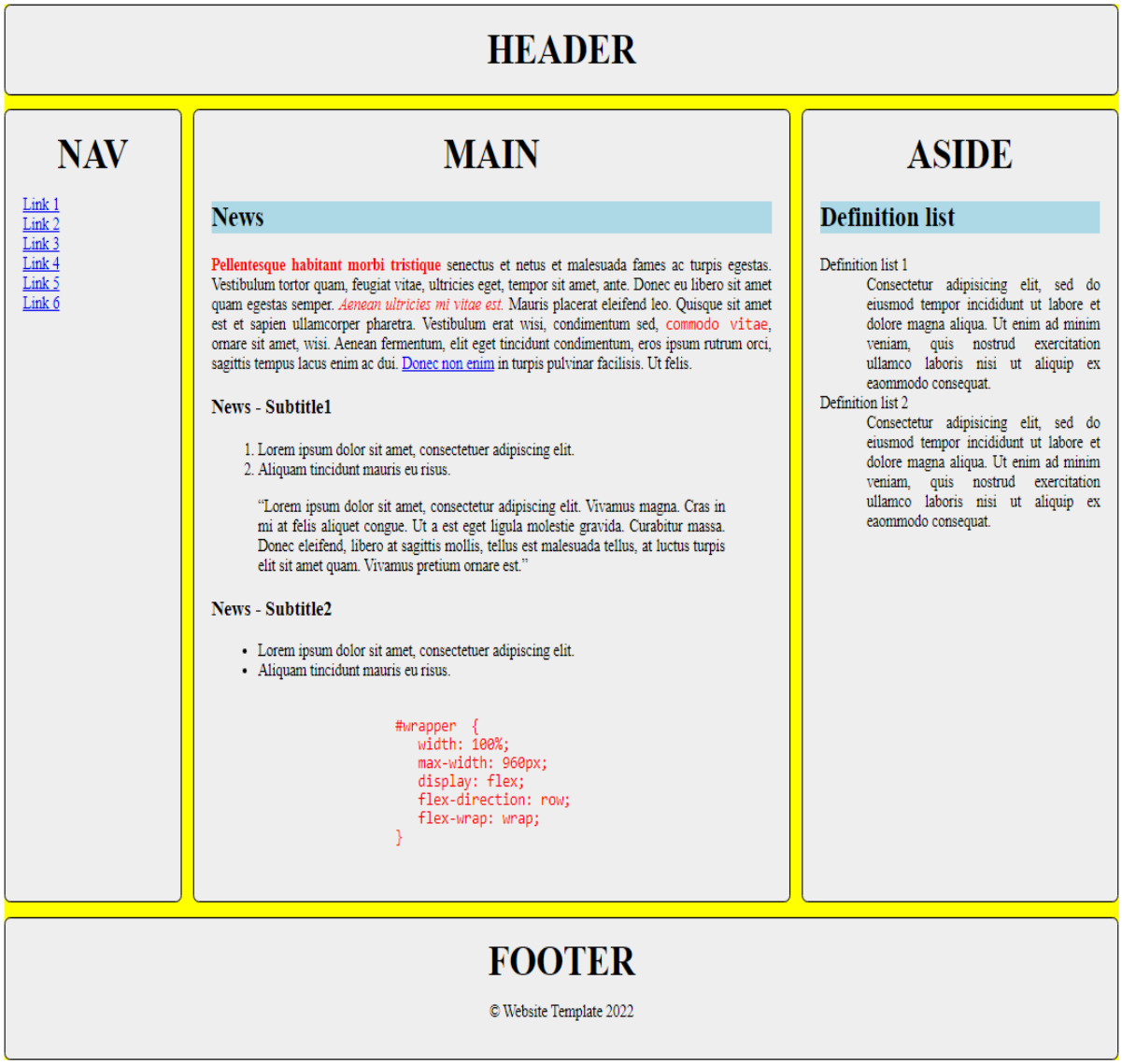
a. Form tìm kiếm
b. Form đăng nhập
c. Form đăng kí thành viên
d. Form liên hệ
e. Form đặt hàng

Bài tập 7

Xây dựng layout-1 có bố cục và nội dung như hình sau:

a. Bố cục chính

b. Bố cục và nội dung trong từng phần con: header, nav, main, aside, footer



Hình 3.8: Bài tập html-7

Bài tập 8

Xây dựng bố cục và nội dung cho các trang của website An Nhiên Café (layout-2):

a. Trang chủ (hình 2.16)

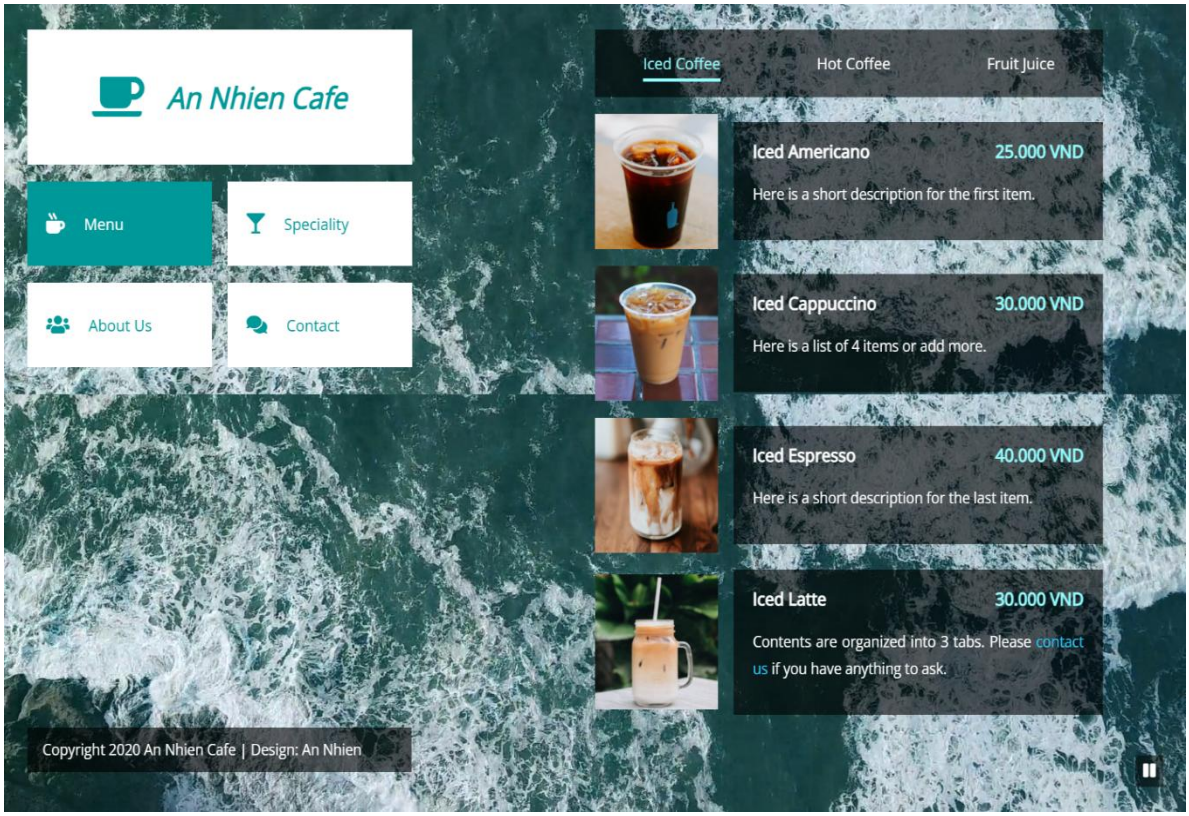
b. Trang Hot Coffee (hình 2.17)

c. Trang Fruit Juice (hình 2.18)

d. Trang Speciality (hình 2.19)

e. Trang About us (hình 2.20)

f. Trang Contact (hình 2.21)

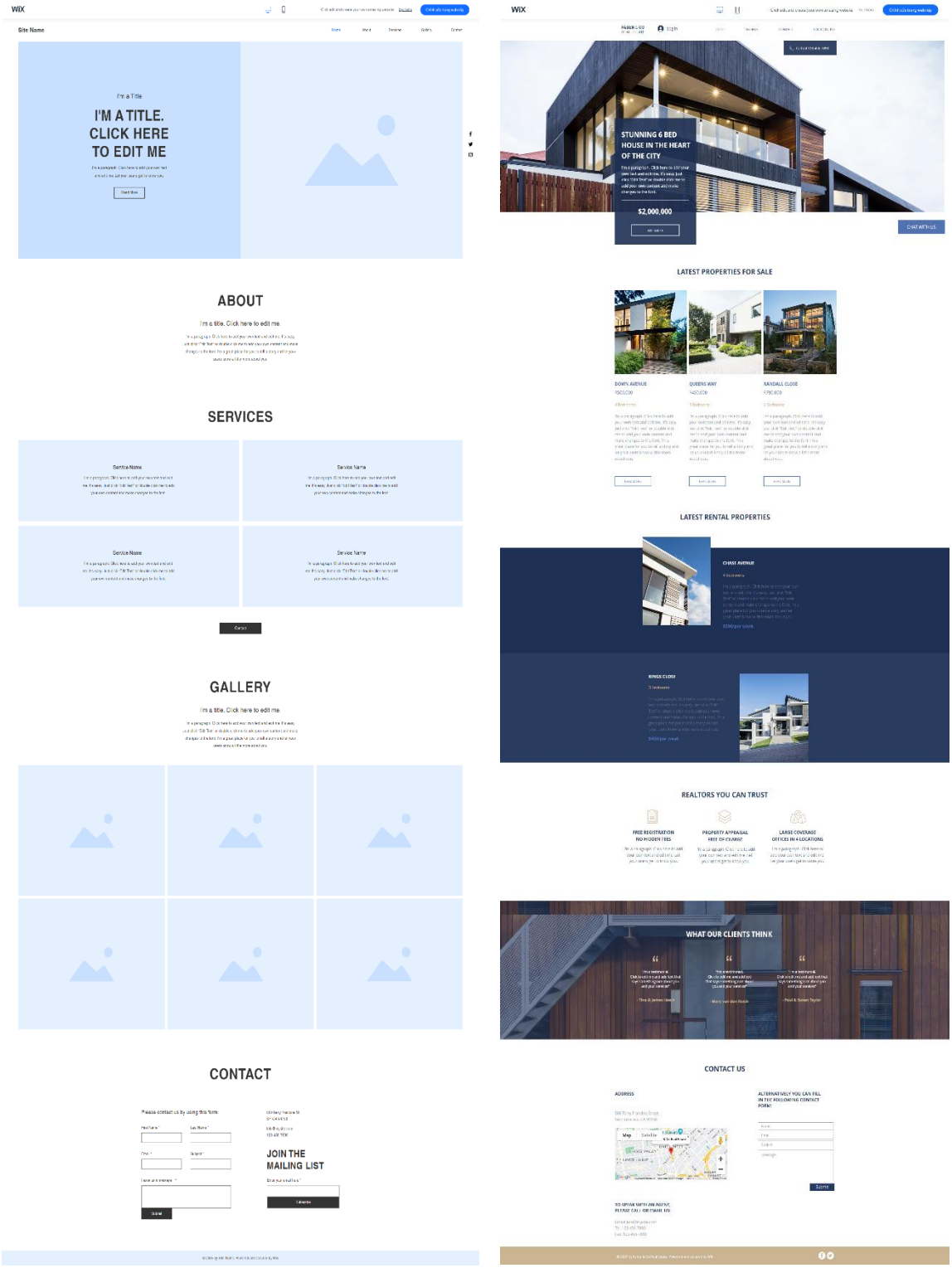


Hình 3.9: Bài tập html-8

CHƯƠNG 4. NGÔN NGỮ CSS

4.1. Tổng quan CSS

- CSS được phát triển bởi W3C vào năm 1996.
- CSS là ngôn ngữ định dạng xếp chồng, viết tắt từ **Cascading Style Sheets**
- CSS là ngôn ngữ tạo phong cách cho trang web.



Hình 4.1: Tổng quan CSS

4.2. Khai báo CSS

4.2.1. Nội dòng

- Khai báo trực tiếp tại thuộc tính style của các phần tử HTML.
- Ưu điểm: định dạng cho phần tử nhanh chóng.
- Nhược điểm: chỉ áp dụng cho 1 phần tử cụ thể, không áp dụng chung cho nhiều phần tử cùng 1 lúc.
- Ví dụ:

```
<body>
 <p style="color:red"> Đoạn văn </p>
</body>
```

4.2.2. Nội tuyến

- Khai báo trong thẻ style nằm trong phần <head> của trang HTML.
- Ưu điểm: code html và CSS được chia làm 2 phần rõ ràng.
- Nhược điểm: chỉ áp dụng được trong phạm vi một file cụ thể nơi mà nó được khai báo, không thể sử dụng chung cho nhiều trang khác nhau.
- Ví dụ:

```
<head>
 <style type="text/css">
       p { color:red; }
 </style>
</head>
```

4.2.3. Ngoại tuyến

- Khai báo trong file có phần mở rộng .css, nằm bên ngoài file HTML.
- Ưu điểm: dùng chung cho nhiều trang khác nhau của website, dễ dàng quản lý.
- Ví dụ:

```
<head>
 <link rel="stylesheet" type="text/css" href="style.css"/>
</head>
Hoặc
<style>
   @import url("/path/stylesheet.css");
// khai báo @import trước các khai báo css khác
</style>
```

4.2.4. Độ ưu tiên khai báo

- Mức 1: !important
- Mức 2: nội dòng
- Mức 3: nội tuyến
- Mức 4: ngoại tuyến

4.3. Đơn vị đo và màu sắc

4.3.1. Đơn vị đo

- Đơn vị tuyệt đối có giá trị cố định không thay đổi và không bị ảnh hưởng bởi các phần tử khác; thường áp dụng cho những thành phần có kich thước cố định, hoặc kích thước nhỏ không quá ảnh hưởng như border.
  - Một số đơn vị tuyệt đối: px, pt, cm, mm, in, pc

- Qui đổi: 1in=96px=2.54cm, 1px=1/96th của 1in, 1pt=1/72th của 1in, 1pc=12pt

Đơn vị tuyệt đối thường dùng là **px**. Pixels là khác nhau giữa các thiết bị. Với các thiết bị có độ phân giải thấp, 1px là một điểm ảnh hiển thị trên màn hình của thiết bị. Với máy in và các thiết bị có độ phân giải cao, 1px bao gồm nhiều điểm ảnh hiển thị trên màn hình của thiết bị.

- Đơn vị tương đối là đơn vị được tính dựa trên tỉ lệ so với phần tử cha của nó hoặc chính nó thông qua giá trị của thuộc tính font-size.. Đơn vị tương đối linh động và hoạt động tốt hơn cho những thiết bị, những màn hình có kích thước và độ phân giải khác nhau.

  Mức độ ưu tiên sẽ tính theo font-size của nó trước, nếu nó không thiết thuộc font-size thì lấy của cha trực tiếp. Nếu cha nó không có thì lấy tiếp cha của cha nó, đến cuối là root.

  - Một số đơn vị tương đối thường dùng: %, em, rem, vw, vh
    - %: phụ thuộc vào phần tử cha trực tiếp của nó
    - em: phụ thuộc vào nó hoặc phần tử cha của nó hoặc cha của cha nó và cuối cùng là root.
    - rem: phụ thuộc vào thẻ <html>
      - Mặc định: 100% = 16px = 1em
      - Nếu thiết lập font-size cho thẻ html là 62.5% hoặc 10px thì các phần tử nào sử dụng đơn vị rem sẽ được qui đổi 1rem = 10px
      - Nếu thiết lập font-size:20px cho phần tử cha thì các phần tử con sử dụng đơn vị em thì được qui đổi 1em=20px
    - vw (view width): tính theo tỉ lệ chiều rộng khung nhìn thiết bị.
    - vh (view height): tính theo tỉ lệ chiều cao khung nhìn thiết bị.
      - 1 vw = 1% chiều rộng view-port (kích thước cửa sổ trình duyệt)
      - 1 vh = 1% chiều cao view-port
    - vmin, vmax: không tỉ lệ theo 1 hướng mà theo cả 2, tuỳ thuộc vào độ lớn của chiều cao và chiều rộng màn hình.
      - 1 vmin = 1 vw hoặc 1 vh (Lấy giá trị nhỏ hơn)

      VD: màn hình có kích thước là 840×640 thì 1 vmin = 6.4px,
      nếu màn hình kích thước là 360×480 thì 1 vmin = 3.6px
      - 1 vmax = 1 vw hoặc 1 vh (Lấy giá trị lớn hơn)

      VD: màn hình có kích thước là 840×640 thì 1 vmax = 8.4px,
      nếu màn hình kích thước là 360×480 thì 1 vmax = 4.8px

4.3.2. Màu sắc

Có thể thiết lập màu sắc bằng các cách sau:

- Tên màu tiếng anh: white, blue, green, … (hỗ trợ 140 tên màu)
- Mã màu hệ hexa: #RRGGBB
  VD: #00FF33 (hoặc #0F3)
- Hàm rgb(red,geen,blue). Mỗi kênh màu red, green, blue có giá trị từ 0-255
  VD: rgb(0,50,255); rgb (20%,70%,40%)

Ngoài ra, có thể kết hợp giá trị opacity để thiết lập làm mờ hoặc rõ màu thêm; opacity có giá trị từ 0-1

VD: rgba(0,50,255,0.5); rgba(0,50,255,.5)

- Tham khảo bảng màu thiết kế: https://colorsui.com/

4.4. Bộ chọn – Selectors

4.4.1. Khái niệm bộ chọn

- Selector là có thể là phần tử, một bộ phận của phần tử, trạng thái, thuộc tính của phần tử được áp dụng định dạng.

4.4.2. Khai báo bộ chọn



Hình 4.2: Khai báo bộ chọn

4.4.3. Các loại bộ chọn

4.4.3.1. Simple selectors

- Element: header, footer, h1, p, a, …
- ID: tự định nghĩa, chỉ được áp dụng duy nhất 1 lần
- Class: tự định nghĩa, có thể áp dụng nhiều lần cho nhiều phần tử khác nhau.
- Ngoài ra, có thể kết hợp 3 loại trên với nhau
  - E#Id, E.Class
  - E #Id, E .Class
  - E1, E2, E3, …
  - *: đại diện cho tất cả elements

4.4.3.2. Combinators selectors

- descendant selector (space): con, cháu, …
- child selector (>): con trực tiếp
- adjacent sibling selector (+): anh em liền kề
- general sibling selector (~): anh em

4.4.3.3. Pseudo-elements selectors

- ::first-letter
- ::first-line
- ::marker
- ::selection
- ::after
- ::before

4.4.3.4. Pseudo-class selectors

- :link, :visited, :hover, :active
- :required, :checked, :focus,
- :enabled, :disabled, :valid, :invalid, :read-only, :read-write
- :first-child, :last-child, :nth-child(n), :nth-last-child(n)

4.4.3.5. Attribute selectors

Tên bộ chọn	Minh họa	Công dụng
[attribute]	[title]	có thuộc tính title
[attribute = ”value”]	[title=”hoa”]	chứa giống y giá trị “hoa” VD: hoa
[attribute ~= ”value”]	[title ~= ”hoa”]	có chứa từ hoa đứng riêng biệt VD: hoa hồng, hồng hoa
[attribute *= ”value”]	[title *= ”hoa”]	có chứa cụm từ hoa ở bất cứ vị trí nào. VD: hoa hồng, hhoahồng
[attribute |= ”value”]	[title |= ”hoa”]	bắt đầu bằng hoa- VD: hoa-hồng, hoa- hồng
[attribute ^= ”value”]	[title ^= ”hoa”]	bắt đầu bằng hoa VD: VD: hoa-hồng, hoahồng
[attribute $= ”value”]	[title $= ”hoa”]	kết thúc bằng hoa VD: hồng hoa, hồng-hoa, hồnghoa

Bảng 4.1: Các bộ chọn dựa vào thuộc tính

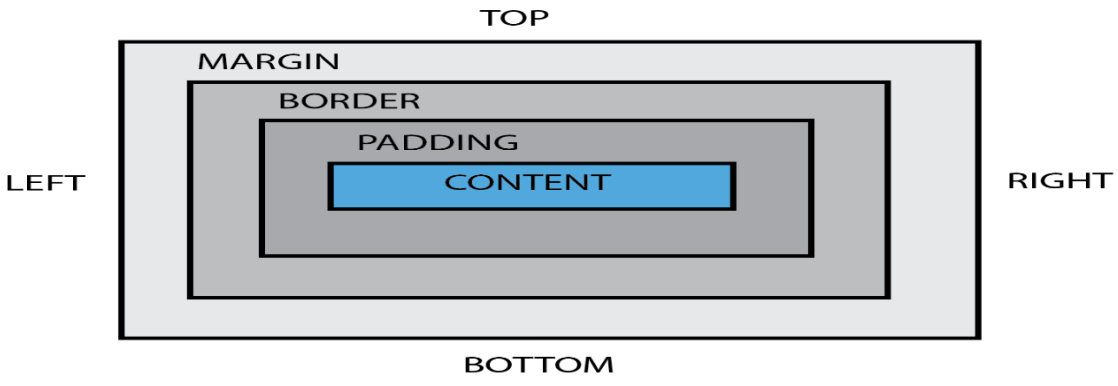
4.4.3.6. Độ ưu tiên của các loại bộ chọn

- !important
- Inline → 1000
- Tag#ID hoặc Tag.Class → 111
- #ID → 100
- .Class → 10
- Tag → 1
- * → 0

4.5. Mô hình hộp

4.5.1. Khái niệm

Mọi thành phần trong trang web (bao gồm cả block và inline) đều được tạo ra bởi mô hình hộp như sau:



Hình 4.3: Mô hình hộp

- **content**: vùng chứa nội dung của một phần tử, với chiều rộng/cao được xác định qua thuộc tính width và height; vùng này thường chứa text, hình ảnh, video…
- **padding**: cho biết độ rộng của vùng padding bao quanh vùng content
- **border**: cho biết độ rộng và kiểu của đường viền bao quanh vùng padding
- **margin**: cho biết độ rộng của vùng margin bao quanh vùng border

4.5.2. Các thuộc tính trong mô hình hộp

- width: độ rộng của phần tử
  - max-width: Một phần tử block luôn chiếm toàn bộ chiều rộng có sẵn. Thiết lập width sẽ ngăn không cho nó kéo dài ra các cạnh của vùng chứa nó. Nếu cửa sổ trình duyệt nhỏ hơn chiều rộng của phần tử thì trình duyệt sẽ xuất hiện thanh cuộn ngang. Để khắc phục vấn đề này thì nên thiết lập thêm thuộc tính max-width cho phần tử.
  - min-width: Thiết lập "chiều rộng tối thiểu" cho phần nội dung của phần tử, nghĩa là chiều rộng phần nội dung của phần tử sẽ không bao giờ ngắn hơn giá trị của thuộc tính này.
    - Ví dụ 1: khai báo cả 2 thuộc tính min-width và width cho phần tử

```
.box1{ width: 200px; min-width:300px; }
// box1 sẽ có độ rộng khi hiển thị là 300px
.box2{ width: 400px; min-width:300px; }
// box2 sẽ có độ rộng khi hiển thị là 400px
```

    - Ví dụ 2: chỉ khai báo thuộc tính min-width cho phần tử. Nếu chiều rộng phần nội dung của cha nó lớn hơn chiều rộng của nó thì chiều rộng của nó sẽ tự động đạt kích cỡ tối đa dựa theo chiều rộng phần nội dung của cha nó

```
.box1{ min-width:300px; background: yellow; }
// box1 có nền vàng 100% độ rộng của phần tử cha đang chứa nó
.box2{ min-width:300px; background: yellow; width: 400px; }
// box2 có nền vàng là 400px
```

- height / min-height / max-height: độ cao của phần tử
- padding: tạo khoảng cách giữa nội dung so với đường viền
  - Cú pháp: padding: top right bottom left;
  - Các thuộc tính con: padding-top, padding-right, padding-bottom, padding-left

```
p{ padding: 20px;}             /* T/R/B/L */
p{ padding: 10px 20px 0px 10px; } /* T R B L */
p{ padding: 10px 20px 0px; }    /* T R/L B */
p{ padding: 10px 20px }         /* T/B R/L */
```

- margin: tạo khoảng cách đến các phần tử khác, không được tính vào chiều rộng của phần tử
  - Cú pháp: margin: top right bottom left;
  - Các thuộc tính con: margin-top, margin-right, margin-bottom, margin-left

```
p{ margin: 20px;}              /* T/R/B/L */
p{ margin: 5px 10px 20px 30px; }      /* T R B L */
p{ margin: 10px 0px 20px; }           /* T R/L B */
p{ margin: 10px 20px }         /* T/B R/L */
```

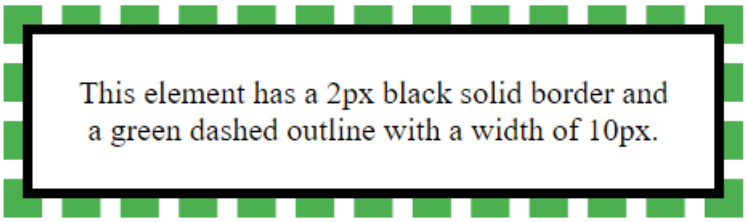
- border: tạo viền của phần tử
  - Cú pháp: border: width style color;

- Các thuộc tính con: border-top, border-right, border-bottom, border-left

```
p{ border: 2px solid green;}
```

- outline: là một đường được vẽ xung quanh các phần tử, bên ngoài của đường viền border, để làm cho phần tử "nổi bật". Nó có thể chồng lên nội dung khác. Ngoài ra, outline không phải là một phần kích thước của phần tử nên tổng chiều rộng và chiều cao của phần tử không bị ảnh hưởng bởi độ dày của outline.

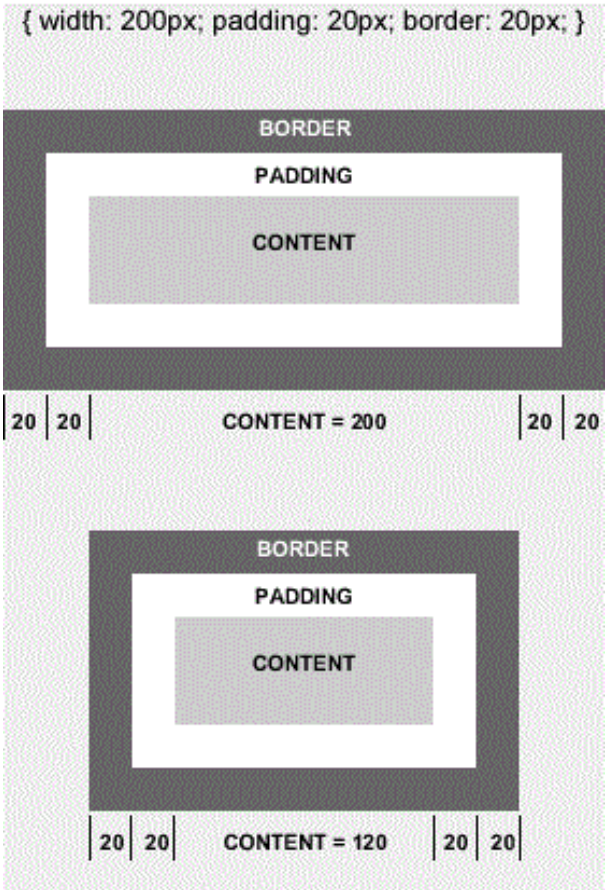
```
p{ outline: thick ridge pink;} /* outline: width style color; */
```



Hình 4.4: Thuộc tính outline

4.5.3. Một số kỹ thuật trong mô hình hộp

- Khi thay đổi giá trị của padding hoặc border có thể sẽ làm tăng chiều rộng ban đầu của phần tử và dẫn đến giao diện bị xê dịch. Để khắc phục vấn đề này thì nên sử dụng thuộc tính **box-sizing**. Thuộc tính này có các giá trị sau:
  - content-box : width = content (mặc định)
  - padding-box: width = content + padding R/L)
  - border-box : width = content + border R/L + padding R/L) (thường dùng)



Hình 4.5: Thuộc tính box-sizing

- Nếu một phần tử có kích thước cố định và nội dung bên trong của nó vượt ra ngoài kích thước đó, thì "overflow" cho phép thiết lập nội dung đó có bị ẩn đi hoặc tràn ra ngoài phần tử bao quanh nó. Thuộc tính này có các giá trị sau:
  - visible: mặc định, phần nội dung bị tràn ra sẽ nằm đè lên các phần tử khác.
  - hidden: ẩn phần nội dung vượt quá chiều cao hoặc chiều rộng của phần tử.
  - scroll: trình duyệt sẽ có thêm thanh cuộn để có thể kéo xem phần nội dung bị ẩn đi. Thanh cuộn này được thêm vào cho cả chiều dọc và chiều ngang của phần tử.
  - auto: tự động hiện thanh cuộn dọc nếu nội dung dài hơn so với phần tử
- Để kiểm soát bố cục của phần tử hay chỉ định cách phần tử hiển thị thì nên sử dụng thuộc tính display. Thuộc tính này có các giá trị sau:
  - none
  - inline/ block/ inline-block/ list-item
  - flex/ inline-flex
  - grid/ inline-grid
  - table/ inline-table/ table-row-group/ table-header-group/ table-footer-group/ table-row/ table-column-group/ table-column/ table-cell/ table-caption/

4.6. Các thuộc tính thường dùng

4.6.1. Định dạng cho text

4.6.1.1. Họ font

Generic Font Family	Examples of Font Names
Serif	Times New Roman Georgia Garamond
Sans-serif	Arial Verdana Helvetica
Monospace	Courier New Lucida Console Monaco
Cursive	Brush Script MT Lucida Handwriting
Fantasy	Copperplate Papyrus

Hình 4.6: Các họ font

4.6.1.2. Cách sử dụng Google Fonts

- Để sử dụng Google fonts cần khai báo font cần sử dụng ở thẻ link trong phần <head>

```
<head>
   <link rel="stylesheet" href="https://fonts.googleapis.com/css?
   family=Audiowide|Sofia|Trirong">
</head>
```

- Để sử dụng các hiệu ứng của font thì cần khai báo thêm hiệu ứng

```
<head>
   <link rel="stylesheet" href="https://fonts.googleapis.com/css?
   family=Audiowide|Sofia|Trirong
```

```
   &effect=fire|outline|emboss|shadow-multiple">
</head>
<body>
   <h1 class="font-effect-fire">Sofia on Fire</h1>
</body>
```

4.6.1.3. Cách sử dụng Icons CSS

Ảnh .bmp hiển thị không tốt khi bị thay đổi kích thước (ảnh bị vỡ khi phóng to, mất chi tiết khi thu nhỏ và mỗi ảnh lại cần một gởi yêu cầu mỗi khi tải trang, điều này cũng góp phần làm giảm tốc độ tải trang. Nếu muốn đẹp phải dùng photoshop tạo ra nhiều kích cỡ, điều này rất mất thời gian.

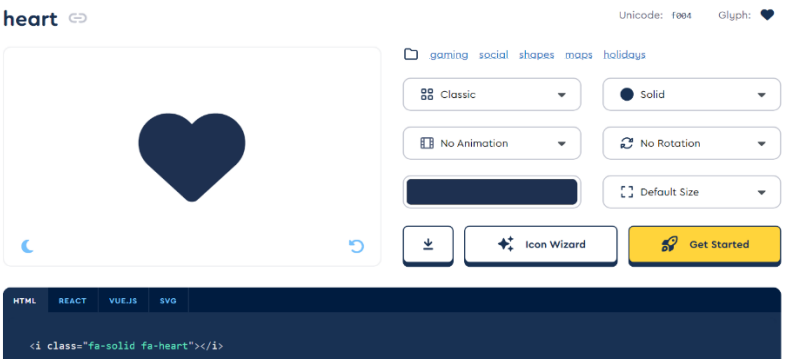
Icon font là dùng những font chữ được thiết kế đặc biệt để thay thế cho các hình ảnh được dùng trong việc thiết kế website, đặc biệt với những website hỗ trợ responsive thì điều này càng cần thiết, vì khi đó các hình ảnh cần có kích thước khác nhau trên các thiết bị khác nhau. Nếu chỉ dùng một hình ảnh thì sẽ không thể tạo được sự sắc nét khi hiển thị với nhiều kích thước.

- Font Awesome Icons
  - Bước 1: Tạo tài khoản tại trang https://fontawesome.com/, sau đó nhận mã.
  - Bước 2: Vào trang html, thêm mã đó vào phần <head>

```
<script src="https://kit.fontawesome.com/acf634688a.js" crossorigin="anon
ymous"></script>
```

  - Bước 3: Vào lại trang https://fontawesome.com/icons để tìm font cần sử dụng
  - Bước 4: Thiết lập các giá trị để có icon như ý muốn, sau đó sao chép mã bên dưới để dán vào trang html

```
<body>
 <ul>
    <li><i class="fa-solid fa-house">Home</i></li>
    <li><i class="fa-solid fa-heart">Contact</i></li>
    <li><i class="fa-solid fa-cart-shopping">Shopping</i></li>
     </ul>
</body>
```

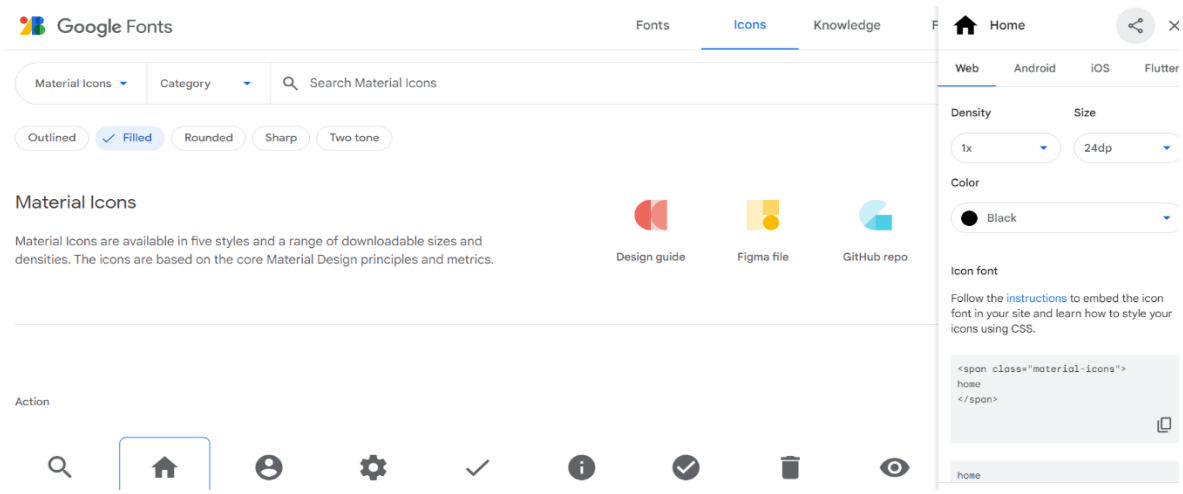


Hình 4.7: Thiết lập Awesome Icons

- Google Icons
  - Bước 1: Vào trang https://fonts.google.com/icons, tìm font cần sử dụng
  - Bước 2: Sao chép mã để dán vào trang html

```
<html>
<head>
```

```
   <link rel="stylesheet"
       href="https://fonts.googleapis.com/icon?family=Material+Icons">
</head>
<body>
   <ul>
  <li><i class="material-icons">home</i>Home</li>
   <li><i class="material-icons">favorite</i>Contact</li>
   <li><i class="material-icons">shopping_cart</i>Shopping</li>
</body>
```



Hình 4.8: Thiết lập Google Icons

- Một số trang icons khác:
  - https://ionic.io/ionicons/
  - https://icomoon.io/app/#/select

4.6.1.4. Các thuộc tính định dạng cho text

STT	Tên thuộc tính	Giá trị	Chức năng
1	font	font-style font-variant font-weight font-size/line-height font-family;	Thiết lập font chữ (dạng rút gọn) Font-size Line-height p{ font:italic bold 12px/30px Georgia, serif;}
	font-family	tên-font, họ-font	Thiết lập tên font và họ font
	font-style	italic/olique	Thiết lập kiểu chữ
	font-variant	small-caps	Thiết lập kiểu chữ in hoa nhỏ
	font-weight	bold/ bolder/ 100…900	Thiết lập độ đậm chữ
	font-size	số (px, %, rem, em)	Thiết lập kích cỡ chữ
2	line-height	số	Thiết lập độ cao của dòng
3	text-align	justify/ center/ right/ left	Căn chỉnh lề cho đoạn văn
4	text-transform	uppercase/ lowercase/ capitalize	Tạo kiểu chữ in hoa, thường
5	text-decoration	none/ overline/ underline/ line-through	Trang trí văn bản (gạch dưới, gạch ngang, gạch trên,…)
6	text-indent	số	Khoảng cách thụt vào của dòng đầu tiên trong đoạn văn

7	letter-spacing	số	Khoảng cách giữa các kí tự
8	word-spacing	số	Khoảng cách giữa các từ
9	color	rgb(), #000000, black	Thiết lập màu chữ

Bảng 4.2: Thuộc tính định dạng cho text

4.6.2. Định dạng cho danh sách

STT	Tên thuộc tính	Giá trị	Chức năng
1	list-style	[type] [position] [image]	Thiết lập các kí hiệu, chỉ số trong danh sách (dạng rút gọn).
2	list-style-type	none	Không hiển thị các kí hiệu, chỉ số.
		circle/ square/ disc/ upper-roman/ lower-alpha/ …	Các dạng kí hiệu, chỉ số
3	list-style-position	outside	Thiết lập các mục nằm bên ngoài nội dung (mặc định).
		inside	Thiết lập các mục nằm bên trong nội dung.
4	list-style-image	url("…")	Thay thế các kí hiệu, chỉ số của danh sách bằng hình ảnh.

Bảng 4.3: Thuộc tính định dạng cho danh sách

4.6.3. Định dạng cho bảng

STT	Tên thuộc tính	Giá trị	Chức năng
1	border	[width] [style] [color]	Thiết lập viền cho bảng.
2	border-collapse	separate	Thiết lập đường viền của bảng và đường viền của các ô trong bảng tách biệt nhau.
		collapse	Thiết lập đường viền của bảng và đường viền của các ô trong bảng gộp lại với nhau.
3	border-spacing	số	Thiết lập khoảng cách giữa các ô
4	caption-side	top	Chú thích ở trên bảng
		bottom	Chú thích ở dưới bảng
5	empty-cell	show	Hiển thị nền và viền các ô rỗng
		hide	Không hiển thị nền và viền các ô rỗng
6	table-layout	auto	Chiều rộng cột của bảng được thiết lập theo ô có chiều rộng lớn nhất trong cột.

		fixed	Chiều rộng chỉ phụ thuộc vào chiều rộng của bảng và chiều rộng của cột, không phụ thuộc vào nội dung của phần tử.

Bảng 4.4: Thuộc tính định dạng cho bảng

4.6.4. Định dạng nền

STT	Tên thuộc tính	Giá trị	Chức năng
1	background	color image repeat position;	Thiết lập nền (dạng rút gọn) Có thể kết hợp thêm thuộc tính Opacity (0-1) để thiết lập độ mờ cùa màu/hình nền
2	background-color	màu	Thiết lập màu nền
3	background-image	url("hình nền")	Thiết lập hình nền
4	background-size	số (px, %, …)	Thiết lập kích thước hình nền
		cover	Tự chia tỷ lệ tới kích thước lớn nhất của chiều rộng hoặc chiều cao để phù hợp với vùng nội dung. - Nếu chiều cao lớn hơn chiều rộng thì background sẽ chỉnh 100% theo chiều cao, chiều rộng sẽ tự động chỉnh tỷ lệ cho phù hợp. - Ngược lại nếu chiều rộng lớn hơn chiều cao thì background sẽ chỉnh 100% theo chiều rộng, chiều cao sẽ tự động chỉnh tỷ lệ cho phù hợp.
		contain	Tự chia tỷ lệ tới kích thước nhỏ nhất của chiều rộng hoặc chiều cao để phù hợp với vùng nội dung. - Nếu chiều cao nhỏ hơn chiều rộng thì background sẽ chỉnh 100% theo chiều cao, chiều rộng sẽ tự động chỉnh tỷ lệ cho phù hợp. - Ngược lại nếu chiều rộng nhỏ hơn chiều cao thì background sẽ chỉnh 100% theo chiều rộng, chiều cao sẽ tự động chỉnh tỷ lệ cho phù hợp.
5	background-repeat	repeat/ no-repeat repeat-x/ repeat-y	Sự lặp đi lặp lại của một hình nền theo chiều dọc hoặc chiều ngang
6	background-attachment	fixed	Thiết lập ảnh nền cố định
		scroll	Thiết lập ảnh nền cuộn theo nội dung

7	background-position	top/ bottom/ left/ right/ center	Thiết lập vị trí của hình nền dựa vào tên vị trí
		số (%, px, …) (số dương hoặc âm)	Thiết lập vị trí của hình nền tại các vị trí thiết lập
8	background-clip	border-box	Thiết lập mức độ phủ của màu nền. Nền phủ đến viền (mặc định)
		padding-box	Nền phủ đến vùng padding (không bao gồm viền)
		content-box	Nền chỉ phủ vùng nội dung phần tử
9	background-origin	border-box	Thiết lập mức độ phủ của hình nền Nền phủ đến viền (mặc định)
		padding-box	Nền phủ đến vùng padding (không bao gồm viền)
		content-box	Nền chỉ phủ vùng nội dung phần tử

Bảng 4.5: Thuộc tính background

Prefix	Organization	Most popular browsers
-ms-	Microsoft	Internet Explorer
-moz-	Mozilla Foundation	Firefox, Camino, Seamonkey
-o-	Opera Software	Opera, Opera Mini, Opera Mobile
-webkit-	Originally Apple; now open source	Safari, Chrome, Android, Silk, Blackberry, WebOS, many others

4.6.5. Định dạng viền

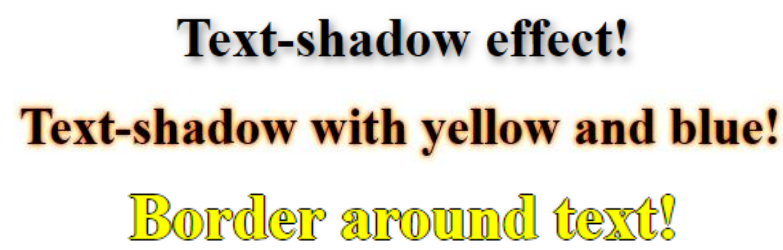
STT	Tên thuộc tính	Giá trị	Chức năng
1	border	width style color;	Thiết lập viền (dạng rút gọn)
2	border-width	số	Độ dày của đường viền
3	border-style	none/ solid/ dotted/ dashed/ double/ groove/ ridge/ inset/ outset	Kiểu đường viền (bắt buộc phải khai báo)
4	color	màu	Màu đường viền
5	border-radius	số (top left – top right – bottom right - bottom left)	Bo góc W=h và border-radius:50% sẽ tạo hình tròn
6	border-image	url( ) slice width outset repeat; - slice: số (px, %) - outset: round/ stretch	Thiết lập hình làm viền border-image: url(border.png) 20 stretch;

Bảng 4.6: Thuộc tính border

4.6.6. Định dạng đổ bóng

- Tạo đổ bóng cho text

```
h1{ text-shadow: 2px 2px 5px #888; }
h1{ text-shadow: 0 0 3px #FF0000, 0 0 5px yellow; }
h1{
   color: yellow;
   text-shadow: -1px 0 black, 0 1px black, 1px 0 black, 0 -1px black;
}
```



Hình 4.9: Thuộc tính text-shadow

- Tạo đổ bóng cho hình ảnh, hộp chứa
  - box-shadow: h-offset v-offset blur spread color inset
    - h-offset: vị trí bóng đổ theo chiều ngang, số dương đẩy bóng xuống dưới, số âm đẩy bóng lên phía trên (bắt buộc).
    - v-offset: vị trí bóng đổ theo chiều dọc, số dương đẩy bóng về trước, số âm đẩy bóng lùi ra sau (bắt buộc).
    - spread: kích thước của bóng (tùy chọn).
    - blur: độ nhòe của bóng (tùy chọn).
    - color: màu sắc của bóng (tùy chọn).
    - inset: tạo bóng bên trong hộp (tùy chọn)
  - Ví dụ: tạo đổ bóng cho hình ảnh

```
img {
 width: 250px;
 box-shadow: 0 4px 8px 0 rgba(0,0,0,0.3), 0 6px 20px 0
               rgba(0,0,0,0.2);
}
```

  - Ví dụ: tạo đổ bóng cho hộp chứa

```
div.card {
  width: 250px;
  text-align: center;
  box-shadow: 0 4px 8px 0 rgba(0,0,0,0.3), 0 6px 20px 0
               rgba(0,0,0,0.2);
}
```



Hình 4.10: Thuộc tính box-shadow

4.7. Thuộc tính định vị trí

4.7.1. Float và Clear

4.7.1.1. Thuộc tính Float

- Chức năng: cho phép thành phần dịch chuyển về 1 bên (trái/phải) để giải phóng vùng không gian còn lại cho thành phần khác lấp vào.
- Giá trị: none/ left/ right/ inherit

1 - float: left;	2 - float: left;	3 - float: left/right;

1 - float: left;	3 - float: left/right;	2 - float: right;

2 - float: left;	3 - float: left/right;	1 - float: right;

3 - float: left/right;	2 - float: right;	1 - float: right;

Hình 4.11: Thuộc tính float



Hình 4.12: Thuộc tính float có giá trị left



Hình 4.13: Thuộc tính float có giá trị right

4.7.1.2. Thuộc tính Clear

- Chức năng: ngăn không cho thành phần A ở bên (trái/phải/2 bên) của thành phần B
- Giá trị: left/ right/ none/ both/ inherit

1 - float: left;	2 - float: left;	
3 - clear: left;		

1 - float: left;		2 - float: right;
3 - clear: left/both;		

Hình 4.14: Thuộc tính clear

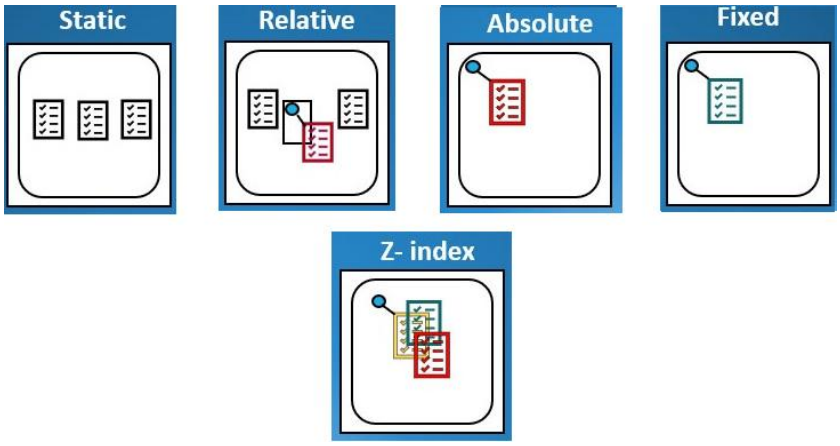


Hình 4.15: Thuộc tính clear có giá trị left



Hình 4.16: Thuộc tính clear có giá trị both

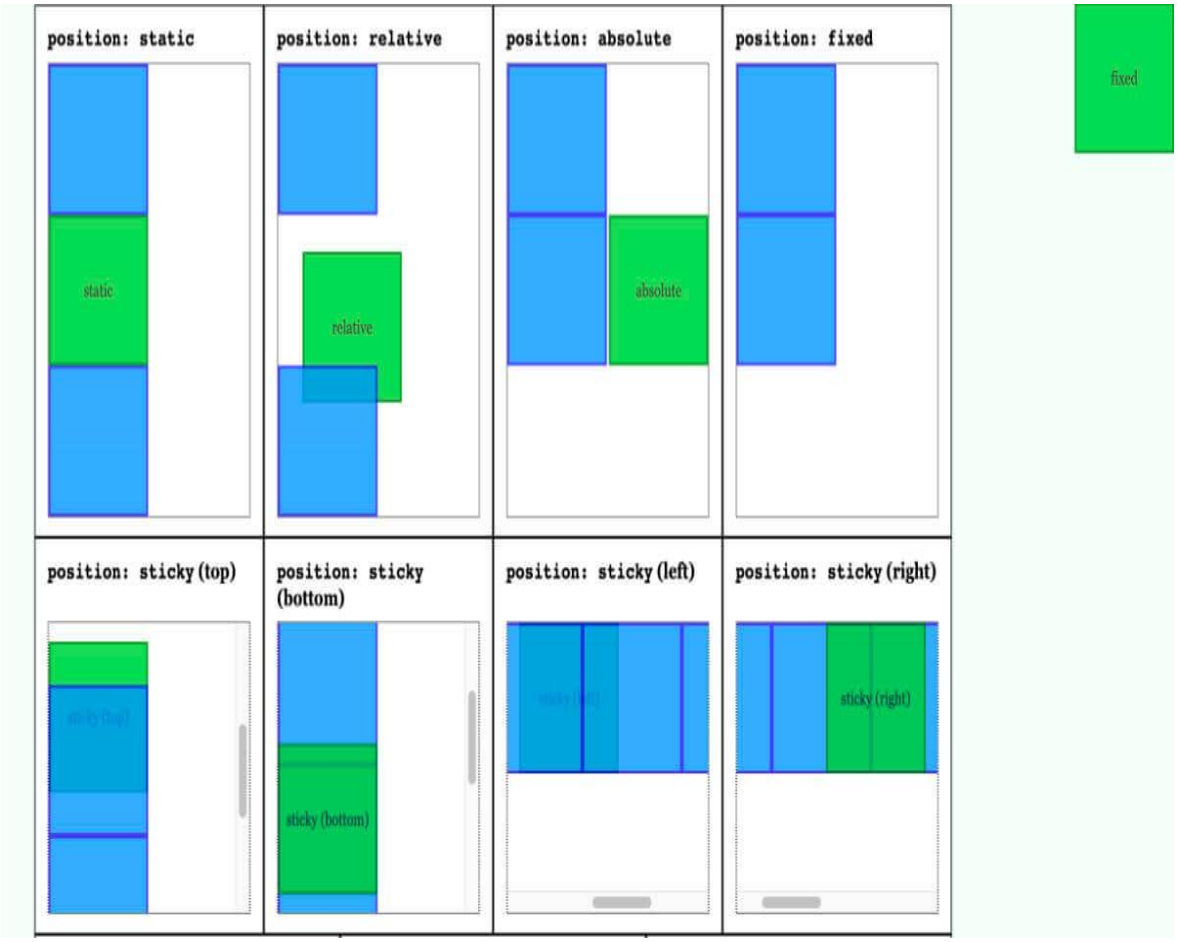
4.7.2. Position và z-index



Hình 4.17: Thuộc tính position và z-index

4.7.2.1. Thuộc tính position

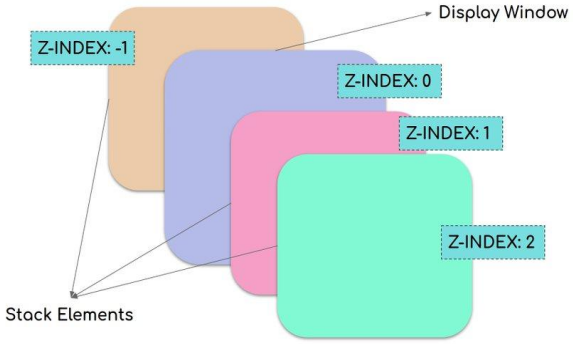
- Chức năng: Định vị trí của 1 thành phần
- Khi sử dụng thuộc tính position thì phải kết hợp thêm thuộc tính top, right, bottom, left để định vị trí.
- Giá trị: relative | absolute | fixed | sticky| inherit| static (mặc định)
  - Static: mặc định, không bị ảnh hưởng bởi các thuộc tính top | left | right | bottom
  - Relative: phần tử dịch chuyển so với vị trí gốc ban đầu của nó, vẫn chiếm vùng không gian của trình duyệt.
  - Absolute: phần tử con (con phải thiết lập absolute) dịch chuyển theo phần tử cha đang chứa nó (cha phải thiết lập relative), không chiếm vùng không gian của trình duyệt (nếu không có phần tử cha thì mặc định cha là body)
  - Fixed: cho phép giữ một phần tử ở vị trí cố định trên trang web, bất kể người dùng cuộn trang đến đâu; được sử dụng để tạo các thành phần cố định như quảng cáo, các nút bấm,... Một phần tử fixed không để lại khoảng trống trên trang.
  - Sticky: cho phép giữ một phần tử ở vị trí cố định trên trang web khi người dùng cuộn trang. Phần tử sticky sẽ thay đổi giữa relative và fixed tùy thuộc vào vị trí cuộn. Nó sẽ relative cho tới khi vị trí tới điểm của khung trình duyệt, khi đó nó sẽ dính ở vị trí đó (khi đó, nó giống với cách hiển thị position: fixed).
    Để thuộc tính này có hiệu lực thì yêu cầu phải có có điều kiện sau:
    - Phần tử phải có một kích thước cụ thể, hoặc đang ở trong một phần tử có kích thước cụ thể.
    - Phần tử phải có một giá trị top, bottom, left hoặc right được định nghĩa.
    - Phần tử cha của nó phải có thuộc tính overflow: hidden hoặc overflow: auto.



Hình 4.18: Thuộc tính position

4.7.2.2. Thuộc tính z-index

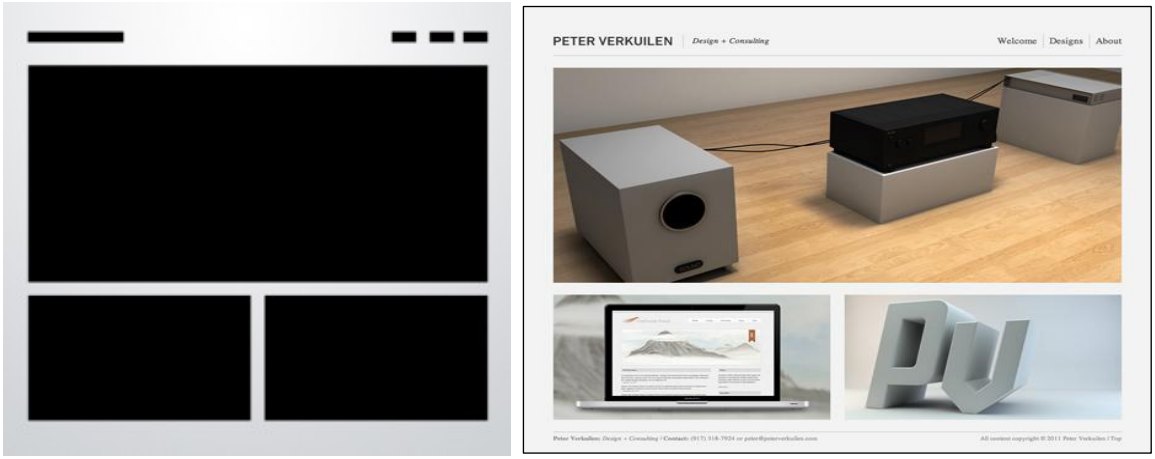
- Chức năng: thiết lập thứ tự xếp chồng nhau của một phần tử vị trí.
- Giá trị: số nguyên dương hoặc âm, mặc định là 0.
  Thứ tự chồng nhau được sắp xếp dựa theo giá trị số, thành phần HTML nào có chỉ số z-index cao hơn sẽ nằm trên, ngược lại sẽ nằm dưới.
- z-index chỉ có tác dụng với các phần tử khi chỉ định thêm 1 trong 3 giá trị absolute, fixed, relative.



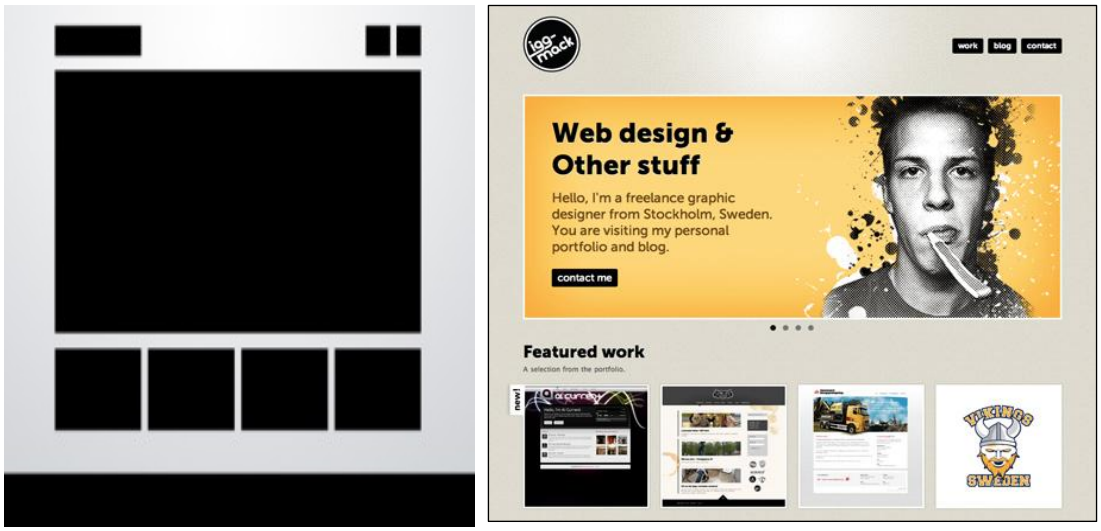
Hình 4.19: Thuộc tính z-index

4.8. Xây dựng giao diện

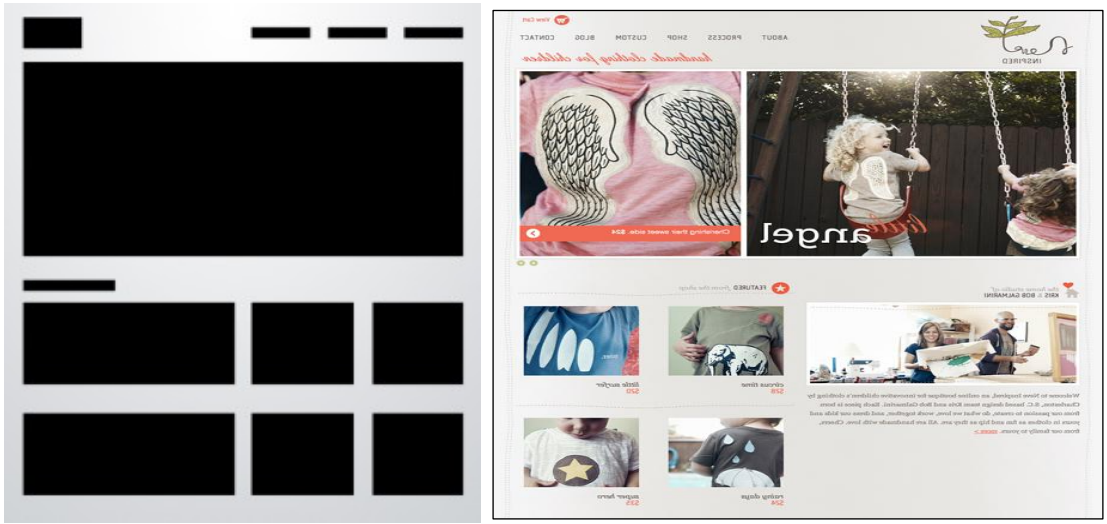
4.8.1. Bố cục giao diện



Hình 4.20: Giao diện 3 khối hộp



Hình 4.21: Giao diện 5 khối hộp



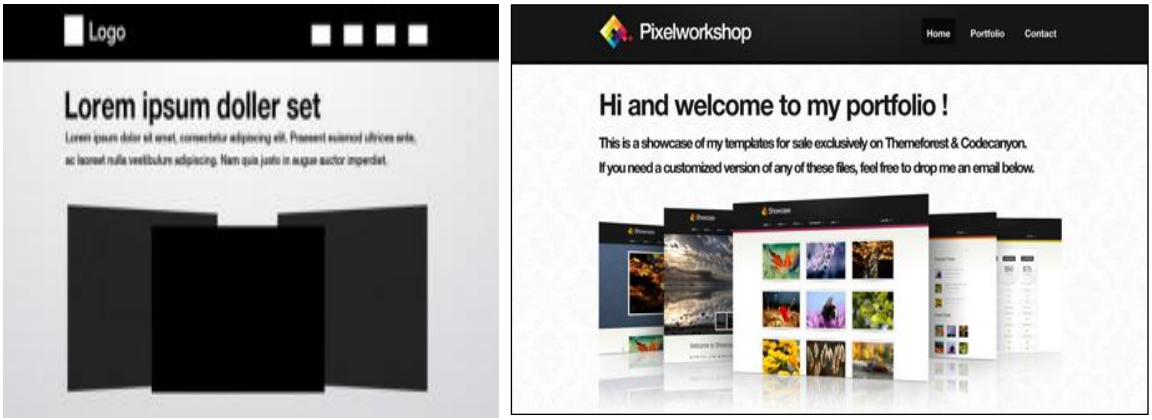
Hình 4.22: Giao diện lưới



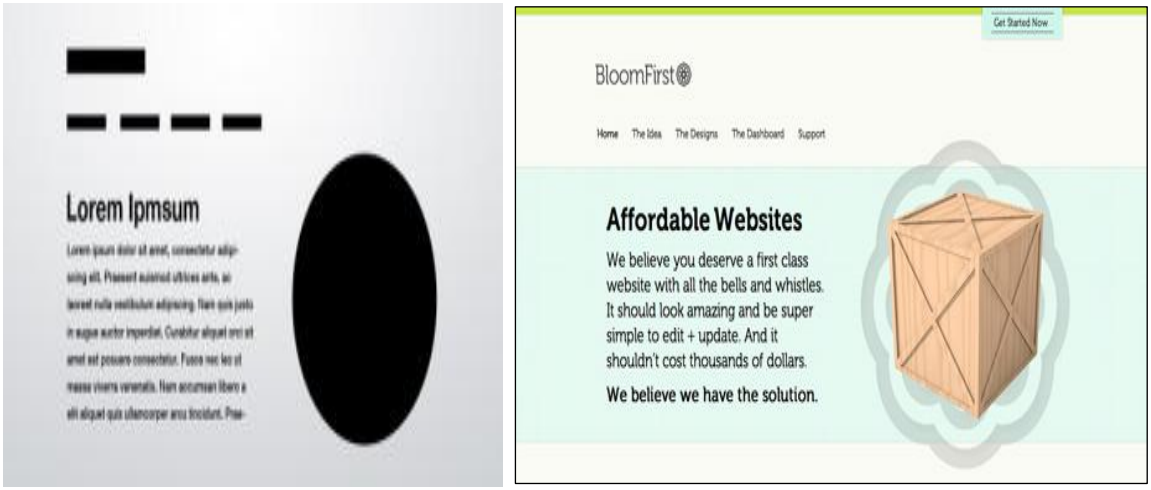
Hình 4.23: Giao diện lưới đa năng



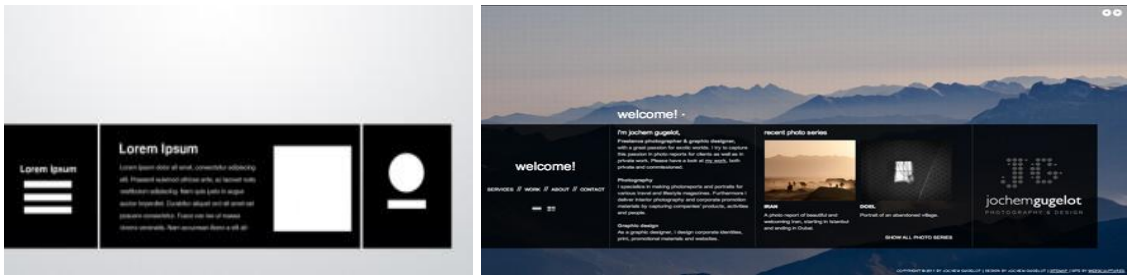
Hình 4.24: Giao diện có thanh lề cố định



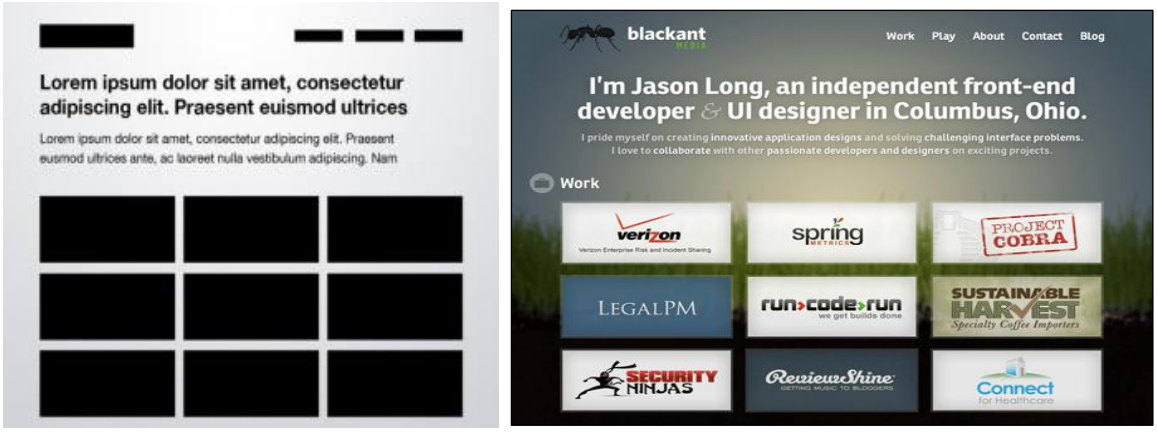
Hình 4.25: Giao diện màn hình 3 chiều



Hình 4.26: Giao diện sử dụng đồ họa làm trung tâm



Hình 4.27: Giao diện có nền kín trang



Hình 4.28: Giao diện thư viện ảnh

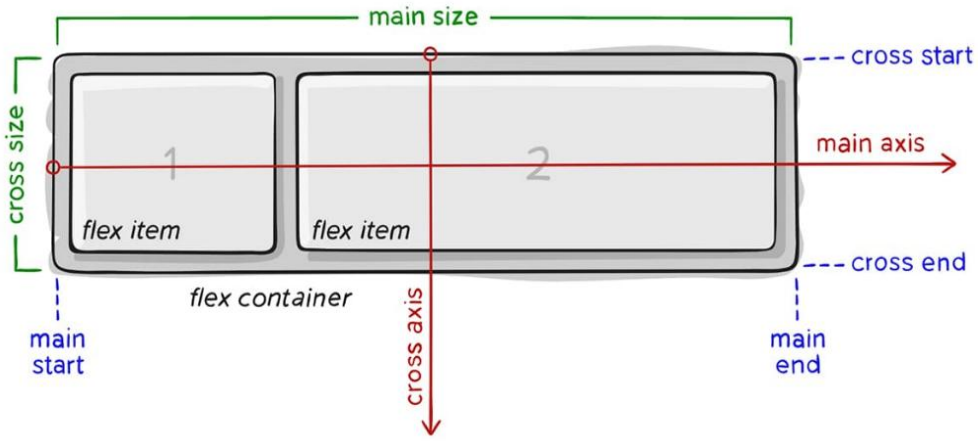
4.8.2. Flexbox

4.8.2.1. Định nghĩa Flexbox

Flexbox là một kiểu bố cục trang có khả năng tự cân đối kích thước, thay đổi chiều rộng/chiều cao và thứ tự phần tử bên trong để phù hợp với tất cả các loại thiết bị hiển thị và kích thước màn hình. Khi đó, không cần thiết lập float, chỉ cần thiết lập hiển thị ngang hay dọc, lúc đó các phần tử bên trong có thể hiển thị theo ý muốn.

Flexbox thường dùng để thiết lập bố cục ở quy mô nhỏ. Khi thiết lập bố cục ở phạm vi lớn hơn (như chia cột website) thì nên sử dụng dàn trang theo dạng lưới (grid layout).

4.8.2.2. Bố cục Flexbox



Hình 4.29: Bố cục Flexbox

Bố cục Flex được thiết lập gồm một khung lớn (parent container) đóng vai trò là khung linh hoạt (flex containter) và các thẻ con bên trong nó (immediate children) đóng vai trò các mục nhỏ linh hoạt (flex item).

- **container:** là thành phần lớn bao quanh các phần tử bên trong, các item bên trong sẽ hiển thị dựa trên thiết lập của container này.
- **item:** là phần tử con của container, có thể thiết lập bao nhiêu cột trong một container, hoặc thiết lập thứ tự hiển thị của nó.

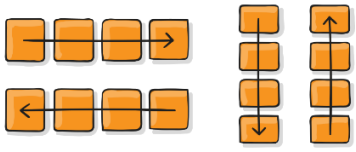
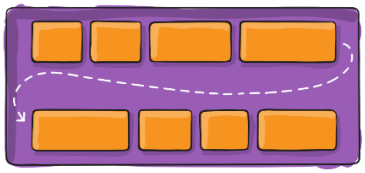
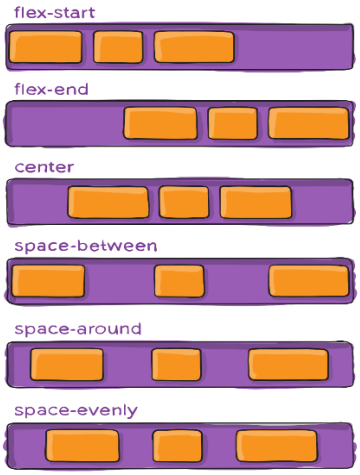
Các item sẽ được bố trí theo trục main axis (bắt đầu từ main-start, kết thúc ở main-end) hoặc theo trục cross axis (bắt đầu từ cross-start, kết thúc ở cross-end).

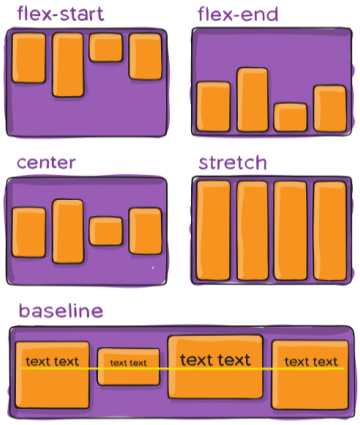
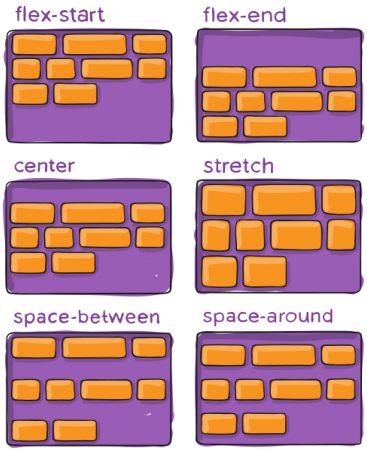
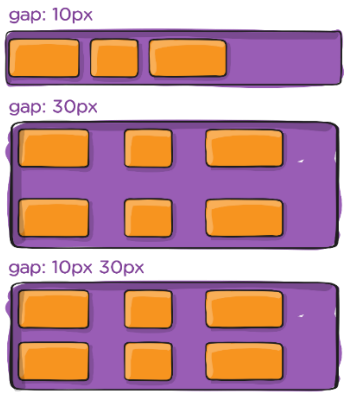
- **main axis**: trục chính để điều khiển hướng mà các item sẽ hiển thị. Lưu ý, main axis không phải lúc nào cũng nằm ngang như sơ đồ trên, có thể sử dụng thuộc tính flex-direction để thay đổi hướng của trục và lúc đó các item sẽ hiển thị theo nó.
- **main-start | main-end**: các item nằm trong container hiển thị từ điểm bắt đầu gọi là main-start tới điểm kết thúc gọi là main-end.
- **main size**: kích thước (chiều rộng hoặc chiều cao) của các item, tùy thuộc vào hướng của main axis.
- **cross axis**: cross axis luôn là trục vuông góc của main axis. Hướng của nó phụ thuộc vào hướng của main axis.
- **cross-start | cross-end**: có ý nghĩa tương tự nhưng luôn vuông góc với main start, main end.
- **cross size**: kích thước (chiều rộng hoặc chiều cao) của các item dựa trên trục cross axis, tùy thuộc vào hướng của main axis.

4.8.2.3. Các thuộc tính trong Flexbox

- Thuộc tính cho thành phần Flex Container

STT	Tên thuộc tính	Chức năng
1	display	- Định nghĩa một flex container. - Giá trị: flex | inline-flex

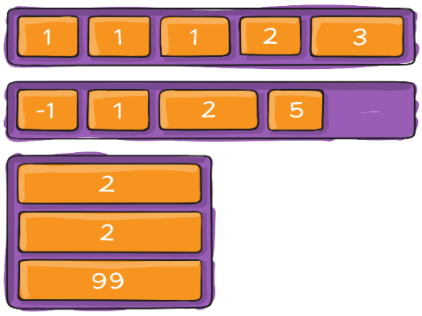
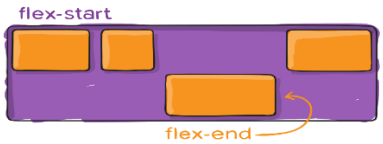
2	flex-direction	- Xác định hướng các flex item sẽ được đặt trong flex container. - Giá trị: row | row-reverse | column | column-reverse 
3	flex-wrap	- Cho phép các item hiển thị trên 1 hàng hay nhiều hàng. - Giá trị: nowrap | wrap | wrap-reverse 
4	flex-flow	- Viết tắt cho 2 thuộc tính flex-direction và flex-wrap. - Giá trị: flex-direction flex-wrap;
5	justify-content	- Điều khiển các phần tử trôi về phía nào đó của hướng chính (main axis) - Giá trị: flex-start | flex-end | center | space-between | space-around | space-evenly 
6	align-items	- Điều khiển phần tử con theo hướng vuông góc (cross axis) với hướng chính. - Giá trị: flex-start | flex-end | center | stretch | baseline

		
7	align-content	- Phân phối khoảng trống giữa các hàng item so với cạnh container (theo trục cross axis). Chỉ có hiệu lực khi có nhiều hàng hoặc nhiều cột. - Giá trị: flex-start | flex-end | center | stretch | space-between | space-around | space-evenly 
8	gap	- Khoảng cách giữa các hàng và giữa các cột - Giá trị: số (gap: column row;) 

Bảng 4.7: Thuộc tính cho thành phần Flex Container

- Thuộc tính cho thành phần Flex Item

STT	Tên thuộc tính	Công dụng
1	order	- Qui định thứ tự hiển thị của các flex item. - Giá trị: số nguyên (..-2, -1, 0, 1, 2, …), mặc định là 0

		
2	flex-grow	- Phần tử này chiếm bao nhiêu phần trong không gian còn lại (trừ đi phần đã chiếm bởi các phần tử có flex-grow bằng 0) khung chứa theo hướng chính. - Giá trị: số nguyên, mặc định là 0 
3	flex-shrink	- Phần tử có thể co lại bao nhiêu phần khi cần thiết - Giá trị: số (mặc định là 1, không được sử dụng số âm)
4	flex-basis	- Gán chiều dài khởi tạo cho phần tử trong hệ thống flex - Giá trị: số (VD: 100px, …)
5	flex	- Viết tắt cho cả 3 thuộc tính flex-grow, flex-shrink, flex-basis - Giá trị: flex-grow flex-shrink flex-basis;
6	align-self	- Căn chỉnh một phần tử trong hệ thống flex - Giá trị: flex-start | flex-end | center | stretch 

Bảng 4.8: Thuộc tính cho thành phần Flex Item

4.8.3. Grid

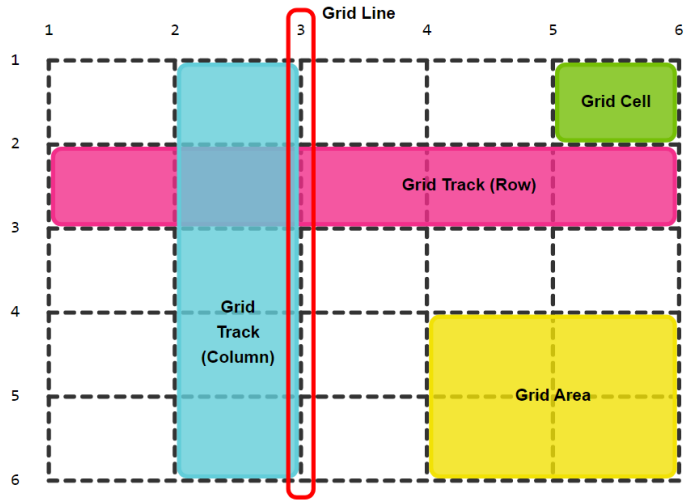
4.8.3.1. Định nghĩa Grid

Grid layout là một hệ thống lưới hai chiều được dùng để chia các phần tử thành các cột và hàng, nhằm tạo ra một bố cục nhất quán và liền mạch cho các ứng dụng web.

4.8.3.2. Bố cục Grid

Grid gồm các thành phần sau:

- Grid container là hộp chứa lưới, đây là khối xây dựng chính của grid.
- Grid item là các ô trên lưới.
- Grid area là một ô đơn lẻ hoặc nhiều ô có dạng hình vuông hoặc hình chữ nhật (nhưng không phải hình chữ L)
- Grid track là tập hợp các hàng và cột, được xác định bằng cách sử dụng thuộc tính grid-template-columns và grid-template-rows
- Grid gaps là khoảng cách giữa các ô trong lưới.



Hình 4.30: Bố cục Grid

4.8.3.3. Các thuộc tính trong Grid

- Thuộc tính cho thành phần Grid Container

STT	Tên thuộc tính	Chức năng
1	display	- Định nghĩa một grid container. - Giá trị: grid | inline-grid
2	grid-template-columns	- Thiết lập số cột của grid. Cách 1: 100px 800px 100px Cách 2: 10% 80% 10% Cách 3: 1fr 8fr 1fr Nếu 3 cột có độ rộng bằng nhau thì repeat(3, 1fr))
3	grid-template-rows	- Thiết lập số dòng của grid. Cách 1: 100px 500px 100px Cách 2: 100px auto 100px Cách 3: auto auto auto
4	justify-content	- Căn chỉnh các phần tử theo trục ngang - Giá trị: start | end | center | stretch | space-between | space-around | space-evenly
5	align-content	- Căn chỉnh các phần tử theo trục dọc - Giá trị: end | center | space-between | space-around | space-evenly
6	gap	- Khoảng cách giữa các hàng và giữa các cột - Giá trị: số (grid-gap: column row;)
7	column-gap	- Khoảng cách các ô theo chiều ngang - Giá trị: số (px, %, …)
8	row-gap	- Khoảng cách các ô theo chiều dọc - Giá trị: số (px, %, …)

Bảng 4.9: Thuộc tính cho thành phần Grid Container

- Thuộc tính cho thành phần Grid Item

STT	Tên thuộc tính	Chức năng
1	grid-column	- Thiết lập vị trí bắt đầu và kết thúc của grid item theo cột. Cách 1: column-line-start/column-line-end (1/4) Cách 2: 1/ span 3 (column-start/total-column)
2	grid-row	- Thiết lập vị trí bắt đầu và kết thúc của grid item theo hàng. Cách 1: row-line-start/row-line-end (1/4) Cách 2: 1/ span 3 (row-start/total-row)
3	justify-self	- Căn chỉnh nội dung bên trong một grid item dọc theo hàng. - Giá trị: start | end | center | stretch

Bảng 4.10: Thuộc tính cho thành phần Grid Item

Ngoài ra, có thể bố cục giao diện bằng cách sử dung thuộc tính **grid-template-areas** cho grid container để định nghĩa các khu vực và **grid-area** cho grid item để áp dụng các vùng đó.

```
img{
 width:100%; height: 100%; border-radius: 10px;
 object-fit: cover;
}
#wrapper{
 width:960px; margin:10px auto;
 display: grid;
 grid-template-columns: repeat(4,1fr);
 grid-template-rows: 250px 100px 250px;
 grid-gap:10px;
 grid-template-areas:
       'h1 h1 h2 h3'
       'h4 h5 h2 h3'
       'h4 h5 h6 h6'
       ;
}
.box1{ grid-area: h1; }
.box2{ grid-area: h2; }
.box3{ grid-area: h3; }
.box4{ grid-area: h4; }
.box5{ grid-area: h5; }
.box6{ grid-area: h6; }
```



Hình 4.31: Thuộc tính grid-template-areas

4.8.4. Responsive website

4.8.4.1. Định nghĩa

- Responsive là một thuật ngữ chỉ một website có thể hiển thị tương thích trên nhiều mọi trình duyệt và trên nhiều thiết bị có kích thước khác nhau.
- Vì sao phải responsive cho website?
  - Nếu người dùng truy cập trang web từ các thiết bị như máy tính bảng hay điện thoại, sẽ chỉ nhìn thấy một phần nội dung và sẽ cần liên tục di chuyển, phóng to/thu nhỏ để xem được toàn bộ những gì mà trang hiển thị. Một số trang đã đưa ra giải pháp sử dụng đồng thời hai phiên bản riêng biệt cho máy tính và điện thoại. Thế nhưng chi phí để tạo và duy trì cả hai phiên bản lại quá cao, đồng thời việc này cũng làm phân tán khả năng tìm kiếm, tiếp cận của website thông qua các công cụ tìm kiếm. Responsive trở thành giải pháp tối ưu nhất trong thời điểm hiện tại để khắc phục tất cả các vấn đề trên.
  - Thêm vào đó, trong trường hợp cần điều chỉnh, bảo trì, thay đổi giao diện thì thiết kế responsive cũng giúp giảm bớt thời gian, công sức và chi phí thực hiện khi chỉ cần cập nhật trên 1 phiên bản duy nhất.
  - Ngoài ra, đứng trên góc độ người sử dụng, việc website được thiết kế responsive sẽ mang đến cho họ những trải nghiệm tốt hơn, thao tác thuận tiện hơn, xem nội dung đơn giản hơn nhờ bố cục giao diện được thiết kế một cách hợp lý bởi với kích thước màn hình của các thiết bị di động thì cần có một layout mới để thuận tiện cho thao tác hơn, đặc biệt là thao tác một tay trên smartphone.



Hình 4.32: Responsive website

4.8.4.2. Một số lưu ý khi responsive website

- **Breakpoint**: Breakpoint giúp cho thiết bị tự xác định được các điểm giới hạn khi hiển thị nội dung.

  Ví dụ: khi nhìn trên giao diện máy tính, nội dung được chia thành ba cột khác nhau, nhưng trên điện thoại, vì có breakpoint ở cột nội dung đầu tiên nên người dùng chỉ nhìn thấy phần nội dung của cột đó, hai cột còn lại sẽ được tách xuống hiển thị bên dưới; nghĩa là nếu ở desktop sẽ hiển thị 3 cột nhưng trên mobile thì sẽ được cắt thành 3 dòng.

- **Max/Min**: Phần nội dung bao gồm cả chữ viết, hình ảnh hay bất kỳ định dạng lưu trữ thông tin khác cũng thường sở hữu kích thước không đồng nhất. Điều này khiến cho một số phần nội dung hiển thị vừa đủ trên giao diện, một số khác lại bị tràn qua, vượt quá bề ngang của thiết bị, hay còn gọi là tràn nội dung.
  Khi đó, các giá trị Max/Min sẽ giúp giới hạn được vị trí, không gian hiển thị các nội dung này trên website theo từng thiết bị (cụ thể theo kích thước màn hình) giúp việc hiển thị nội dung được tối ưu hơn.
- Có 2 cách thiết kế responsive: Desktop-first | Mobile-First
  - **Desktop-first:** Thiết kế giao diện trang web tương thích với tỉ lệ giao diện máy tính, sau đó sẽ điều chỉnh, thu nhỏ các nội dung sao cho phù hợp với giao diện máy tính bảng, cuối cùng là điện thoại.
  - **Mobile-first:** Thiết kế giao diện trang web theo tỉ lệ màn hình điện thoại sau đó tiến hành bố trí, sắp xếp và điều chỉnh kích thước các nội dung cho phù hợp với máy tính bảng, cuối cùng là máy tính và các thiết bị có kích thước màn hình lớn.
    Mobile-first thường sẽ là phương thức thuận tiện, hiệu quả và phù hợp với sự phổ biến của smartphone. Hầu hết các website hiện nay đều được thiết kế theo xu hướng mobile-first bởi số lượng thiết bị di động hiện nay là quá lớn, bao gồm nhiều kích thước màn hình khác nhau cũng như sự phổ biến của smartphone khiến cho các developer phải ưu tiên cho giao diện này hơn, sau đó mới đến giao diện desktop. Tuy nhiên, không có nghĩa là bỏ qua phiên bản máy tính, bởi giao diện desktop cũng rất quan trọng và cần phải được tối ưu. Tùy vào đặc thù của website và ý định của người làm webite mà có thể thay đổi và nên ưu tiên giao diện nào trước.

4.8.4.3. Các bước thiết kế responsive website

- Bước 1: Khai báo thẻ meta viewport cho website (bắt buộc). Thẻ này được sử dụng để thiết lập việc hiển thị tương ứng kích thước màn hình cho các trình duyệt ở các thiết bị khác nhau; có thể lựa chọn các giá trị khác nhau cho thẻ meta viewport tùy theo đặc điểm và nhu cầu thiết kế.

```
<meta name="viewport" content="width=device-width, maximum-scale=1.0,
initial-scale=1.0, user-scalable=no">
```

- Một số thuộc tính có thể thiết lập trong meta viewport
  - width=device-width: chiều rộng của khung nhìn bằng chiều rộng của thiết bị
  - height= device-height: chiều cao của khung nhìn bằng chiều cao của thiết bị.
  - maximum-scale: mức thu phóng tối đa. (1.0 tương ứng với 100%)
  - minimum-scale: mức phóng to tối thiểu. (1.0 tương ứng với 100%)
  - initial-scale: mức thu phóng ban đầu và chiều rộng của chế độ xem, thường được đặt giá trị bằng 1 nghĩa là không phóng to.
    Khi giá trị đã được thiết lập thì sẽ cố định và không thể phóng to.
  - user-scalable: Cho phép người dùng phóng to màn hình hay không, giá trị “no” là không cho phép, giá trị “yes” là cho phép; không nên sử dụng thuộc tính này trong thẻ meta viewport.

- Bước 2: Viết code css tương ứng theo từng tỉ lệ của các loại thiết bị khác nhau. Việc tối ưu hiển thị trên bao nhiêu thiết bị phụ thuộc vào người lập trình web. Tuy nhiên không phải là thiết bị nào cũng được tối ưu, bởi như vậy thì mất nhiều thời gian nên thường các công ty thiết kế website chỉ code CSS cho một số thiết bị phổ biến, những kích thước màn hình chung chứ không code cho từng thiết bị, vì vậy một số thiết bị có kích thước màn hình đặc biệt có thể sẽ không được tối ưu.
  - Sử dụng thẻ truy vấn @media trong CSS3 (@media query)

```
/* áp dụng cho màn hình desktop, máy tính bảng, smartphone và có kích
thước màn hình tối thiểu là 320px nhưng nhỏ hơn 960px */
@media only screen and (min-width: 320px) and (max-width: 960px)

/* áp dụng cho màn hình desktop, máy tính bảng, smartphone và có kích
thước màn hình tối đa 960px */
@media all screen and (max-width: 960px)
```

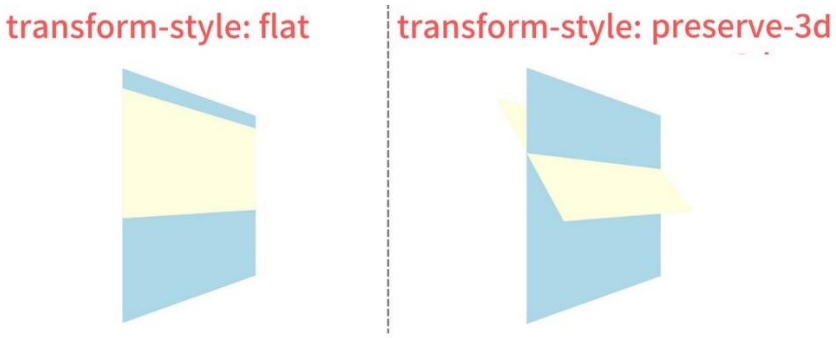
  - Một số kích thước màn hình thường dùng:
    - max-width: 320px (điện thoại di động, hiển thị chiều dọc)
    - max-width: 480px (điện thoại di động, hiển thị chiều ngang)
    - max-width: 600px (máy tính bảng, hiển thị chiều dọc)
    - max-width: 800px (máy tính bảng, hiển thị chiều ngang)
    - max-width: 768px (máy tính bảng loại to, hiển thị chiều dọc)
    - max-width: 1024px (máy tính bảng loại to, hiển thị chiều ngang)
    - max-width: 1025px (từ size này trở lên là dành cho desktop thông thường)
  - Khi viết code Responsive nên lấy đơn vị % để định dạng chiều rộng của website, với mục đích tối ưu và thuận tiện xử lý.
- Bước 3: Kiểm tra responsive có chuẩn chưa?
  - Một số tiêu chí để kiểm tra responsive:
    - Văn bản, hình ảnh, các nút được căn chỉnh chính xác
    - Trạng thái hover hay bấm chọn được làm nổi bật và thay đổi màu sắc khi thực hiện.
    - Khu vực click phù hợp (các phần tử click quá gần nhau), gây khó khăn trong việc tương tác.
    - Màu sắc, đổ bóng và gradient phù hợp.
    - Kiểm tra phần padding chính xác.
    - Văn bản, hình ảnh, nút điều hướng và khung không rộng hơn màn hình.
    - Cỡ chữ, phông cữ và màu sắc phù hợp với từng loại văn bản.
    - Biểu mẫu điền nội dung, nội dung đã nhập được hiện thị đúng.
    - Kiểu tra tất cả các trang trên website.
    - Không bao giờ dùng thanh ngang (nội dung bị tràn màn hình)
    - Nội dung được xác định là quan trọng cần phải được hiển thị ở tất cả điểm dừng (breakpoint)
  - Cách kiểm tra thông qua các trang web và công cụ:
    - https://m3.material.io/
    - https://ui.dev/amiresponsive

- https://trulyblogger.com/responsive/
- https://search.google.com/test/mobile-friendly
- Công cụ phát triển trình duyệt web (Brower Dev Tool) trên hầu hết các trình duyệt như Google Chrome, Opera hay Mozilla Firefox.
  - Bước 1: Mở website cần kiểm tra
  - Bước 2: Nhấp chuột phải, chọn Kiểm Tra (hoặc Inspect) hoặc phím tắt F12, cửa sổ Chrome Dev Tools sẽ mở.
  - Bước 3: Trên cửa sổ Dev Tools, chọn Toggle device toolbar. Trình giả lập màn hình sẽ xuất hiện
  - Bước 4: Sử dụng trình giả lập màn hình này, bạn có thể kiểm tra responsive trên thiết bị và kích thước cụ thể.

4.9. Hiệu ứng chuyển động

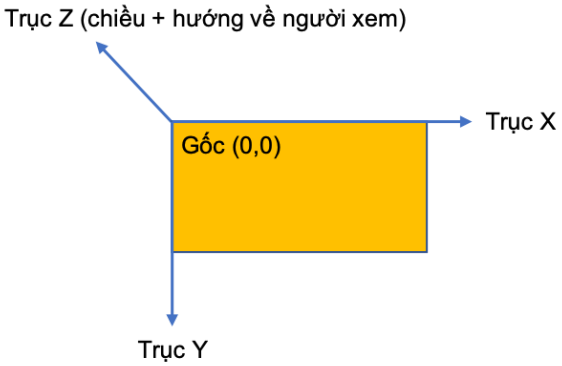
4.9.1. Thuộc tính transform

- Chức năng: Transform là những thuộc tính được dùng để xử lý các hiệu ứng của các phần tử như di chuyển, xoay, phóng to hay thu nhỏ, …
- Cú pháp: transform: value(argument);
- Thuộc tính transform-style: Các thành phần bên trong sẽ giữ vị trí 3D của nó
  Giá trị: flat | preserve-3d



Hình 4.33: Thuộc tính transform-style

- Thuộc tính transform-origin:
  - Hệ tọa độ ban đầu:
    - Gốc tọa độ là tại góc trên trái của phần tử
    - Trục X dương: hướng từ trái sang phải
    - Trục Y dương: hướng từ trên xuống
    - Trục Z dương: hướng từ tới mắt người xem



- Mặc định: transform-origin: 50% 50% (tâm của phần tử)
- Thuộc tính transform-origin cho phép thay đổi gốc hệ tọa độ



Hình 4.34: Thuộc tính transform-origin

- Các thuộc tính trong transform
  - Transform 2D

STT	Tên thuộc tính	Giá trị	Chức năng
1	translate(x,y)	số	Di chuyển theo trục X, Y
2	translateX(n)	số	Di chuyển theo trục X
3	translateY(n)	số	Di chuyển theo trục Y
4	scale(x,y)	số	Biến dạng 2D theo trục X, Y
5	scaleX(n)	số	Biến dạng 2D theo trục X
6	scaleY(n)	số	Biến dạng 2D theo trục Y
7	skew(x,y)	deg	Nghiêng 2D theo trục X, Y
8	skewX(angle)	deg	Nghiêng 2D theo trục X
9	skewY(angle)	deg	Nghiêng 2D theo trục Y
10	rotate(angle)	deg	Xoay phần tử, tối đa 360 độ lấy tâm của phần từ làm gốc
11	rotateX(angle)	deg	Xoay 2D theo trục X
12	rotateY(angle)	deg	Xoay 2D theo trục Y
13	matrix(n,n,n,n,n,n)	scaleX, skewY, skewX, caleY, translateX,translateY	Thuộc tính rút gọn của scale, skew và translate

Bảng 4.11: Thuộc tính transform 2D



Hình 4.35: Thuộc tính translate 2D



Hình 4.36: Thuộc tính scale 2D



Hình 4.37: Thuộc tính skew 2D

- Ví dụ: kết hợp nhiều thuộc tính

```
img:hover, img:focus {
transform: scale(1.5) rotate(-5deg) translate(50px,30px);}
```



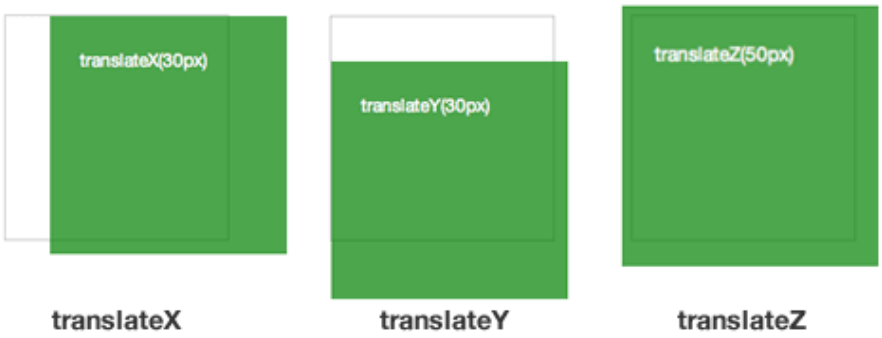
Hình 4.38: Thuộc tính transform 2D

- Transform 3D

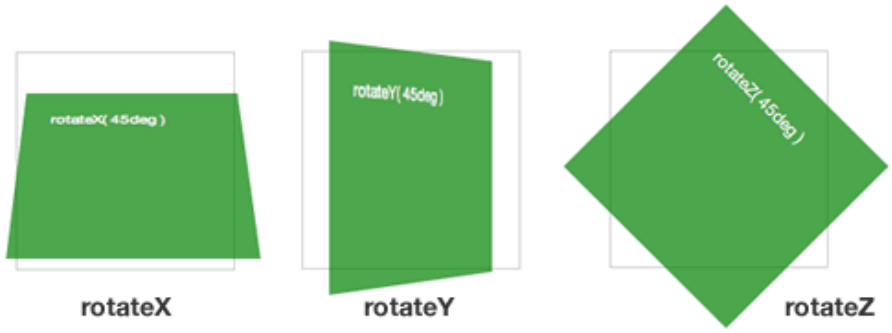
STT	Tên thuộc tính	Giá trị	Chức năng
1	translate3d(x,y,z)	số	Dịch chuyển 3D theo trục X, Y, Z
2	translateX(n)	số	Di chuyển theo trục X
3	translateY(n)	số	Di chuyển theo trục Y
4	translateZ(n)	số	Di chuyển theo trục Z
5	scale3d(x,y,z)	số	Biến dạng 3D theo trục X, Y, Z
6	scaleX(n)	số	Biến dạng 3D theo trục X
7	scaleY(n)	số	Biến dạng 3D theo trục Y
8	scaleZ(n)	số	Biến dạng 3D theo trục Z
9	skew3d(x,y,z)	deg	Nghiêng 3D theo trục X, Y, Z

10	skewX(angle)	deg	Nghiêng 3D theo trục X
11	skewY(angle)	deg	Nghiêng 3D theo trục Y
12	skewZ(angle)	deg	Nghiêng 3D theo trục Z
13	rotate3d(x,y,z,angle)	x,y,z: số angle là góc xoay quanh vecto hướng	Xoay 3D theo trục X, Y, Z
14	rotateX(angle)	deg	Xoay 3D theo trục X
15	rotateY(angle)	deg	Xoay 3D theo trục Y
16	rotateZ(angle)	deg	Xoay 3D theo trục Z
17	perspective(n)	số	Thiết lập chiều sâu để chuyển phần tử từ 2D sang 3D (khoảng cách từ mắt người xem tới màn hình) (thường đặt khoảng cách perspective cao gấp 3 lần chiều cao của đối tượng)
18	matrix3d(n, n, n, n, n, n, n, n, n, n, n, n, n, n, n, n)	số	Phép biến đổi 3D, sử dụng ma trận 4x4 gồm 16 giá trị

Bảng 4.12: Thuộc tính transform 3D



Hình 4.39: Thuộc tính translate 3D



Hình 4.40: Thuộc tính rotate 3D

4.9.2. Thuộc tính transition

- Chức năng: điều khiển tốc độ xảy ra của hiệu ứng.
- Các thuộc tính trong transition

STT	Tên thuộc tính	Giá trị	Chức năng
1	transition	[property] [duration] [timing] [delay]	Thuộc tính rút gọn của các thuộc tính.
2	transition-property	all	Xác định hiệu ứng chuyển động cho tất cả thuộc tính.
		none	Hiệu ứng chuyển đổi không được hiển thị.
		width, height, …	Xác định thuộc tính của phần tử sẽ thay đổi để tạo hiệu ứng chuyển động Mỗi thuộc tính cách nhau bằng dấu phẩy.
3	transition-duration	thời gian	Xác định thời gian diễn ra của hiệu ứng
4	transition-delay	thời gian	Xác định thời gian trễ trước khi bắt đầu thực hiện hiệu ứng.
5	transition-timing-function	ease	Chuyển động ban đầu chậm, sau đó nhanh, đến lúc kết thúc sẽ từ từ (mặc định)
		linear	Chuyển động sẽ cùng tốc độ từ lúc bắt đầu tới lúc kết thúc.
		ease-in	Chuyển động ban đầu chậm, sau đó nhanh dần.
		ease-out	Chuyển động ban đầu nhanh, sau đó sẽ chậm dần.
		ease-in-out	Chuyển động ban đầu chậm, sau đó nhanh, đến lúc kết thúc sẽ chậm dần.
		cubic-bezier(n,n,n,n)	Giá trị riêng cho chuyển động, giá trị từ 0 tới 1.

Bảng 4.13: Thuộc tính transition

4.9.3. Thuộc tính animation

- Chức năng: Tạo các hiệu ứng hoạt hình trên các phần tử mà không cần dùng đến javasript hay flash.
- Xây dựng kịch bản cho hiệu ứng hoạt hình bằng **@keyframes** theo cú pháp sau:

```
@keyframes Tên-hiệu-ứng
{
 /* cách 1 */
 from{…}
 to{…}

/* cách 2 */
 0%{…}
 20%{…}
 …
 100%{…}
```

```
}
```

- Các thuộc tính trong animation

STT	Tên thuộc tính	Giá trị	Chức năng
1	animation	[name] [duration] [timing] [delay] [interaction-count] [direction]	Thuộc tính rút gọn của các thuộc tính.
2	animation-name	tên hiệu ứng	Xác định tên cho một hiệu ứng.
3	animation - duration	thời gian	Xác định thời gian hoàn thành 1 hiệu ứng
4	animation - timing-function	ease	Chuyển động ban đầu chậm, sau đó nhanh, đến lúc kết thúc sẽ từ từ (mặc định)
		linear	Chuyển động sẽ cùng tốc độ từ lúc bắt đầu tới lúc kết thúc.
		ease-in	Chuyển động ban đầu chậm, sau đó nhanh dần.
		ease-out	Chuyển động ban đầu nhanh, sau đó sẽ chậm dần.
		ease-in-out	Chuyển động ban đầu chậm, sau đó nhanh, đến lúc kết thúc sẽ chậm dần.
		cubic-bezier(n,n,n,n)	Giá trị riêng cho chuyển động, giá trị từ 0 tới 1.
5	transition-delay	thời gian	Xác định thời gian trễ trước khi bắt đầu thực hiện hiệu ứng.
6	interaction-count	số tự nhiên	Xác định số lần thực hiện chuyển động
		infinitive	Chuyển động sẽ thực hiện không giới hạn số lần.
7	animation-direction	alternate	Chuyển động sẽ được đảo ngược ở chu kỳ tiếp theo
8	animation-play-state	paused	Xác định chuyển động dừng lại.
		running	Xác định chuyển động chạy
9	animation-fill-mode	none	Xác định trạng thái của phần tử trước khi animation chạy và sau khi animation kết thúc.

		forwards	Sau khi animation hoàn tất, trạng thái của thành phần này trả về cái cuối cùng
		backwards	Sau khi animation hoàn tất, trạng thái của thành phần này trả về cái đầu tiên
		both	Trước khi animation bắt đầu, thành phần sẽ trả về cái đầu tiên và khi nó kết thúc sẽ là cái cuối cùng.

Bảng 4.14: Thuộc tính animation

BÀI TẬP CHƯƠNG 4

Bài tập 1

Xây dựng menu có định dạng như hình 2.6, 2.7, 2.8, 2.9, 2.10.

HD: Sử dụng thuộc tính float hoặc display

Bài tập 2

Xây dựng bài báo có định dạng như hình 2.11

HD: Sử dụng thuộc tính column-count, column-gap, column-rule

Bài tập 3

Xây dựng header có định dạng như hình 2.12

HD: Sử dụng thuộc tính float hoặc flexbox

Bài tập 4

Xây dựng footer có định dạng như hình 2.13

HD: Sử dụng thuộc tính float hoặc flexbox hoặc grid

Bài tập 5

Xây dựng layout-1 có định dạng như hình 2.14

HD: Sử dụng flexbox hoặc grid

Bài tập 6

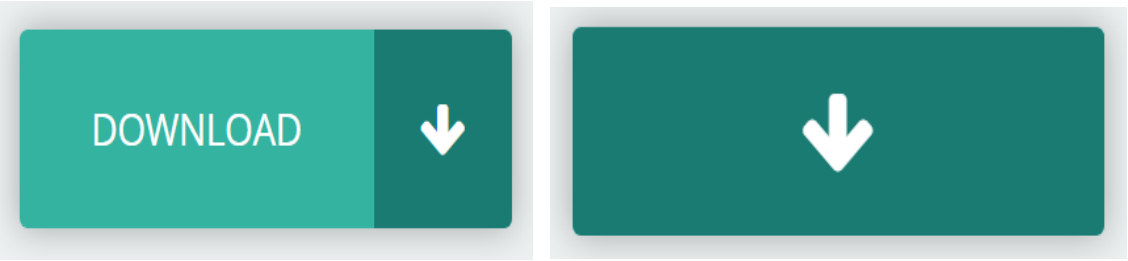
Xây dựng các trang của website An Nhiên Café (layout-2) có định dạng như:

a. Trang chủ (hình 2.15)
b. Trang Hot Coffee (hình 2.16)
c. Trang Fruit Juice (hình 2.17)
d. Trang Speciality (hình 2.18)
e. Trang About us (hình 2.19)
f. Trang Contact (hình 2.20)

Bài tập 7

Tạo nút lệnh theo yêu cầu sau:

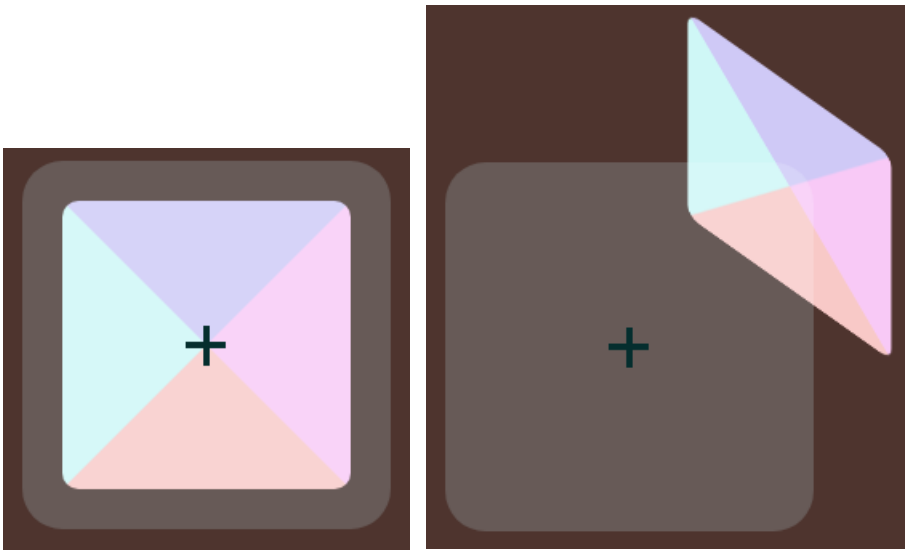
a. Ban đầu nút lệnh có định dạng như hình bên trái
b. Khi hover vào vùng mũi tên thì sẽ có định dạng như hình bên phải



Hình 4.41: Bài tập tạo nút lệnh download

Bài tập 8

Tạo khối hộp có định dạng như hình bên trái, thiết lập giá trị các thuộc tính trong transform để có được định dạng hình bên phải.

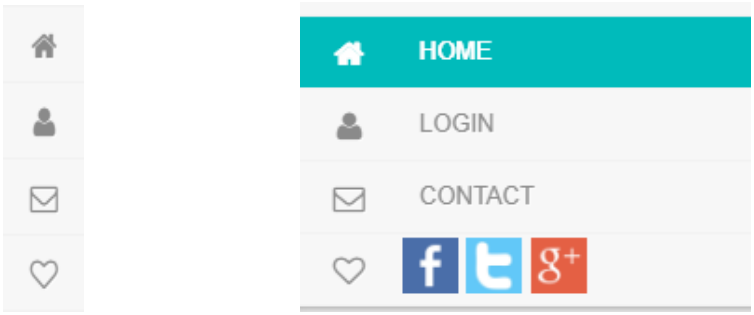


Hình 4.42: Bài tập transform

Bài tập 9

Xây dựng menu theo yêu cầu sau:

a. Ban đầu menu có định dạng như hình bên trái
b. Khi hover vào các mục thì sẽ hiện menu hoành chỉnh như hình bên phải



Hình 4.43: Bài tập tạo menu

Bài tập 10

Tạo hình vuông có kích thước tùy ý. Tạo hiệu ứng cho hình vuông theo yêu cầu sau:

a. Thời gian diễn ra hiệu ứng là 2s.
b. Hình vuông bắt đầu chuyển sang hình tròn cho đến hết, màu sắc hình tròn lần lượt chuyển từ vàng sang xanh da trời đến xanh lá cây và kết thúc màu tím.
c. Hình tròn giảm kích thước dần dần từ to đến nhỏ và khi hết thúc hiệu ứng thì không thấy nữa.
d. Hiệu ứng lần tiếp theo diễn ra theo hướng ngược lại với hiệu ứng lần trước đó.
e. Hiệu ứng lặp mãi mãi.

CHƯƠNG 5. NGÔN NGỮ JAVASCRIPT

5.1. Tổng quan về Javascript

5.1.1. Khái niệm JS

- JS được phát triển bởi Brendan Eich lần đầu vào năm 1995, xuất hiện trên trình duyệt Netscape. Ban đầu, ngôn ngữ này có tên LiveScript
- JS là ngôn ngữ kịch bản phía client có cấu trúc, hướng đối tượng và chạy trên trình duyệt, sau này còn có cả cho phía server (NodeJS)
- JS là ngôn ngữ thông dịch, bộ thông dịch được tích hợp sẵn trên trình duyệt nên không cần bất cứ công cụ nào để lập trình.

5.1.2. Ưu điểm của JS

- Tích hợp sẵn trong hầu hết các trình duyệt
- Một ngôn ngữ vô cùng linh hoạt
- Khả năng tự tương thích với trình duyệt và hệ điều hành
- Hỗ trợ lập trình hướng đối tượng
- Học một ngôn ngữ dùng mọi nơi

5.1.3. Ứng dụng của JS

- Thay đổi nội dung của trang, các định dạng CSS.
- Kiểm tra tính hợp lệ của người dùng nhập vào (form validations).
- Xác nhận dữ liệu người dùng nhập vào trước khi nó được chuyển đến server.
- Javascript không chỉ giới hạn trong khuôn khổ xây dựng ứng dụng web, mà còn sử dụng rộng rãi trong phát triển ứng dụng, game trên điện thoại hay các ứng dụng dành cho server.
  - Web app: ReactJS, VueJS, Angular...
  - Mobile app: React Native, Ionic...
  - Game: Phaser, Kiwi.js...
  - Server app: Nodejs
  - Graphic: two.js (2D), three.js (3D)...
  - AI: brain.js...

5.1.4. Khai báo JS

- Khai báo trong thẻ <head> hoặc <body>:

  <script> Các lệnh </script>

  - Đặt trong thẻ <head>: Thực hiện JS trước khi tải trang hoàn tất.
  - Đặt trước khi thẻ đóng </body>: Trang tải trước rồi mới thực hiện JS
- Tạo file bên ngoài, sau đó nhúng vào tài liệu html

  <script src="demo.js"></script>

5.1.5. Chú thích JS

- // Chú thích 1 dòng
- /* Chú thích nhiều dòng */

5.1.6. Strict Mode

- Strict Mode là một quy mẫu nghiêm khắc của Javascript. Nếu như coi việc viết code bình thường là Normal Mode, thì Strict Mode sẽ có thêm nhiều quy định khác so với Normal Mode. Việc đó khiến cho một thao tác vốn bình thường có thể chạy ngon lành trở nên lỗi, và ném ra các lỗi.
- Strict được tạo ra nhằm:
  - Ngăn chặn sử dụng, và ném lỗi khi người lập trình thực hiện những xử lý được coi là unsafe, những xử lý mà có thể là ngoài ý muốn.
  - Vô hiệu hoá các tính năng có thể gây nhầm lẫn, hoặc không nên được sử dụng.
  - Ngăn chặn sử dụng một số từ mà có thể sẽ được sử dụng làm keywork trong tương lai.
- Strict Mode có nhiều hạn chế hơn so với normal mode. Với việc tuân theo những quy định đố sẽ làm cho code Javascript trở nên sáng sủa, dễ đọc hơn, cũng như ít vướng phải những lỗi không mong muốn.
- Khai báo Strict Mode toàn cục là khai báo sau thẻ script, cục là ngay sau dấu bắt đầu các hàm con

```
<script>
  "use strict";
</script>
```

5.1.7. Developer Tools

- Dev Tools (developer tools) là công cụ được tích hợp sẵn trong trình duyệt, giúp các lập trình viên có thể dễ dàng truy cập vào nội bộ của trình duyệt và ứng dụng trang web của họ.
- Google Chrome Developer Tools là Dev tools trên trình duyệt Chrome. Để xem lỗi chi tiết, mở Chrome Dev Tools lên bằng cách nhấn phím **F12** hoặc tổ hợp phím **Ctrl + Shift + I** trên Windows (**Ctrl + Opt + J** trên MacOS). Nhấn vào tab Console để xem thông báo lỗi (nếu có)
  - Dòng màu đỏ: thông báo lỗi ("Uncaught ReferenceError: bug is not defined").
  - Bên phải hoặc ở dưới: hiển thị vị trí lỗi, chính xác với số dòng ("bug:20" - ở đây là dòng 20). Nhấn vào đó, Chrome Dev Tools sẽ hiển thị chính xác dòng bị lỗi trong mã nguồn.
  - Bên dưới dòng lỗi màu đỏ là kí tự > màu xanh. Đó là nơi có thể gõ lệnh JavaScript vào và chạy trực tiếp bằng cách nhấn Enter.
  - Để gõ mã nguồn trên nhiều dòng, nhấn tổ hợp phím Shift + Enter
- Console.log trong JavaScript là một hàm được sử dụng để in ra bất kể thứ gì (với trình duyệt thì nơi hiển thị kết quả đó là tab console của Dev Tools).
  Nhấn F12 để mở Dev Tools, sau đó nhấn vào tab Console và nhập dòng lệnh sau console.log(“Hello”). Kết quả in ra trên màn hình chữ Hello.
- Ngoài ra, có thể truy cập vào các trang web Text Editor Online để code JS như:
  - Jsfiddle.net
  - Playcode.io
  - Codesandbox.io

5.2. Biến và Hằng

5.2.1. Biến

- Biến là đại lượng có giá trị thay đổi trong chương trình
- Khai báo biến: **let tên-biến**
- Gán giá trị cho biến: tên-biến = giá-trị
- Quy tắc đặt tên biến
  - Bắt đầu bằng chữ cái, dấu gạch dưới (_) hoặc kí tự "đô la" ($).
  - Sau kí tự đầu tiên, ngoài những kí tự trên, có thể sử dụng thêm số (0-9).
  - Không sử dụng từ khoá và từ dự trữ.

  Danh sách các từ khóa

  ```
  break  case  catch  class  const  continue  debugger  default  delete
  do  else  enum  export  extends  false  finally  for  function  if
  implements  import  in  instanceof  interface  let  new  null
  package  private  protected  public  return  static  super  switch
  this  throw  true  try  typeof  var  void  while  with  yield
  ```

- Chú ý:
  - Tên biến trong JavaScript có phân biệt hoa thường.
  - Có thể sử dụng những chữ cái không phải chữ cái Latinh để đặt tên biến. Tuy nhiên nên sử dụng chữ cái Latinh (a-z) để đặt tên biến.
  - Nên đặt theo chuẩn tiếng Anh để mọi người đều có thể hiểu code của mình.
  - Nên đặt tên biến dễ nhìn, dễ đọc và thống nhất theo một chuẩn.
    Tên biến có nhiều từ thì từ đầu viết thường và viết hoa chữ cái đầu tiên của các từ phía sau (gọi là camelCase). Ví dụ: totalStudent

5.2.2. Hằng

- Hằng là đại lượng có giá trị không thay đổi trong chương trình
- Khai báo hằng: **const tên-hằng = giá-trị-hằng**

```
const PI = 3.14159, MAX = 1000, MIN = 0;
```

- Quy tắc đặt tên hằng tương tự như tên biến

5.3. Các kiểu dữ liệu trong JS

5.3.1. Các kiểu dữ liệu

- Đối với nhiều ngôn ngữ lập trình, khi khai báo một biến, phải chỉ rõ biến đó thuộc kiểu dữ liệu gì. Nhưng đối với JS, một biến có thể thuộc bất kỳ kiểu dữ liệu nào.
  - Ưu điểm: kiểu dữ liệu linh động giúp việc lập trình trở nên linh hoạt hơn.
  - Nhược điểm: khi chương trình trở nên phức tạp, khó kiểm soát kiểu dữ liệu của biến, dẫn đến sai trong việc xử lý, tính toán.
- JS có 8 kiểu dữ liệu cơ bản: 7 kiểu dữ liệu và 1 kiểu dữ liệu tham chiếu (object)

5.3.1.1. Kiểu null

Kiểu dữ liệu null là một kiểu dữ liệu đặc biệt, chỉ bao gồm một giá trị là null.

5.3.1.2. Kiểu undefined

- Kiểu undefined là một kiểu dữ liệu đặc biệt, chỉ bao gồm một giá trị undefined.

- Kiểu dữ liệu undefined có nghĩa là giá trị chưa được gán.
- Khác nhau cơ bản giữa undefined và null:
  - Kiểu dữ liệu null là kiểu dữ liệu được gán cho biến, thường được hiểu là không biết (không có). Ví dụ: a = null;
  - Trong khi đó, kiểu dữ liệu undefined là giá trị mặc định của biến sau khi khai báo mà không gán giá trị cho biến.

5.3.1.3. Kiểu boolean

Bao gồm 2 giá trị là true và false

5.3.1.4. Kiểu number

- JS có 2 loại số: số nguyên và số thực

```
let a1=10, a2=1e3, a3=4.5e9, a4=1_000_000, a5=1e-3; // 0.001
let b=-3, c=5.2, d=-9.6;
let n1=3e2;
let n2= 0XFE; // hệ thập lục phân
let n3=067;  // hệ hệ bát phân
let n4=0b10; // hệ nhị phân
```

- Ngoài ra, JavaScript còn có 3 số đặc biệt là: Infinity, -Infinity và NaN.
  - Infinity: là số dương vô cùng, lớn hơn bất kỳ số nào khác.
    Có thể sử dụng giá trị này trực tiếp hoặc lấy số dương bất kỳ chia cho 0.
  - -Infinity: là số âm vô cùng, nhỏ hơn bất kỳ số nào khác
    Có thể sử dụng giá trị này trực tiếp hoặc lấy số âm bất kỳ chia cho 0.
  - NaN: là viết tắt của Not a Number, được sử dụng để đại diện cho những trường hợp tính toán sai hoặc kết quả của một phép tính không xác định.

5.3.1.5. Kiểu bitint

- Dùng để biểu diễn số nguyên cực kì lớn.
- Để biểu diễn số nguyên với kiểu bigint, chỉ cần thêm chữ cái n ở phía sau số.

```
let n = 123564384965656565656553n;
```

5.3.1.6. Kiểu string

- String là kiểu dữ liệu dùng để biểu diễn chữ, văn bản, đoạn văn bản,...
- String trong JS có thể chỉ gồm 1 kí tự "a", nhiều kí tự "abc" hoặc không kí tự nào "" (empty string).
- Có ba cách để biểu diễn string trong JavaScript:
  - Dùng dấu nháy đơn (')
  - Dùng dấu nháy kép (")
  - Dùng dấu "backtick" (`): có thể sử dụng biến, hằng hoặc một biểu thức trong đó, với cú pháp ${…}

```
let name = "Dung";
console.log(`Ten toi la ${name}`); // Ten toi la Dung
console.log(`5 * 2 = ${5 * 2}`); // 5 * 2 = 10
```

5.3.1.7. Kiểu symbol

Symbol là một kiểu dữ liệu nguyên thủy dùng để tạo ra các giá trị duy nhất (unique value) và bất biến (immutable). Symbol thường được dùng làm key cho kiểu dữ liệu object.

- Để tạo symbol mới, bạn gọi hàm Symbol() với giá trị truyền vào là description (không bắt buộc) để miêu tả symbol.
- Symbol là duy nhất dù có tạo ra các symbol với cùng mô tả thì chúng vẫn khác nhau.
- Để trả về symbol giống nhau với cùng một key, có thể dùng Symbol.for(key). Phương thức này kiểm tra trong global, nếu tồn tại symbol với key như vậy thì trả về symbol tương ứng, ngược lại thì tạo mới symbol.
- Symbol thường được sử dụng để làm thuộc tính trong object, giúp tránh gây xung đột về tên.

5.3.1.8. Kiểu object

Object là kiểu dữ liệu tham chiếu, nghĩa là tập hợp của các thuộc tính và giá trị (key - value). Trong đó, số lượng các key có thể thay đổi, giá trị ứng với key cũng có thể thay đổi.

5.3.2. Xác định kiểu dữ liệu

Để xác định kiểu dữ liệu hiện tại của biến, có thể dùng toán tử **typeof tên-biến** hoặc hàm **typeof(tên-biến)**

```
let a;
console.log(typeof a); // undefined
x = true;
console.log(typeof x); // boolean
y = Symbol("id");
console.log(typeof y); // symbol
z = null;
console.log(typeof z); // object
// đây là lỗi của JS; thực chất null không phải là object
```

5.3.3. Chuyển đổi kiểu dữ liệu

- Chuyển sang kiểu string: string(value)

```
console.log(String(13));        // "13"
console.log(String(NaN));       // "NaN"
console.log(String(Infinity)); // "Infinity"
console.log(String(true));      // "true"
console.log(String(null));      // "null"
console.log(String(undefined));// "undefined"
```

- Chuyển sang kiểu số: number(value)

```
console.log(Number(""));        // 0
console.log(Number(" "));       // 0
console.log(Number("10"));      // 10
console.log(Number(" 10"));     // 10
console.log(Number(" 10 "));    // 10
console.log(Number("10abc"));   // NaN
console.log(Number("10 abc")); // NaN
console.log(Number("Hello"));   // NaN
console.log(Number(true));      // 1
console.log(Number(false));     // 0
console.log(Number(null));      // 0
console.log(Number(undefined));// NaN
```

- Chuyển đổi sang boolean: boolean(value)

```
console.log(Boolean(0));        // false
console.log(Boolean(""));       // false
console.log(Boolean(NaN));      // false
console.log(Boolean(null));     // false
```

```
console.log(Boolean(undefined)); // false
console.log(Boolean(1));         // true
console.log(Boolean(10n));       // true
console.log(Boolean(Infinity));// true
console.log(Boolean("hello")); // true
console.log(Boolean("  "));     // true
```

5.4. Các toán tử trong JS

Trong JS có các loại toán tử sau:

- Toán tử số học bao gồm:
  - Toán tử cộng (+)
  - Toán tử trừ (-)
  - Toán tử nhân (*)
  - Toán tử chia (/)
  - Toán tử chia lấy dư (%)
  - Toán tử lũy thừa (**)
- Toán tử ghép chuỗi +
- Toán tử tăng/giảm 1 đơn vị ++/--
- Toán tử gán (=)
- Toán tử bitwise: áp dụng trên đối tượng bit
  - Toán tử And (&)
  - Toán tử Or (|)
  - Toán tử Xor (^)
  - Toán tử Not (~)
  - Toán tử dịch trái (<<)
  - Toán tử dịch phải (>>)
  - Toán tử dịch phải (chèn thêm số 0 ở đầu) (>>>)
- Toán tử dấu phẩy: các biểu thức cách nhau bằng dấu phẩy
- Toán tử so sánh
  - Toán tử so sánh lớn hơn > và toán tử so sánh nhỏ hơn <.
  - Toán tử so sánh lớn hơn hoặc bằng >= và toán tử so sánh nhỏ hơn hoặc bằng <=.
  - Toán tử so sánh bằng "không nghiêm ngặt" == và toán tử so sánh bằng "nghiêm ngặt" ===.
  - Toán tử so sánh khác "không nghiêm ngặt" != và toán tử so sánh khác "nghiêm ngặt" !==.
  - So sánh 2 chuỗi bằng cách so sánh từng kí tự từ trái sang phải:
    - So sánh kí tự đầu tiên của hai string (dựa vào bảng mã Unicode của kí tự).
      - Nếu kí tự đầu tiên của string(1) lớn hơn (hoặc nhỏ hơn) kí tự đầu tiên của string(2) thì string(1) lớn hơn (hoặc nhỏ hơn) string(2).
      - Ngược lại, nếu hai kí tự đầu tiên bằng nhau thì tiếp tục so sánh đến các kí tự thứ hai.
    - Lặp lại việc so sánh như trên, cho đến khi so sánh kết thúc kết thúc hoặc đã so sánh hết các kí tự của ít nhất một string.

        - Nếu hai string có cùng độ dài thì chúng bằng nhau.
        - Ngược lại, string nào dài hơn thì string đó lớn hơn.
    - Kết quả của biểu thức so sánh là trả về giá trị true hoặc false
- Toán tử logic:
    - Toán tử logic là toán tử kết nối hai hay nhiều biểu thức, dùng để kiểm tra mối quan hệ logic giữa các biểu thức. Kết quả cuối cùng phụ thuộc vào giá trị của từng biểu thức và loại toán tử logic.
    - Toán tử logic áp dụng cho bất kỳ kiểu dữ liệu nào và kết quả trả về cũng là bất kỳ kiểu dữ liệu nào.
    - Để chuyển một giá trị về kiểu dữ liệu boolean, có thể dùng hàm Boolean()
    - Các toán tử logic: not (!), and (&&), or (||)

5.5. Các lệnh cơ bản

5.5.1. Lệnh rẽ nhánh

- Có hai cấu trúc rẽ nhánh trong JavaScript là:
    - Câu lệnh điều kiện rẽ nhánh - if/else
    - Toán tử rẽ nhánh - ?:
- Câu lệnh if(...) sẽ kiểm tra điều kiện biểu thức bên trong cặp dấu ngoặc đơn ().
    - Nếu kết quả là true thì một khối code sẽ được thực thi.
    - Nếu kết quả là false thì có thể dùng mệnh đề else để thực thi một khối lệnh tương ứng khác.
- Toán tử ? thường chỉ dùng để gán giá trị cho một biến dựa trên một điều kiện.
- Ví dụ:

```
// Ví dụ 1
<script>
      let value = prompt("Nhập vào số nguyên chẵn:", 0);
      if (value % 2 === 0){
        alert("Bạn đã nhập đúng!");
      }
      else{
        alert("Bạn đã nhập sai!");
      }
</script>

// Ví dụ 2
let value= prompt("Nhập vào số nguyên chẵn:", 0);
let result;
result = (Number(value%2)==0) ? "Số chẵn" : "Số lẻ";
console.log(result);
```

- Toán tử ?? là toán tử 2 ngôi dùng để trả về giá trị khác null và undefined đầu tiên, a ?? b nghĩa là:
    - Nếu a khác null và undefined thì kết quả của a ?? b là a.
    - Ngược lại, nếu a bằng null hoặc undefined thì kết quả của a ?? b là b.

  Toán tử ?? thường dùng để gán giá trị mặc định cho biến (có thể null hoặc undefined)

Toán tử ?? có độ ưu tiên cao hơn toán tử gán = và toán tử dấu hỏi ?, nhưng thấp hơn hầu hết các toán tử còn lại. Vì vậy, nên sử dụng cặp dấu ngoặc đơn () để tránh mắc lỗi không mong muốn.

JavaScript cấm sử dụng toán tử ?? trực tiếp với toán tử || hoặc toán tử && - trừ khi sử dụng cùng với cặp dấu ngoặc đơn ().

5.5.2. Lệnh lựa chọn

- Lệnh lựa chọn (switch – case) là một cấu trúc rẽ nhánh, dùng để xác định một danh sách các trường hợp và khối lệnh tương ứng với mỗi trường hợp.
- Cú pháp:

```
switch(x) {
  case 'value1':  // if (x === 'value1')
    ...
    [break]
  case 'value2':  // if (x === 'value2')
    ...
    [break]
  default:
    ...
    [break]
}
```

  - Giá trị x được kiểm tra bằng nghiêm ngặt lần lượt với các giá trị value1, value2,...
  - Khi tìm thấy giá trị value thỏa mãn thì khối lệnh bắt đầu từ case đó được thực hiện cho đến khi gặp từ khóa break gần nhất, hoặc kết lúc lệnh switch case.
  - Nếu không có trường hợp nào thỏa mãn thì khối lệnh ứng với default được thực thi. Từ khóa break là không bắt buộc.

5.5.3. Lệnh lặp

5.5.3.1. Lệnh lặp for

- Cú pháp:

```
for([khởi tạo]; [điều kiện]; [biến đổi]){
      // các lệnh;
}
```

- Ví dụ:

```
let count;
for (count = 1; count <= 10; count++) {
  console.log(count);
}
```

5.5.3.2. Lệnh lặp while

- Cú pháp:

```
while (điều kiện) {
  // các lệnh;
}
```

- Ví dụ:

```
let count = 1;
while (count <= 10) {
  console.log(count);
  count++;
}
```

5.5.3.3. Lệnh lặp do...while

- Khác với vòng lặp while, vòng lặp do...while luôn thực hiện ít nhất một lần lặp. Sau đó mới kiểm tra điều kiện lặp.
- Cú pháp:

```
do {
  // các lệnh;
} while (điều kiện);
```

- Ví dụ:

```
let count = 1;
do {
  console.log(count);
  count++;
} while (count <= 10);
```

5.5.3.4. Break và Continue

- Từ khóa break dùng để thoát khỏi vòng lặp
- Từ khóa continue dùng để dừng lượt lặp hiện tại và chuyển tới lượt lặp tiếp theo.

5.6. Hàm

5.6.1. Khai báo hàm

- Khi nào nên viết hàm:
  - Nếu một đoạn chương trình được lặp đi, lặp lại nhiều lần thì nên nhóm chúng lại thành một hàm, sẽ giúp chương trình ngắn gọn hơn, dễ hiểu hơn. Ngoài ra, khi cần thay đổi, chỉ cần sửa một lần trong nội dung hàm, thay vì phải sửa nhiều chỗ.
  - Khi cần tách toàn bộ chương trình thành những phần nhỏ hơn, còn gọi là module hoá. Lúc này, có thể xây dựng logic cho toàn bộ chương trình trước khi định nghĩa nội dung từng hàm.
- Hàm cần thỏa mãn các điều kiện sau:
  - Một hàm chỉ thực hiện một chức năng duy nhất
  - Dễ dàng tái sử dụng
  - Độc lập với các yếu tố bên ngoài (biến toàn cục, môi trường thực thi,...)
- Cú pháp: hàm dạng function declaration

```
function functionName([parameter1], [parameter2],...) {
  statement1;
  statement2;
  ...
}
```

  Trong đó:

  - function: là một từ khóa dùng để định nghĩa hàm.
  - functionName: là tên của hàm (tên hàm đặt theo qui tắc tương tự tên biến).
  - ([parameter1], [parameter2],...): danh sách các tham số của hàm (không bắt buộc).
  - Thân hàm bao gồm một hoặc nhiều câu lệnh nằm trong cặp dấu ngoặc {}.
- Tạo hàm và sử dụng hàm

```
function view() {
  console.log("Hello");
```

```
}
view();
```

- Biến cục bộ là biến được khai báo bên trong hàm, chỉ sử dụng được bên trong thân hàm đó

```
function view() {
  let message = "Hello";
  console.log(message);
}
view();
console.log(message);
```

- Biến toàn cục là biến được khai báo ngoài hàm, biến này có thể sử dụng và thay đổi giá trị ở mọi nơi trong chương trình (Hạn chế sử dụng biến toán cục). Nếu trong hàm con có khai báo biến cục bộ trùng tên với biến toàn cục thì biến toàn cục sẽ bị bỏ qua.

```
let message = "Hello";
function view() {
  console.log(message);
}
view();
// thay đổi giá trị của biến toàn cục
let message = "Hello";
function view() {
  let message = "Goodbye";
  console.log(message);
}
view();
```

- Truyền tham số cho hàm

```
function view(message) {
  console.log(message);
}
view("Hello!");
view("Goodbye!");
```

- Từ khóa return

```
function sum(a, b) {
  return a + b;
}
let result = sum(15,3);
console.log(result);
```

  - Từ khóa return có thể đặt ở bất kỳ đâu trong thân hàm. Khi gặp từ khóa return, hàm sẽ dừng lại và trả về giá trị sau return.
  - Không được xuống dòng ngay sau return vì JS sẽ hiểu là không return giá trị nào.

```
function sum(a, b) {
  if (a === null || a === undefined) {
    console.log("Không hợp lệ!");
    return; // undefined
  }

  if (b === null || b === undefined) {
    console.log("Không hợp lệ!");
    return; // undefined
  }
  return a + b;
```

```
}
let result = sum();
```

5.6.2. Biểu thức hàm

- Biểu thức hàm (Function Expression) là hàm được định nghĩa trong 1 biểu thức.
- Cú pháp: hàm dạng function expression

```
let tên-hàm = function () {
  các lệnh;
};
```

- Ví dụ:

```
let view = function () {
  console.log("Hello");
};
console.log(view);
```

- Biểu thức hàm cũng là một giá trị, nên có thể gán nó cho một biến khác.

```
let view = function () {
  console.log("Hello");
};
let view1 = view;
view();
view1();
```

- So sánh giữa function declaration và function expression
  - Function declaration
    - Định nghĩa độc lập
    - Được xử lý trước khi chương trình chạy đến đó.
    - Có phạm vi trong block, nên có thể gọi hàm ở bất kỳ đâu trong block.
  - Function expression
    - Định nghĩa bên trong một biểu thức
    - Được tạo ra tại thời điểm chương trình chạy đến vị trí khởi tạo hàm, nên chỉ gọi được hàm sau khi đã khởi tạo.
- Hàm "callback" là hàm được truyền vào bên trong hàm khác để gọi lại khi cần thiết.

5.6.3. Hàm mũi tên

- Hàm mũi tên (Arrow function) là hàm sử dụng kí hiệu => để định nghĩa, giúp việc khai báo hàm trở nên ngắn gọn hơn.
- Cú pháp:

```
let tên-hàm = (arg1, arg2,..., argN) => expression;
```

Đây thực chất là cách biểu diễn rút gọn của biểu thức hàm

```
let tên-hàm = function(arg1, arg2,..., argN) {
      return expression;
    }
```

- Ví dụ:

```
let sum = (a,b) => a + b;
console.log(sum(10,5)); // 15
```

- Sử dụng hàm

```
// Ví dụ khởi tạo hàm dựa trên điều kiện
let isStudent = confirm("Bạn có phải sinh viên không?");
```

```
let welcome = isStudent
  ? () => console.log("Chào bạn!")
  : () => console.log("Chào!");
welcome();

// Ví dụ sử dụng hàm làm hàm callback
function ask(question, handleYes, handleNo) {
  const answer = confirm(question);
  if (answer) {
    handleYes();
  } else {
    handleNo();
  }
}
ask(
  "Bạn muốn tiếp tục thực hiện chương trình không?",
  () => console.log("Bạn đã chọn Yes!"),
  () => console.log("Bạn đã chọn No!"),
);
```

5.6.4. Các hàm tương tác người dùng

- alert("Nội dung"): Hàm hiển thị hộp thoại thông báo bao gồm nội dung và nút OK
- confirm("Nội dung"): Hàm hiển thị hộp thoại xác nhận bao gồm nội dung, nút OK và Cancel
- prompt("Nội dung"): Hàm hiển thị hộp thoại cho phép người dùng nhập chuỗi

5.7. Object

5.7.1. Khái niệm Object

- Object là một khái niệm trừu tượng dùng để biểu diễn một đối tượng cụ thể. Trong đó, các thuộc tính dùng để miêu tả đặc điểm, tính chất của đối tượng.
- Cú pháp:

```
let tên-object={
  key1: value1,
  key2: value2,
  key3: value3,
  ...
}
```

Trong đó:

- key1, key2, key3,...: là tên thuộc tính, có kiểu dữ liệu là string hoặc symbol.
- value1, value2, value3,...: là giá trị tương ứng của các thuộc tính. Value có thể thuộc bất kỳ kiểu dữ liệu nào (kể cả object).
- Các cặp key-value ngăn cách nhau bởi dấu phẩy ,

- Ví dụ:

```
let myComputer = {
  type: "laptop",
  brand: "Dell",
  os: "Windows 8",
};
// Object rỗng không có thuộc tính nào
let myComputer = {};

// Hoặc sử dụng hàm khởi tạo object (ít dùng)
let yourComputer = new Object();
```

5.7.2. Cách sử dụng Object

- Có 2 cách để lấy ra giá trị của một thuộc tính trong object:
  - Sử dụng toán tử chấm . theo sau là tên thuộc tính.
    Toán tử . chỉ dùng được trong trường hợp tên thuộc tính không có kí tự đặc biệt (ngoại trừ kí tự _ và $).
  - Sử dụng toán tử ngoặc vuông [ ], bên trong là tên thuộc tính.
- Ví dụ:

```
let myComputer = {
  type: "laptop",
  brand: "Dell",
  os: "Windows 8",
};
console.log(myComputer.type);   // laptop
console.log(myComputer.brand); // Dell
console.log(myComputer.os);     // Windows 8
// hoặc
console.log(myComputer["type"]);
console.log(myComputer["brand"]);
console.log(myComputer["os"]);
// hoặc
console.log(myComputer);
```

- Thay đổi giá trị thuộc tính của Object

```
let myComputer = {
  type: "laptop",
  brand: "Dell",
  os: "Windows 8",
};
myComputer.type = "desktop";
console.log(myComputer.type); // desktop
```

- Thêm thuộc tính cho Object

```
let myComputer = {
  type: "laptop",
  brand: "Dell",
  os: "Windows 8",
};
myComputer.status = "sleep";
console.log(myComputer.status); // sleep
```

- Xóa thuộc tính của Object

```
let myComputer = {
  type: "laptop",
  brand: "Dell",
  os: "Windows 8",
};
delete myComputer.os;
console.log(myComputer.os); // undefined
```

- Kiểm tra tính tồn tại của thuộc tính

```
let myComputer = {
  type: "laptop",
  brand: "Dell",
  os: "Windows 8",
};
console.log("type" in myComputer); // true
```

- Hàm khởi tạo Object

```
function makeComputer(type, brand, os) {
  return {
    type: type,
    brand: brand,
    os: os,
  };
}
// hoặc rút gọn hơn
function makeComputer(type, brand, os) {
  return { type, brand, os, };
}
// Ví dụ
let myComputer = makeComputer("laptop", "Dell", "Windows 8");
console.log(myComputer.type); // laptop
console.log(myComputer.brand); // Dell
console.log(myComputer.os); // Windows 8
```

- Duyệt tất cả các thuộc tính của Object

```
let myComputer = {
  type: "laptop",
  brand: "Dell",
  os: "Windows 8",
};
for (let a in myComputer) {
  console.log(a, ":", myComputer[a]);
}
// kết quả
// type:laptop
// brand:Dell
// os:Windows 8
```

- Thứ tự các thuộc tính trong Object

Khi duyệt tất cả các thuộc tính của object thì có hai trường hợp xảy ra:

  - Đối với thuộc tính kiểu số nguyên không dấu: thứ tự được sắp xếp theo chiều tăng dần (từ số nhỏ đến số lớn).
  - Đối với thuộc tính kiểu dữ liệu còn lại, thứ tự giống với thứ tự thêm vào object.

- Các đối tượng có sẵn trong JS
  - Đối tượng Arguments: có thuộc tính length cho biết số lượng tham số truyền vào của hàm

```
function maxOf() {
  let max = 0;
  for (let i = 0; i < arguments.length; i++) {
    if (arguments[i] > max) max = arguments[i];
  }
  return max;
}
console.log(maxOf(1,5,8));                    // max = 8
console.log(maxOf(10, 2, 6, 4, 19, 3, 5)); // max = 19
```

  - Đối tượng Math: gồm các thuộc tính như Math.min, Math.abs, Math.sqrt, …
  - Đối tượng Global: object lớn nhất là window, mọi biến toàn cục đều thuộc nó.

5.7.3. So sánh 2 Object

- So sánh 2 object là so sánh theo địa chỉ (chứ không phải theo giá trị).

- Khi gán một object cho một biến, thực chất là biến đó đang lưu địa chỉ trong bộ nhớ của object. Khi truy cập vào một thuộc tính của object, trình thông dịch sẽ dựa theo địa chỉ đã lưu để truy xuất đúng giá trị cần lấy trong bộ nhớ.
- Ví dụ

```
let p1 = { x:5, y:7 };
let p2 = p1;
p2.x = 8;
console.log(p2.x); // 8
console.log(p1.x); // 8
// p1 và p1 cùng trỏ tới cùng 1 vùng nhớ
```

- Các cách so sánh 2 object:
  - Bằng tham chiếu: hai object được gọi là bằng nhau khi và chỉ khi chúng cùng tham chiếu đến cùng một địa chỉ bộ nhớ, nghĩa là chúng hoàn toàn giống nhau.
    - Toán tử = = trả về true khi hai biến có cùng giá trị và có thể khác kiểu dữ liệu (JS sẽ chuyển về cùng kiểu dữ liệu để so sánh), ngược lại thì trả về false.
    - Toán tử = = = trả về true khi và chỉ khi hai biến có cùng kiểu dữ liệu và cùng giá trị, ngược lại thì trả về false.

```
// Ví dụ 1: x và y đang trỏ đến cùng 1 một địa chỉ
let x = {};
let y = x;
console.log(y == x); // true
console.log(y === x); // true
// Ví dụ 2: x và y đang tham chiếu tới 2 địa chỉ khác nhau nên không bằng
nhau
let x = {};
let y = {}; // khởi tạo object độc lập
console.log(y == x); // false
console.log(y === x); // false
// Ngoài ra, có thể so sánh bằng cách sử dụng hàm Object.is(value1,
value2)
console.log(Object.is(y, x)); // true
```

  - Qua giá trị cách thủ công

```
// Cách 1
let point1 = { x: 1, y: 2 };
let point2 = { x: 1, y: 2 };
let point3 = { x: 2, y: 3 };

if (point2.x === point1.x && point2.y === point1.y) {
  console.log("point2 bằng point1");
} else {
  console.log("point2 khác point1");
}
if (point3.x === point1.x && point3.y === point1.y) {
  console.log("point3 bằng point1");
} else {
  console.log("point3 khác point1");
}
// point2 bằng point1
// point3 khác point1
```

```
// Cách 2
function isPointEqual(p1, p2) {
```

```
  return p1.x === p2.x && p1.y === p2.y;
}
let point1 = { x: 1, y: 2 };
let point2 = { x: 1, y: 2 };
let point3 = { x: 2, y: 3 };
console.log(isPointEqual(point1, point2));
```

- So sánh nông (shallow equality): duyệt tất cả các thuộc tính trong mỗi object để so sánh giá trị tương ứng theo một cấp độ.

```
// Nếu Object có nhiều thuộc tính
function isShallowEqual(obj1, obj2) {
  for (let prop in obj1) {
    if (obj1[prop] !== obj2[prop]) return false;
  }
  for (let prop in obj2) {
    if (obj2[prop] !== obj1[prop]) return false;
  }
  return true;
}
let point1 = { x: 1, y: 2 };
let point2 = { x: 1, y: 2 };
let point3 = { x: 2, y: 3 };

console.log(isShallowEqual(point1, point2)); // true
console.log(isShallowEqual(point1, point3)); // false
```

Kết quả so sánh point1 và point2 lại trả về false. Nguyên nhân là thuộc tính metadata có giá trị kiểu object. Nên khi so sánh hai metadata cho nhau, chúng sẽ khác nhau. Vì vậy, thuật toán so sánh nông chỉ đúng khi giá trị các thuộc tính trong object có kiểu dữ liệu nguyên thủy.

- So sánh sâu (deep equality): thay vì duyệt một cấp, so sánh sâu sử dụng thuật toán đệ quy để duyệt tất cả các cấp của object (trường hợp object lồng nhau).

```
// Hàm kiểm tra một giá trị là object
function isObject(obj) {
  return obj != null && typeof obj === "object";
}

// Hàm so sánh sâu
function isDeepEqual(obj1, obj2) {
  const keys1 = Object.keys(obj1); // trả về mảng các thuộc tính của obj1
  const keys2 = Object.keys(obj2); // trả về mảng các thuộc tính của obj2

  // nếu số lượng keys khác nhau thì chắc chắn khác nhau
  if (keys1.length !== keys2.length) {
    return false;
  }
  for (const key of keys1) {
    const val1 = obj1[key];
    const val2 = obj2[key];

    // kiểm tra xem hai giá trị có cùng là object hay không
    const areObjects = isObject(val1) && isObject(val2);

    // nếu cùng là object thì phải gọi đệ quy để so sánh 2 object
    if (areObjects && !isDeepEqual(val1, val2)) {
      return false;
    }
```

```
    // nếu không cùng là object thì so sánh giá trị
    if (!areObjects && val1 !== val2) {
      return false;
    }
  }
  return true;
}

let point1 = { x: 1, y: 2, metadata: { type: "point" } };
let point2 = { x: 1, y: 2, metadata: { type: "point" } };

console.log(isDeepEqual(point1, point2)); // true
```

5.7.4. Sao chép Object

- Sao chép object là sao chép địa chỉ
- Các cách sao chép object
  - Copy từng thuộc tính sử dụng vòng lặp for...in.

```
let p1 = { x: 1, y: 2 };
let p2 = {};
for (let key in p1) {
  p2[key] = p1[key];
}
console.log(p2.x); // 1
console.log(p2.y); // 2

p2.x = 5;
console.log(p2.x); // 5
console.log(p1.x); // 1
```

  - Copy nông (shallow copy) sử dụng hàm Object.assign() hoặc cú pháp spread (...)

```
let p1 = { x: 1, y: 2 };
let p2 = {};
Object.assign(p2, p1);
```

  - Copy sâu (deep copy) sử dụng JSON.stringify() để chuyển object về dạng JSON và JSON.parse() để tạo lại một object mới từ JSON hoặc thư viện có sẵn. Khi object bao gồm nhiều đối tượng lồng nhau, bạn cần copy sâu (deep copy) để có thể tạo ra đối tượng độc lập

```
let point1 = {
  x: 1,
  y: 2,
  metadata: { type: "point",},
};
// chuyển object về dạng JSON
let jsonPoint1 = JSON.stringify(point1);
console.log(jsonPoint1); // {"x":1,"y":2,"metadata":{"type":"point"}}
// parse JSON lại thành object mới
let point2 = JSON.parse(jsonPoint1);
console.log(point2.metadata.type); // point

point2.metadata.type = "CHANGED";
console.log(point2.metadata.type); // CHANGED
console.log(point1.metadata.type); // point
```

5.7.5. Phương thức trong JS

- Phương thức chính là hành động của đối tượng

- Ví dụ:

```
// Cách 1
let user = {
  name: "Minh",
  age: "36",
};
user.view = function () {
  console.log("Hello!");
};
user.view(); // Hello!

// Cách 2
let user = {
  name: "Minh",
  age: "36",
};
function view() {
  console.log("Hello!");
}
user.view = view;
user.view(); // Hello!

// Cách 3
let user = {
  name: "Minh",
  age: "36",
  view(){
    console.log("Hello!");
  },
};
user.view();
```

5.7.6. Toán tử this

- This thường dùng để truy cập vào thuộc tính của object ngay bên trong phương thức.
- Hàm arrow function không có this

```
let user = {
  name: "Minh",
  age: "36",
  view(){
    console.log("Hello " + this.name);
  },
};
user.view();
```

5.7.7. Khởi tạo đối tượng bằng new

- Hàm khởi tạo là hàm dùng để khởi tạo Object
  - Hàm khởi tạo thường được viết hoa chữ cái đầu (không bắt buộc) để dễ dàng phân biệt với các hàm bình thường.
  - Hàm khởi tạo chỉ nên sử dụng với toán tử new. Khi đó, JavaScript ngầm định tạo ra một object rỗng ở đầu hàm và gán cho this. Sau đó, cuối hàm sẽ return về this.
  - Thường hàm khởi tạo không có return. Nếu hàm khởi tạo có return thì quy tắc là return với một object sẽ trả về object đó, ngược lại thì trả về this.

```
function Point(x, y) {
  this.x = x;
```

```
  this.y = y;}
let root = new Point(0, 0);
console.log(root.x, root.y); // 0 0
```

5.7.8. Chuyển đổi object về kiểu nguyên thủy

- Việc chuyển đổi object về kiểu nguyên thủy được sử dụng với nhiều hàm và toán tử trong JavaScript.
- Có ba kiểu "hint" là:
  - string: áp dụng cho hàm alert và các toán tử yêu cầu toán hạng là string.
  - number: áp dụng cho các phép toán số học.
  - default: áp dụng cho một số ít toán tử dùng được cho cả string và number (không biết chắc nên chuyển về kiểu dữ liệu nào).
- Quy luật chuyển đổi object về kiểu nguyên thủy là:
  - Gọi phương thức obj[Symbol.toPrimitive](hint) nếu nó tồn tại.
  - Trường hợp phương thức trên không tồn tại, nếu "hint" là string: ưu tiên gọi phương thức obj.toString(), nếu không tồn tại thì gọi phương thức obj.valueOf().
  - Nếu "hint" là number hoặc default: ưu tiên gọi phương thức obj.valueOf(), nếu không tồn tại thì gọi phương thức obj.toString().

5.8. Các phương thức của kiểu dữ liệu nguyên thủy

5.8.1. Phân biệt giữa kiểu dữ liệu nguyên thủy và object

- Kiểu dữ liệu nguyên thủy

Có 7 kiểu dữ liệu nguyên thủy là: string, number, bigint, boolean, symbol, null và undefined. Kiểu dữ liệu này chỉ lưu một giá trị nguyên thủy.

- Kiểu object
  - Là tập hợp của các cặp key-value.
  - Có thể khởi tạo object bằng cặp dấu {}, ví dụ { x:1, y:2 }.
  - Một số loại object đặc biệt như: function, Date, Set, Map,...
  - Điều đặc biệt nhất của object là có thể lưu hàm thành một thuộc tính trong object. Hàm đó gọi là phương thức của object.
  - Có nhiều object mặc định của JavaScript giúp thao tác với ngày tháng, trình duyệt, xử lý lỗi,... Mỗi đối tượng có những thuộc tính và phương thức khác nhau.
  - Tuy nhiên, object **tốn nhiều tài nguyên để lưu trữ** hơn.
- Để giải quyết vấn đề này, JS tạo ra một “wrapper object” chứa giá trị nguyên thủy và phương thức sử dụng. Sau khi phương thức gọi xong thì “wrapper object” được giải phóng.

  Những "wrapper object" ứng với các kiểu dữ liệu nguyên thủy là: Number, String, BigInt, Boolean, Symbol. Những đối tượng này cung cấp các phương thức khác nhau để làm việc với kiểu nguyên thủy, kiểu dữ liệu null và undefined không có "wrapper object”.

5.8.2. Kiểu Number

- Làm tròn
  - Math.ceil(): làm tròn lên

- Math.floor(): làm tròn xuống
- Math.round(): làm tròn về số nguyên gần nhất
- Math.trunc(): Làm tròn về số nguyên bằng cách xóa hết phần thập phân
- Làm tròn thành số thực với chính xác n chữ số thập phân
  - Cách 1

```
let a = 1.2345;
// a nhân với 100
let b = a * 100; // 123.45
// làm tròn b với Math.round
let c = Math.round(b); // 123
// chia c cho 100
let d = c / 100; // bằng 123 / 100
console.log(d); // 1.23
```

  - Cách 2: sử dụng phương thức toFixed(n)

```
let a = 1.2345;
let b = a.toFixed(2);     // trả về chuỗi "1.23"
let c = Number(b);        // chuyển từ chuỗi sang số
console.log(c, typeof c); // 1.23 number
```

- Sai số

```
console.log((0.1).toFixed(17)); // 0.10000000000000001
console.log((0.2).toFixed(17)); // 0.20000000000000001
console.log(0.1 + 0.2 = = = 0.3); // false vì kết quả
0.1+0.2=0.30000000000000002
// khắc phục
let a = 0.1;
let b = 0.2;

// Cách 1: Sử dụng nhân-rồi-chia
let sum1 = (a * 10 + b * 10) / 10;
console.log(sum1); // 0.3

// Cách 2: Sử dụng toFixed(1)
let sum2 = (a + b).toFixed(1);
console.log(sum2); // 0.3
```

- Kiểm tra có phải số không
  - isNaN(value): Hàm này chuyển đổi value thành dạng số rồi kiểm. Nếu trả về true không phải là số, false là số.

```
console.log(isNaN("1")); // false - "1" chuyển sang số thành 1
console.log(isNaN("abc")); // true - "abc" không thể chuyển thành số
```

  - isFinite(value): Hàm này chuyển đổi giá trị value thành dạng số và trả về true nếu số đó khác Infinity, -Infinity và NaN, ngược lại thì trả về false. Ngoài ra, hàm này cũng dùng để kiểm tra một giá trị string có phải là số hay không.
- Chuyển sang số nguyên, số thực
  - parseInt(string): tách lấy số nguyên
  - parseFloat(string): tách lấy số thực

```
console.log(parseInt("100px"));      // 100
console.log(parseFloat("1.1em"));    // 1.1
console.log(parseInt("1.2"));        // 1
console.log(parseFloat("1.2.3"));    // 1.2
console.log(parseInt("$100"));       // NaN
```

- Hàm toán học
  - Math.random(): trả về một giá trị ngẫu nhiên từ 0 đến 1 (không bao gồm 1)
  - Math.min(a,b,c, …): trả về giá trị nhỏ nhất
  - Math.max(a,b,c, …): trả về giá trị lớn nhất
  - Math.pow(x,y): trả về x^y

5.8.3. Kiểu String

- Các kí hiệu đặc biệt:
  - \n: dòng mới
  - \' hoặc \": dấu nháy đơn hoặc dấu nháy kép.
  - \\: dấu backslash.
  - \t: dấu tab.
  - \xXX: biểu diễn kí tự unicode, trong đó XX là mã unicode ở hệ thập lục phân. Ví dụ: \x7A tương đương với z.
  - \uXXXX: biểu diễn symbol unicode, trong đó XXXX là mã unicode ở hệ thập lục phân của symbol với UTF-16. Ví dụ: \u00A9 tương đương với biểu tượng copyright ©.
  - \u{X..XXX} (từ 1 đến 6 kí tự): biểu diễn symbol unicode, trong đó X..XXX là mã unicode ở hệ thập lục phân của symbol với UTF-32.
- Độ dài chuỗi: thuộc tính length
- Để lấy giá trị của kí tự ở vị trí bất kì trong string, có thể dùng str[vị trí] hoặc str.charAt(vị trí) (vị trí bắt đầu là 0).

  Điểm khác nhau giữa hai cách này là: nếu không tìm thấy vị trí thì [vị trí] trả về undefined, còn charAt(vị trí) trả về string rỗng.

```
let str = "hello";
for (let i = 0; i < str.length; i++) {
  console.log(str[i]);
}
// cách 2
for (let c of str) {
  console.log(c);
}
```

- Đổi sang hoa, thường
  - toLowerCase(): đổi sang thường
  - toUpperCase(): đổi sang hoa
- Tìm chuỗi con
  - str.indexOf(substr, pos): Phương thức này tìm kiếm substr bên trong string str bắt đầu từ vị trí pos. Kết quả trả về là vị trí kí tự đầu tiên của substring nếu tìm thấy, ngược lại thì trả về -1

```
let str = "I am a js dev";
let target = "a";

let pos = 0;
while (true) {
  let foundPos = str.indexOf(target, pos);
  if (foundPos === -1) break;
```

```
  console.log("found pos:", foundPos);
  pos = foundPos + 1;
}
```

- str.lastIndexOf(substr, pos): Tương tự như trên nhưng tìm kiếm từ cuối lên đầu.

```
let str = "I am a js dev";
console.log(str.lastIndexOf("a")); // 5
```

- str.includes(substr, pos): Phương thức này cũng tìm substr trong str, nhưng trả về true nếu tìm thấy và trả về false nếu không tìm thấy.
- str.startsWith(substr): Trả về true nếu str bắt đầu bằng substr, ngược lại trả về false
- str.endsWith(substr): Trả về true nếu str kết thúc bằng substr, ngược lại trả về false.
- str.slice(start [, end]): Trả về substring bắt đầu từ vị trí start đến vị trí end (không bao gồm vị trí end và start luôn phải nhỏ hơn end)
  - Nếu không truyền vào giá trị end thì phương thức này sẽ lấy đến cuối string
  - Nếu giá trị start và end là số âm thì vị trí được đếm từ cuối lên đầu (vị trí cuối cùng ứng với -1)
- str.substring(start [, end]): Trả về substring trong string giữa vị trí start và vị trí end (không bao gồm vị trí end). Phương thức này gần giống phương thức str.slice(start, [, end]), chỉ khác là phương thức substring cho phép start có thể lớn hơn end và substring không áp dụng được cho trường hợp start, end có giá trị âm.
- str.substr(start [, length]): Trả về substring bắt đầu từ vị trí start và lấy số kí tự là length. Nếu giá trị start là âm, thì việc lấy substring được tính từ cuối (vị trí kí tự cuối cùng là -1)

- So sánh chuỗi: thực hiện so sánh từng kí tự từ trái sang phải. Và khi so sánh string, kí tự viết thường lớn hơn kí tự viết hoa (Vì string trong JavaScript được encode bằng UTF-16, nghĩa là mỗi kí tự tương ứng với một số)
  - str.codePointAt(pos): Trả về mã code (số) tương ứng với kí tự tại vị trí pos

```
console.log("a".codePointAt(0)); // 97
console.log("A".codePointAt(0)); // 65
```

  - String.fromCodePoint(code): Trả về kí tự ứng với mã code

```
console.log(String.fromCodePoint(97)); // a
console.log(String.fromCodePoint(65)); // A

// sử dụng mã Unicode bằng cách thêm kí tự \u trước mã hexa
console.log("\u0061"); // a - mã hexa của a là 61
console.log("\u0041"); // A - mã hexa của A là 41
```

  - So sánh chuỗi Tiếng Việt: str1.localeCompare(str2)
    - Nếu kết quả trả về nhỏ hơn 0 thì str1 nhỏ hơn str2.
    - Nếu kết quả trả về lớn hơn 0 thì str1 lớn hơn str2.
    - Nếu kết quả trả về bằng 0 thì str1 bằng str2.

```
let str1 = "Hải Dương";
let str2 = "Hòa Bình";
console.log(str1.localeCompare(str2)); // -1 có nghĩa str1<str2
```

5.8.4. Kiểu Array

- Khởi tạo mảng
  - Cách 1: Sử dụng dấu ngoặc vuông [].
  - Cách 2: Sử dụng hàm khởi tạo new Array().

```
let a1 = [];
let a2 = new Array();
```

  - Các phần tử trong mảng được sắp xếp theo thứ tự, phần tử đầu tiên có chỉ số là 0. Phần tử cuối cùng có chỉ số bằng độ dài mảng trừ đi 1 đơn vị.

```
let letters = ["a", "b", "c"];
console.log(letters[0]); // a
console.log(letters[1]); // b
console.log(letters[2]); // c
// trường hợp chỉ số ngoài phạm vi giới hạn thì kết quả là undefined
console.log(letters[-1]); // undefined
console.log(letters[3]); // undefined
```

  - Số phần tử trong mảng: thuộc tính length
  - Thay đổi giá trị của phần tử trong mảng

```
let letters = ["a", "b", "c"];
// thay đổi giá trị của mảng tại chỉ số 0
letters[0] = "A";
console.log(letters[0]); // A
```

  - Thêm phần tử vào mảng

```
let letters = ["a", "b", "c"];
// thêm phần tử vào mảng tại ví trí số 3
letters[3] = "d";
console.log(letters[3]); // d
```

- Duyệt các phần tử của mảng:
  - Cách 1: Vòng lặp for với chỉ số for (let i=0; i < arr.length; i++): cách này chạy nhanh nhất.
  - Cách 2: Vòng lặp for (let item of arr): cách này ngắn gọn, nên dùng khi không quan tâm chỉ số.
  - Cách 3: Vòng lặp for (let i in arr): cách này không tối ưu cho array, không nên dùng.
  - Cách 4: sử dụng phương thức forEach

```
["a", "b", "c"].forEach(function (item, index, array) {
  console.log(`item ${item} at index ${index} in array ${array}`);
});
```

- Các phương thức của mảng
  - arr.pop(): Lấy ra và trả về phần tử cuối cùng của mảng.

```
let letters = ["a", "b", "c"];
let item = letters.pop();
console.log(item); // c
console.log(letters); // (2) ['a', 'b']
```

  - arr.push(): Thêm một hoặc nhiều phần tử vào cuối mảng và trả về độ dài mới của mảng.

```
let letters = ["a", "b", "c"];
// thêm một phần tử vào cuối mảng
```

```
let length = letters.push("d");
console.log(length); // 4
console.log(letters); // (4) ['a', 'b', 'c', 'd']
// thêm nhiều phần tử vào cuối mảng
length = letters.push("e", "f");
console.log(length); // 6
console.log(letters); // (6) ['a', 'b', 'c', 'd', 'e', 'f']
```

- arr.shift(): Lấy ra và trả về phần tử đầu tiên của mảng.

```
let letters = ["a", "b", "c"];
let item = letters.shift();
console.log(item); // a
console.log(letters); // (2) ['b', 'c']
```

- arr.unshift(): Thêm một hoặc nhiều phần tử vào đầu mảng.

```
let letters = ["a", "b", "c"];
// thêm một phần tử vào đầu mảng
let length = letters.unshift("d");
console.log(length); // 4
console.log(letters); // (4) ['d', 'a', 'b', 'c']
// thêm nhiều phần tử vào đầu mảng
length = letters.unshift("e", "f");
console.log(length); // 6
console.log(letters); // (6) ['e', 'f', 'd', 'a', 'b', 'c']
```

- arr.splice(start[, deleteCount, elem1,..., elemN]): có thể xóa, thêm hoặc thay thế phần tử của mảng
  - Phương thức splice xử lý mảng tại vị trí có chỉ số start bằng cách: xóa đi deleteCount phần tử, rồi chèn thêm các phần tử elem1,... elemN vào đúng vị trí đó. Sau đó, trả về mảng của những phần tử bị xóa.

```
// xóa đi 3 phần tử tại vị trí đầu tiên và thêm 2 phần tử khác
let letters = ["a", "b", "c"];
// từ vị trí có chỉ số 0, xóa đi 3 phần tử, rồi thêm vào 2 phần tử
letters.splice(0, 3, "d", "e");
console.log(letters); // (2) ['d', 'e']
// thêm phần tử vào mảng mà không cần xóa đi phần tử nào, bằng cách
truyền vào giá trị deleteCount bằng 0
let letters = ["a", "b", "c"];
letters.splice("1", 0, "d", "e");
console.log(letters); // (5) ['a', 'd', 'e', 'b', 'c']
```

  - splice chấp nhận chỉ số âm, nếu chỉ số âm thì thứ tự đếm là từ cuối lên đầu.

```
let letters = ["a", "b", "c"];
// chỉ số bằng -1 tức đếm 1 đơn vị từ cuối lên
// xóa đi 0 phần tử
// rồi chèn thêm 2 phần tử "d" và "e"
letters.splice(-1, 0, "d", "e");
console.log(letters); // (5) ['a', 'b', 'd', 'e', 'c']
```

- arr.slice([start], [end]): Phương thức này trả về mảng mới bằng cách copy mảng ban đầu từ vị trí start đến vị trí end (không bao gồm end). Cả hai giá trị start và end đều có thể âm, khi đó việc đếm được tính từ cuối của mảng.
  Phương thức của array arr.slice tương tự như phương thức của string str.slice, chỉ khác là trả về subarray chứ không phải là substring.

- arr.concat(arg1, arg2,...): Phương thức arr.concat trả về array mới bao gồm các giá trị của arr ban đầu, cộng thêm giá trị các phần tử trong array thêm vào hoặc các giá trị khác.
- Phương thức này chấp nhận số lượng tham số tùy ý. Và giá trị của arg1, arg2,... có thể là mảng hoặc giá trị khác.
  Giả sử phần tử argN là mảng thì tất cả các phần tử trong mảng argN được sao chép. Ngược lại, nếu giá trị của argN không phải mảng thì giá trị của chính nó được copy vào mảng

```
let arr1 = [1, 2];

// tạo mảng mới từ mảng arr1 và mảng [3, 4]
let arr2 = arr1.concat([3, 4]);
console.log(arr2); // (4) [1, 2, 3, 4]

// tạo mảng mới từ mảng arr1 và mảng [3, 4] và [5, 6]
let arr3 = arr1.concat([3, 4], [5, 6]);
console.log(arr3); // (6) [1, 2, 3, 4, 5, 6]

// tạo mảng mới từ mảng arr1 và mảng [3, 4] cùng với các giá trị 5, 6
let arr4 = arr1.concat([3, 4], 5, 6);
console.log(arr4); // (6) [1, 2, 3, 4, 5, 6]
```

- indexOf(), lastIndexOf() và includes():
  - Phương thức arr.indexOf(item, from): tìm kiếm item trong mảng, bắt đầu từ vị trí from và trả về chỉ số vị trí tìm thấy, ngược lại thì trả về -1.
  - Phương thức arr.lastIndexOf(item, from): tương tự phương thức indexOf là tìm kiếm item trong mảng và bắt đầu từ vị trí from nhưng thứ tự tìm kiếm từ phải sang trái.
  - Phương thức arr.includes(item, from): tìm kiếm item trong mảng từ vị trí from, và trả về true nếu tìm thấy, ngược lại trả về false.

```
let arr = [1, 0, 1, false];
console.log(arr.indexOf(0)); // 1
console.log(arr.indexOf(false)); // 3
console.log(arr.indexOf(null)); // -1

console.log(arr.indexOf(1)); // 0
console.log(arr.lastIndexOf(1)); // 2

console.log(arr.includes(1)); // true
```

- Find() và findIndex()
  - Phương thức arr.find(fn) tìm kiếm một phần tử trong mảng thỏa mãn hàm fn (nói cách khác là hàm fn trả về true) và trả về phần tử tìm được, ngược lại thì trả về undefined.

```
let users = [
  { id: 1, name: "Alex" },
  { id: 2, name: "John" },
  { id: 3, name: "Anna" },
];

let user = users.find((item) => item.id === 2);
console.log(user.name); // John
```

- Phương thức arr.findIndex có cú pháp hoàn toàn giống với arr.find, chỉ khác là, arr.findIndex trả về chỉ số của phần tử tìm thấy, ngược lại thì trả về -1

```
let users = [
  { id: 1, name: "Alex" },
  { id: 2, name: "John" },
  { id: 3, name: "Anna" },
];

let index = users.findIndex((item) => item.id === 2);
console.log(index); // 1
```

- Filter(): arr.find và arr.findIndex chỉ tìm kiếm phần tử đầu tiên thỏa mãn, để tìm kiếm nhiều phần tử thỏa mãn, có thể dùng phương thức arr.filter.
  Phương thức arr.filter trả về một mảng các phần tử thỏa mãn, ngược lại thì trả về mảng rỗng

```
let users = [
  { id: 1, name: "Alex" },
  { id: 2, name: "John" },
  { id: 3, name: "Anna" },
];

let results = users.filter((item) => item.id <= 2);
console.log(results.length); // 2

let others = users.filter((item) => item.id > 5);
console.log(others.length); // 0
```

- arr.map(): thực hiện một hàm trên mỗi phần tử của mảng và trả về một mảng các kết quả

```
let result = arr.map(function (item, index, array) {
  // trả về giá trị mới từ mỗi item
});
```

  Ví dụ:

```
let lengths = ["Dog", "Fish", "Elephant"].map((item) => item.length);
console.log(lengths); // (3) [3, 4, 8]
```

- arr.sort(): sắp xếp các phần tử trong mảng theo thứ tự, mặc định các phần tử được sắp xếp theo thứ tự string.

```
function compare(a, b) {
  if (a > b) return 1; // a đứng sau b
  if (a == b) return 0; // a, b bằng nhau
  if (a < b) return -1; // a đứng trước b
}

let arr = [1, 2, 15];
arr.sort(function (a, b) {  if (a > b) return 1; // a đứng sau b  if (a
== b) return 0; // a, b bằng nhau  if (a < b) return -1; // a đứng trước
b});
console.log(arr); // (3) [1, 2, 15]
```

- arr.reverse(): giúp đảo ngược mảng gốc
- split() và join()
  - Phương thức arr.split(delim) giúp tách string thành một mảng với giá trị dùng để phân tách là delim.

- Phương thức arr.join xử lý ngược lại với phương thức arr.split. Phương thức này trả về một string bằng cách ghép các phần tử mảng với "một kí tự kết nối".

- Reduce() và reduceRight(): dùng để tính toán và trả về một giá trị duy nhất từ các phần tử mảng, reduceRight() duyệt từ phải sang trái

```
let arr = [1, 2, 3, 4, 5];

// giá trị khởi tạo không truyền nên mặc định là 0
// tại mỗi lần duyệt, giá trị tích lũy được cộng với giá trị phần tử hiện tại
let result = arr.reduce((sum, current) => sum + current);

console.log(result); // 15
```

- So sánh mảng: tương tự với so sánh 2 object

5.8.5. Kiểu Date

- Date trong JavaScript được biểu diễn bởi đối tượng Date, trong đó:
  - Tháng đếm từ 0 (tháng 1) đến 11 (tháng 12).
  - Ngày trong tuần đếm từ 0 (chủ nhật) đến 6 (thứ 7).
- Đối tượng Date có tính năng tự động điều chỉnh tăng hay giảm nếu giá trị truyền vào vượt quá phạm vi biểu diễn của ngày, tháng, năm,... Tính năng này hữu ích khi muốn cộng/trừ với ngày/tháng/giờ,...
- Ngoài ra, có thể trừ hai giá trị date cho nhau, kết quả trả về là số mili giây chênh lệch. Bởi vì, Date trở thành timestamp khi chuyển thành number.
- Phương thức Date.now() trả về timestamp hiện tại mà không cần tạo mới đối tượng Date.
- Khởi tạo Date:

```
// Cách 1
let currentDate = new Date();
console.log(currentDate);

// Cách 2
// 0 tương ứng với ngày 01/01/1970 UTC+0
let jan01_1970 = new Date(0);
console.log(jan01_1970); // Thu Jan 01 1970 07:00:00 GMT+0700 (Indochina Time)
// cộng thêm 24 giờ => ngày 02/01/1970 UTC+0
let jan02_1970 = new Date(24 * 3600 * 1000);
console.log(jan02_1970); // Fri Jan 02 1970 07:00:00 GMT+0700 (Indochina Time)

// cách3
let date = new Date("2021-12-19");
console.log(date); // Sun Dec 19 2021 07:00:00 GMT+0700 (Indochina Time)

// Thời gian không được cài đặt, nên mặc định đó là 00:00:00 tại GMT+0000,
// hoặc 07:00:00 tại GMT+0700.

// Cách 4
new Date(2021, 0, 1, 0, 0, 0, 0); // 1 Jan 2021, 00:00:00
new Date(2021, 0, 1); // giống câu lệnh trên
```

- Truy cập các phần tử của Date()
  - Phương thức getFullYear(): trả về năm (gồm 4 chữ số).
  - Phương thức getMonth(): trả về tháng (từ 0 đến 11).
  - Phương thức getDate(): trả về ngày trong tháng (từ 1 đến 31).
  - Phương thức getHours(): trả về giờ.
  - Phương thức getMinutes(): trả về phút.
  - Phương thức getSeconds(): trả về giây.
  - Phương thức getMilliseconds(): trả về mili giây.
- Thay đổi các phần tử của Date()
- Phương thức setFullYear(0
- Phương thức setMonth()
- Phương thức setDate()
- Phương thức setHours()
- Phương thức setMinutes()
- Phương thức setSeconds()
- Phương thức setMilliseconds()

5.9. Biểu thức qui tắc

5.9.1. Khái niệm

- Biểu thức qui tắc (Regular expression - RegExp) là một cách để biểu diễn khuôn mẫu của string.

5.9.2. Khởi tạo biểu thức qui tắc

- Cách 1: Sử dụng hàm khởi tạo của đối tượng RegExp
- Cách 2: Viết trực tiếp sử dụng cặp dấu "/ /"

```
var re1 = new RegExp("abc");
var re2 = /a-z/;
```

- Cách 2: tạo đối tượng RegExp một cách linh động

```
var name = "lampv";
var text = "LamPV is a suspicious character.";
var regexp = new RegExp("\\b(" + name + ")\\b", "gi");
console.log(text.replace(regexp, "_$1_"));
// => _LamPV_ is a suspicious character.
```

Trong đó:

- tham số g: chỉ ra rằng khuôn mẫu áp dụng Global, nghĩa là nó áp dụng cho tất cả thành phần trùng khớp.
- tham số i: chỉ ra rằng khuôn mẫu không phân biệt chữ hoa và chữ thường.

5.9.3. Phương thức của biểu thức qui tắc

- Test(): kiểm tra xem một string có chứa khuôn mẫu đã định nghĩa hay không. Nếu có thì kết quả trả về là true và ngược lại thì là false.
  - JS sử dụng cặp dấu ngoặc vuông [] để biểu thị việc kiểm tra string có chứa bất kì kí tự nào có trong cặp dấu [] hay không. Trong đó, dấu "-" giữa hai kí tự dùng để chỉ 1 khoảng giữa hai kí tự đó.
    - \[0-9\] là các chữ số từ 0 đến 9

    - \[a-z\] là các chữ cái từ a đến z
  - Ngoài ra, JS cũng cung cấp sẵn một số cách biểu diễn một tập hợp các kí tự:
    - \\d : bất kì chữ số nào từ 0 đến 9
    - \\w : một chữ cái
    - \\s : kí tự trắng (dấu cách, tab, dòng mới,...)
    - \\D : kí tự không phải số
    - \\W : kí tự không phải chữ cái
    - \\S : kí tự không phải kí tự trắng
    - . : bất kì kí tự nào trừ dòng mới.

```
// kiểm tra ngày giờ có định dạng: dd-mm-yyyy hh:mm
var dateTime = /\d\d-\d\d-\d\d\d\d \d\d:\d\d/;
console.log(dateTime.test("30-01-2003 15:20"));
// => true
console.log(dateTime.test("30-jan-2003 15:20"));
// => false
```

- Lặp lại khuôn mẫu
  - "+": biểu thị phần tử xuất hiện >= 1 lần
  - "*": biểu thị phần tử xuất hiện >= 0 lần (có thể không xuất hiện)
  - "?": biểu thị phần tử xuất hiện 0 hoặc 1 lần
  - {n}: biểu thị phần tử xuất hiện đúng n lần
  - {x,y}: biểu thị phần tử xuất hiện từ x đến y lần
  - {x, }: biểu thị phần tử xuất hiện >= x lần
  - Muốn lặp lại cả một nhóm các phần tử. Khi đó, phải nhóm các phần tử đó lại sử dụng cặp dấu ngoặc đơn "( )".
- Exec(): trả về một đối tượng chứa thông tin thành phần trùng khớp với khuôn mẫu, ngược lại thì trả về null.
- Trích xuất dữ liệu từ chuỗi

```
function findDate(string) {
  var dateTime = /(\d{1,2})-(\d{1,2})-(\d{4})/;
  var match = dateTime.exec(string);
  return {
    day: match[1],
    month: match[2],
    year: match[3],
  };
}
var obj = findDate("30-1-2003");
console.log(obj);
// => Object {day: "30", month: "1", year: "2003"}

// Cách 2
function findDate(string) {
  var dateTime = /^(\d{1,2})-(\d{1,2})-(\d{4})$/;
  var match = dateTime.exec(string);
  if (!match) return null;
  return {
    day: match[1],
    month: match[2],
    year: match[3],
  };
```

```
}
var obj1 = findDate("30-1-2003");
console.log(obj1);
// => Object {day: "30", month: "1", year: "2003"}

var obj2 = findDate("0030-1-200300");
console.log(obj2);
// => null

var obj3 = findDate("Hello 30-1-2003 Haha");
console.log(obj3);
// => null

// Cách 3
function findDate(string) {
  var dateTime = /\b(\d{1,2})-(\d{1,2})-(\d{4})\b/;
  var match = dateTime.exec(string);
  if (!match) return null;
  return {
    day: match[1],
    month: match[2],
    year: match[3],
  };
}
var obj1 = findDate("30-1-2003");
console.log(obj1);
// => Object {day: "30", month: "1", year: "2003"}

var obj2 = findDate("0030-1-200300");
console.log(obj2);
// => null

var obj3 = findDate("Hello 30-1-2003 Haha");
console.log(obj3);
// => Object {day: "30", month: "1", year: "2003"}
```

5.9.4. Một số khuôn mẫu cơ bản thường dùng

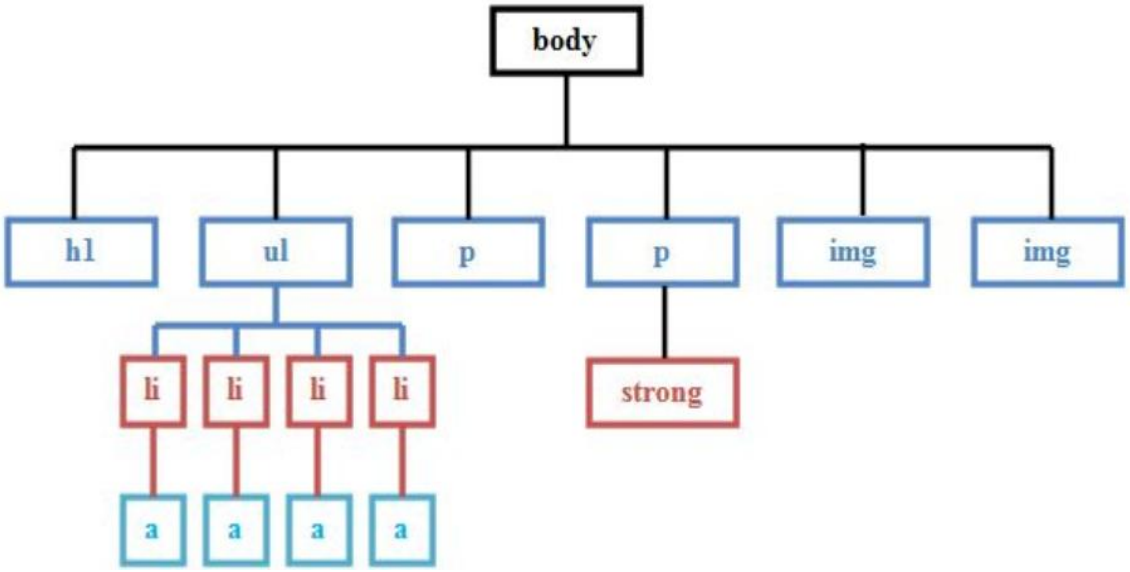
- /abc/ : chuỗi các kí tự abc
- /\[abc\]: bất kỳ kí tự nào thuộc tập hợp a, b, c
- /\[^abc\]: bất kỳ kí tự nào không thuộc tập a, b, c
- /\[0-9\]/: bất kỳ kí tự nào thuộc đoạn từ 0 đến 9
- /x+/: thành phần x xuất hiện >= 1 lần
- /x*/: thành phần x xuất hiện >= 0 lần (có thể không xuất hiện)
- /x?/: thành phần x xuất hiện 0 hoặc 1 lần
- /x{2, 4}/: thành phần x xuất hiện từ 2 đến 4 lần
- /(abc)/: cụm abc
- /a|b|c/: bất kì pattern nào trong 3 loại a, b, c
- /\\d/: chữ số từ 0 đến 9
- /\\w/: chữ cái
- /\\s/: kí tự trắng (dấu cách, tab, dòng mới,...)
- /./ : bất kỳ kí tự nào trừ dòng mới
- /\\b/: ranh giới từ
- /^/: bắt đầu string

- /$/: kết thúc string

5.10. Mô hình đối tượng tài liệu DOM

5.10.1. Khái niệm

- DOM là viết tắt của Document Object Model - một thành phần javascript cơ bản. DOM trong JavaScript chính là những phần tử biểu diễn cấu trúc dữ liệu của một trang web.
- Cấu trúc biểu diễn DOM
  - Cấu trúc dữ liệu của một trang web được biểu diễn dạng cây (tree). Trong đó, mỗi node sẽ tương ứng với một DOM.
  - Mỗi DOM node sẽ có thuộc tính nodeType - là một số nguyên - để biểu diễn kiểu của node



Hình 5.1: Các phần tử trong tài liệu phân cấp theo mô hình DOM

```
<body>
   <h1>Tiêu đề 1</h1>
   <ul id="wrapper">
      <li><a href="#">HTML</a></li>
      <li><a href="#">CSS</a></li>
      <li><a href="#">Javascript</a></li>
      <li><a href="#">JQuery</a></li>
    </ul>
    <p class="item">Đoạn 1</p>
    <p><strong>Đoạn 2</p>
    <img src="image1.jpg" alt="tree"/>
    <img src="image2.jpg" alt="plant"/>
 </body>
```

Tài liệu HTML như 1 "cây phả hệ" (document tree) thì body chính là gốc của "cây phả hệ" đó. Các thẻ h1, ul, 2 thẻ p và 2 thẻ img được bao bởi thẻ body chính là "con" của gốc body. Thẻ ul chứa trong nó 4 thẻ li, các thẻ li này là "con" của thẻ ul và là "cháu" của gốc body trên "cây phả hệ".

5.10.2. Liên kết giữa các node

- Node.parentNode: trỏ đến node cha của một node, nếu một node không có node cha (document) thì parentNode là null.
- Node.childNodes: trỏ đến một đối tượng gần giống với array, chứa những node con của một node.
- Node.firstChild: trỏ đến node con đầu tiên của một node trong childNodes.
- Node.lastChild: trỏ đến node con cuối cùng của một node trong childNodes.
- Node.previousSibling: trỏ đến node liền kề phía trước (có cùng parentNode).
- Node.nextSibling: trỏ đến node liền kề phía sau (có cùng parentNode).

5.10.3. Một số phương thức trong DOM

- Node.getElementsByTagName('tagName'): trả về một danh sách các node con, cháu của một node với tag là 'tagName'.
- Node.getElementsByClassName('className'): Tương tự như phương thức trên, phương phức này trả về một danh sách các node con, cháu của một node với thuộc tính class = 'className'.
- Node.getElementById('id'): trả về node có thuộc tính id = 'id'.
- Node.removeChild(child): bỏ đi node child của một node và trả về node bị bỏ đi.
- Node.appendChild(child): thêm node child vào cuối danh sách childNodes của một node.
- Node.insertBefore(newNode, referenceNode): chèn thêm node newNode vào phía trước node referenceNode.
- Node.replaceChild(newChild, oldChild): thay thế node oldChild bằng node newChild.
- Document.createTextNode(data): Tạo ra một node kiểu text với giá trị text là data.
- Document.createElement(tagName): Tạo ra một node mới có kiểu là tagName (h1, h2, p, img, div,...)
- Document.querySelector(selectors): Trả về node đầu tiên thoả mãn selectors. Trong đó, selectors là một string chứa một hay nhiều CSS-selector phân cách nhau bằng dấu phẩy.
- Document.querySelectorAll(selectors): Trả về danh sách các node thoả mãn selectors, theo thứ tự sử dụng thuật toán DFS.

5.10.4. Sự kiện trong DOM

- Đăng kí sự kiện: Node.addEventListener('tên sự kiện', hàm xử lý);
  Trong đó:
  - Node: là một phần tử DOM như: document, body, h1, p, img, button,... Nếu không ghi gì thì mặc node đó là document.
  - Tên sự kiện: ứng với sự kiện muốn nhận như: click, mousedown, mouseup, mousemove, keydown, keyup, keypress,...
  - Hàm xử lý: là hàm được gọi khi sự kiện đăng ký xảy với node trên.
- Hủy đăng kí sự kiện: Node.removeEventListener('tên sự kiện', hàm xử lý);

- Đăng kí sự kiện cho nhiều node dựa trên đối tượng sự kiện: sử dụng phương thức addEventListener 1 lần và sử dụng thuộc tính target của đối tượng event để phân biệt node được gọi.

```
<button>A</button>
<button>B</button>
<button>C</button>
<p>Hello</p>
<script>
  document.body.addEventListener("click", function (event) {
    if (event.target.nodeName == "BUTTON")
      console.log("Clicked", event.target.textContent);
    else if (event.target.nodeName == "P")
      console.log("Paragraph clicked", event.target.textContent);
  });
</script>
```

- Hàm hủy thực hiện mặc định: preventDefault()
  Một số Event JavaScript có sẵn hàm thực hiện mặc định. Ví dụ: khi bạn click vào một link thì bạn sẽ được chuyển hướng đến trang của link đó, hay khi bạn click chuột phải thì sẽ có một context menu hiện ra,...

```
<a href="https://developer.mozilla.org/">MDN</a>
<script>
  var link = document.querySelector("a");
  link.addEventListener("click", function (event) {
    console.log("Nope.");
    event.preventDefault();
  });
</script>
```

- Một số sự kiện với form, bàn phím, chuột: xem trong phần form của chương 3

5.10.5. Chuyển HTML String sang DOM

- Bước 1: Tạo mới một đối tượng của DOMParser.
- Bước 2: Sử dụng phương thức parseFromString của DOMParser.

```
let domParser = new DOMParser();
let doc = domParser.parseFromString("<div>Hello</div>", "text/html");
console.log(doc);
```

  - Trong hàm parseFromString, tham số đầu tiên là nội dung HTML string mà bạn muốn parse; và tham số tiếp theo là kiểu string.
  - Có 3 kiểu string được support là: application/xml, image/svg+xml và text/html.
- Bước 3: Lấy ra mảng các node mới được tạo ra

```
let nodes = document.body.chidNodes;
console.log(nodes);
```

5.10.6. Chuyển HTML Template sang DOM

- Trước khi sử dụng template, bạn cần kiểm tra xem trình duyệt của mình có hỗ trợ template hay không.

```
if ("content" in document.createElement("template")) {
  console.log("Support Template");
} else {
  console.log("Not Support Template");
}
```

Template có một thuộc tính quan trọng là content. Khi bạn tạo mới một template sử dụng createElement, thì phần tử đó phải tồn tại thuộc tính content. Nghĩa là trình duyệt có hỗ trợ template, ngược lại thì không hỗ trợ.

- Lấy nội dung của HTML Template: sử dụng phương thức querySelector (hoặc getElementById) để lấy ra phần tử DOM của template đó, đồng thời trích xuất ra content - dạng DocumentFragment.

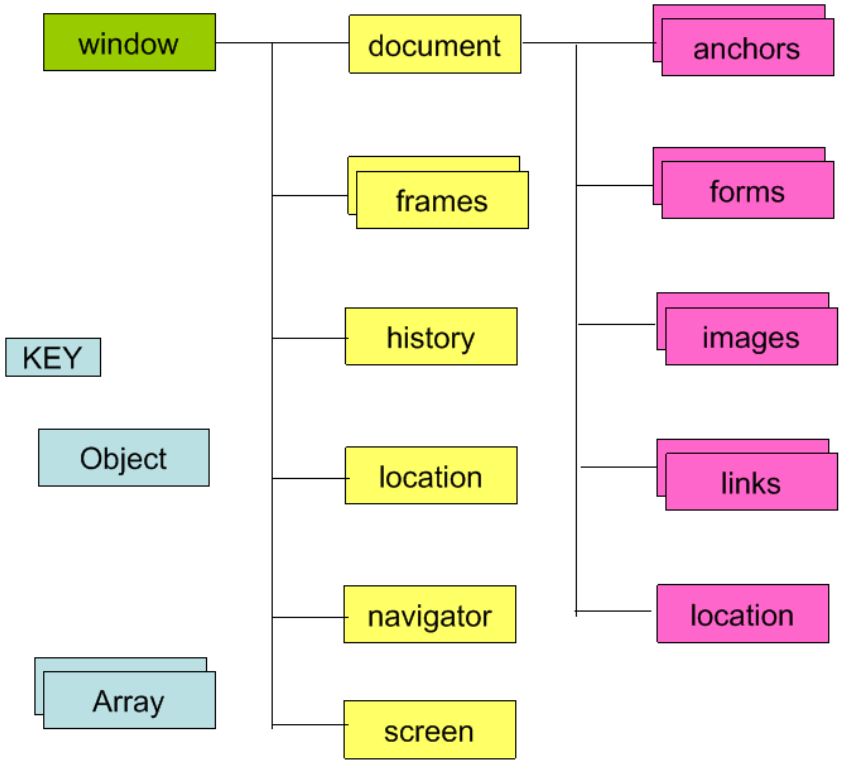
```
let templateFrag = document.querySelector("#my-temp").content;
```

- Render HTML Template lên giao diện
  - Nội dung các bình luận là một mảng của các object - bao gồm 2 thuộc tính là name và comment.
  - Khi người dùng nhấn vào button Add, mình sẽ sử dụng phương thức forEach để duyệt mảng trên. Với mỗi phần tử của mảng, clone lại template DocumentFragment trên để dùng lại sau này.
  - Tiếp theo, để apply nội dung của bình luận vào template, có thể tiếp tục sử dụng phương thức querySelector để thay đổi nội dung trong span với class là name, content tương ứng với nội dung trong mảng, rồi thêm nó vào bên trong thẻ div với id="container".
  - Cuối cùng, để xóa đi các thành phần DOM mới thêm vào, có thể sử dụng vòng lặp, liên tục xóa đi phần tử con đầu tiên của container, cho đến khi container không còn node con nào nữa là hoàn thành nhiệm vụ.

```
let data = [
  { name: "John", comment: "That is great" },
  { name: "Alex", comment: "It's helpful" },
  { name: "David", comment: "Thanks a lot" },
];
btnAdd.addEventListener("click", () => {
  data.forEach((item) => {
    let tmpl = templateFrag.cloneNode(true);
    tmpl.querySelector(".name").innerText = item.name;
    tmpl.querySelector(".comment").innerText = item.comment;
    container.appendChild(tmpl);
  });
});
btnRm.addEventListener("click", () => {
  while (container.firstChild) {
    delete container.removeChild(container.firstChild);
  }
});
```

5.11. Mô hình đối tượng trình duyệt BOM

- BOM là 1 hệ thống phân cấp hình cây gồm các đối tượng: Window (trình duyệt), History, Location, Navigator, Screen, Document, Frames



Hình 5.2: Mô hình đối tượng trình duyệt BOM

- Các đối tượng trong BOM
  - Window là đối tượng thể hiện cửa sổ hiển thị hiện tại trên trình duyệt
    - Các thuộc tính và phương thức có thể gọi trực tiếp hoặc thông qua window
    - VD: alert("Hello"); hoặc window.alert("Hello");
    - Thuộc tín: closed, defaultStatus, opener, status, innerHeight, document
    - Phương thức: alert(); prompt(); confirm(); focus(); blur(); close(); open(); printf(); moveTo(); resizeTo();
  - Screen là đối tượng dùng để lấy thông tin về màn hình của người truy cập
    - Thuộc tính: avaiHeight, avaiWidth, height, width, pixelDepth, colorDepth
  - Navigator là đối tượng cung cấp thông tin về trình duyệt
    - Thuộc tính: appcodeName, appName, version, cookieEnabled, platform
    - Phương thức: javaEnabled()
  - Location là đối tượng chứa thông tin URL hiện tại
    - Thuộc tính: host, hostname, href, pathname, port, protocol
    - Phương thức: assign(), reload()
  - History là đối tượng chứa thông tin URL của người dùng truy cập
    - Thuộc tính: length
    - Phương thức: back(); forward(); go();

BÀI TẬP CHƯƠNG 5

Bài tập 1

Tạo object rỗng có tên là point theo yêu cầu sau:

a. Thêm thuộc tính x = 5, thuộc tính y = 10, thuộc tính z-index = 20
b. Tính tổng các thuộc tính
c. Thay đổi thuộc tính x = 2
d. Xóa thuộc tính y khỏi point

HD: Sử dụng object (phần 5.7)

Bài tập 2

Viết hàm kiểm tra xem một số có phải là số nguyên tố hay không. Nếu đúng thì trả về true, ngược lại trả về false.

a. Sử dụng hàm thông thường (function declaration)
b. Sử dụng hàm biểu thức (function expression)
c. Sử dụng hàm mũi tên (arrow function)

HD: Sử dụng hàm (phần 5.6)

Bài tập 3

Viết hàm upperFirst(str) nhận vào tham số là string str và trả về string mới với kí tự đầu tiên được viết hoa.

HD: Sử dụng string

Bài tập 4

Viết hàm sumInput thực hiện các yêu cầu sau:

a. Sử dụng hàm prompt để yêu cầu người dùng nhập vào các số và lưu vào một mảng.
b. Dừng hỏi người dùng khi giá trị nhập vào không phải là số bình thường.
c. Tính và in ra tổng các số người dùng nhập.

Bài tập 5

Viết các hàm sau:

a. Đảo ngược chuỗi
b. Tìm độ dài của từ dài nhất trong câu

HD: Sử dụng String.prototype.split(), String.length, Array.prototype.reduce()

Bài tập 6

Viết hàm lặp lại một chuỗi với số lần cho trước

Bài tập 7

Viết chương trình kiểm tra một chuỗi có phải là palindrome hay không (Một chuỗi được gọi là palindrome khi và chỉ khi đọc ngược và đọc xuôi đều giống nhau, với điều kiện bỏ qua các dấu câu, kí tự đặc biệt, dấu cách; ví dụ eye, boob,…)

HD: Sử dụng biểu thức qui tắc

Bài tập 8

Viết chương trình trả về một mảng gồm các số lớn nhất của mỗi mảng con

HD: Sử dụng toán tử so sánh, Array.prototype.push()

Bài tập 9

Viết chương trình tạo slide hình ảnh

Bài tập 10

Viết chương trình hiển thị trên màn hình những thông tin sau:

a. Màn hình đang sử dụng để truy cập trang web
b. Trình duyệt đang sử dụng để truy cập web
c. URL của trang hiện tại

TÀI LIỆU THAM KHẢO

- Giáo trình
  - Nhóm tác giả Elicom (2000). *Cẩm nang thiết kế website*, NXB Hà Nội
  - Nhóm tác giả Elicom (2000). *Tự học Adobe Photoshop 5.5*, NXB Hà Nội
- Địa chỉ website
  - https://css-tricks.com/
  - http://www.w3schools.com/
  - https://webfieldmanual.com/
  - https://developer.mozilla.org/en-US/docs/Web